

# 基于分位数预测与滚动线性规划的 微网购电与储能协同调度

## 摘 要

对于含光伏和储能的小区微网购电调度问题，本文以 10 分钟为基本时段，建立统一的能量平衡、储能运行和合同执行模型。根据负荷、光伏及电价信息的可获得程度，分别研究确定性调度、日前滚动签约、日内合同调整和波动电价下的重新调度。

针对问题一，在负荷、光伏和电价已知的条件下，以购电费用最小为目标建立周期线性规划模型，得到典型日购电与储能充放电方案，总费用为 35126.85 元。

针对问题二，考虑普通购电过量与紧急购电不足造成的非对称损失，采用净负荷分位数预测，并结合三日前瞻规划和库存保护规则滚动签订合同。通过一月 45 组候选方案回放，选定浅层分位数树、0.8 控制分位数和 0 库存保护系数，2 月至 12 月总费用为 14046079.80 元。

针对问题三，先将整点光伏预报插值至 10 分钟尺度，再利用历史残差修正预测结果；将负载分位数预测减去光伏预测电量，构造规划净需电量，并通过线性规划调整尚未执行的合同。一月 1800 种策略的比较结果选出浅层分位数树、0.65 控制分位数、0.5 库存保护系数、0.75 光伏收缩系数以及 6:00、12:00 和 18:00 三个调整节点。正式期总费用为 13365464.69 元，较仅在 0:00 签约减少 1130235.69 元。

针对问题四，以最近同槽均价和残差学习模型预测波动电价，规划时使用预测电价，结算时使用真实电价。重新选型后，日前滚动方案和日内调整方案的正式期费用分别为 14666963.63 元和 14050673.91 元。

本文的创新点在于明确区分合同发布与生效时刻、规划库存与实际库存，并将分位数预测、三日前瞻规划、库存保护和日内合同调整纳入统一框架；同时采用一月选型、正式期冻结评价的方式，避免根据后验结果重新选择策略。

**关键词：**微网调度；储能控制；分位数预测；滚动优化；光伏预测；波动电价

## 一、问题重述

某小区微网由用户负荷、光伏发电、外网购电和储能设备构成<sup>[1]</sup>，调度以 10 分钟为基本时段。普通购电需提前签订计划；运行中若出现供电缺口，可通过紧急购电补足，其单位电价为普通购电的 5 倍。储能运行受容量、充放电功率及效率约束。负荷和储能均无法消纳的剩余电量不得向外部电网出售。

四个问题共用同一套微网运行与储能模型，主要区别在于决策时可获得的信息和购电计划的调整方式。问题一在负荷、光伏出力和电价均提前已知的条件下，求解典型日的购电与储能调度方案。问题二仅能利用历史数据预测未来净负荷，并据此逐日滚动签订购电合同。问题三在问题二的基础上引入日内信息更新。每天 0:00、6:00、12:00 和 18:00 发布未来 24 小时的整点光伏预测，调度据此更新后续供需判断，并调整尚未执行的购电合同。问题四则将前三问中的固定日内电价替换为波动电价，在新的价格条件下分别重新求解问题二和问题三。

因此，本文首先建立统一的微网能量平衡与储能约束模型，并以问题一的完全信息优化作为基准。在此基础上，问题二引入预测误差和跨日滚动决策，问题三进一步加入日内信息更新与合同修正机制，问题四则考察波动电价对上述两种调度策略的影响。上述安排在保持设备模型一致的基础上，逐步考察预测信息、更新频率和电价变化对调度结果的影响。

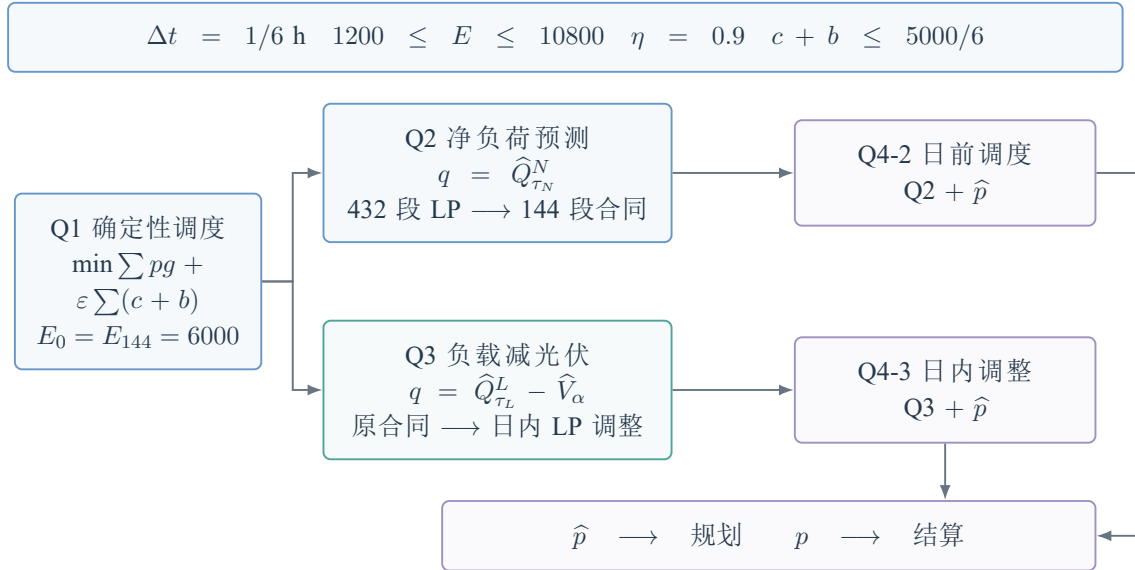


图 1 四问统一建模框架

## 二、问题分析

## 2.1 时间与合同边界

原始数据的时间标签间隔为 10 分钟。本文将标签视为对应区间的起点。例如，日期行中的 00:10 对应当天 00:10 至 00:20，最后一列对应次日 00:00 至 00:10。

合同在节点时刻发布，从下一个 10 分钟区间开始生效。正在执行的区间继续使用原合同。因此，0:00 发布的合同实际覆盖当天 00:10 至次日 00:10，0:00 至 00:10 这一交接区间仍沿用上一份合同。这一处理保证了合同调整与实际执行的时序一致。若当前区间立即采用新合同，就会提前使用尚未生效的计划，并将当前节点获得的信息用于已经开始的区间，造成信息泄漏。

## 2.2 预测目标与经济损失

净负荷预测误差造成的经济损失并不对称。若普通购电计划偏高，会增加合同购电支出，多余电量还可能因负荷和储能无法消纳而成为未利用电量；若计划偏低，则需要通过紧急购电补足缺口，而紧急购电价格达到普通购电价格的 5 倍。

因此，在问题二和问题三中，预测目标采用条件分位数<sup>[2]</sup>，而非单纯预测条件均值。较高的分位数有助于降低供电缺口概率，但也可能增加过量购电。其经济效果还取决于电价水平、储能余量和光伏富余程度。因此，控制分位数最终依据实际运行费用选取，不能仅凭预测误差确定。

## 2.3 日内调整与节点选择

问题三提供了日内更新的光伏预测，使系统能够利用最新信息重新判断后续供需状态<sup>[3]</sup>。合同增购或调减均有相应成本，因此增加调整频次未必能降低总费用。

因此，模型将允许调整的节点组合作为策略变量，不预先要求使用全部日内节点。节点选择与控制分位数、库存保护系数、光伏收缩系数等参数共同进入历史数据回放，并以最终运行费用作为统一评价标准。由此比较不同调整频率带来的预测收益与合同修改成本。

## 2.4 波动电价处理

问题四引入波动电价后，规划阶段还需要严格区分预测电价与实际电价。决策节点只能使用当时可得的电价预测，未来真实电价不得提前进入优化模型。真实电价仅在对应该区间结束后用于费用结算，并在信息成熟后作为后续价格预测模型的训练标签。

Q4-2 沿用净负荷预测框架，并加入电价预测；Q4-3 继承 Q3 以负载分位数减去光伏预测的构造方式及日内调整机制。两条分支分别继承前问的数量信息、初始化、真实交接库存和数量模型拟合频率，并在波动电价条件下分别利用一月数据选择参数。

### 三、模型假设

为统一描述各问的调度过程，并明确预测与执行的信息边界，本文作如下假设。

1. 原始数据仅给出每 10 分钟节点处的负荷、光伏出力和电价。本文采用区间左端点分段常值近似，即对任意区间  $[t_k, t_{k+1})$ ，相应物理量在区间内均取左端点观测值，即

$$\hat{p}(t) = p(t_k), \quad t \in [t_k, t_{k+1}).$$

上式及示意图中的  $p(t)$  泛指所讨论的采样物理量，下文带离散下标的  $p_k$  专指电价。因此，每个 10 分钟区间内的功率均按该节点值计算，相应电量由功率乘以 1/6 小时得到。附件未提供区间内部的连续观测，因此该离散化存在近似误差。各问均采用相同处理方式。

2. 普通购电量非负，且不额外设置购电容量上限。微网不向外部电网反向售电，当光伏发电、购电及储能放电的总量超过负荷与储能的可消纳量时，超出部分记为未利用电量。
3. 储能系统的充电效率和放电效率均取 0.9。研究期内储能容量、最大充电功率和最大放电功率保持不变，不考虑设备老化造成的参数变化。
4. 在合同调整节点发布的新购电计划从下一 10 分钟区间开始执行。已经完成的区间以及节点所在的交接区间均沿用原合同，不允许追溯修改。
5. 在 Q2、Q3 及其 Q4 继承分支中，补充假设当前全段数量在段初可知。当前 10 分钟区间的实际负荷和光伏量用于该区间的运行结算，再以真实期末库存规划下一阶段。由此，新一轮优化从下一 10 分钟区间开始生效。
6. Q4 完整继承前问的数量信息与库存衔接规则。新增电价模型只使用已完成区间的价格历史；当前及未来区间尚未结算的真实电价不进入规划，仅在区间结束后用于结算，随后作为成熟训练标签。
7. 为将候选筛选与最终评价分开，一月数据用于筛选模型及策略参数，二月至十二月数据用于冻结方案后的正式评价。正式评价期间，可用逐步成熟的新数据重新拟合同类预测模型；模型类型、控制分位数及其他调度参数保持不变。

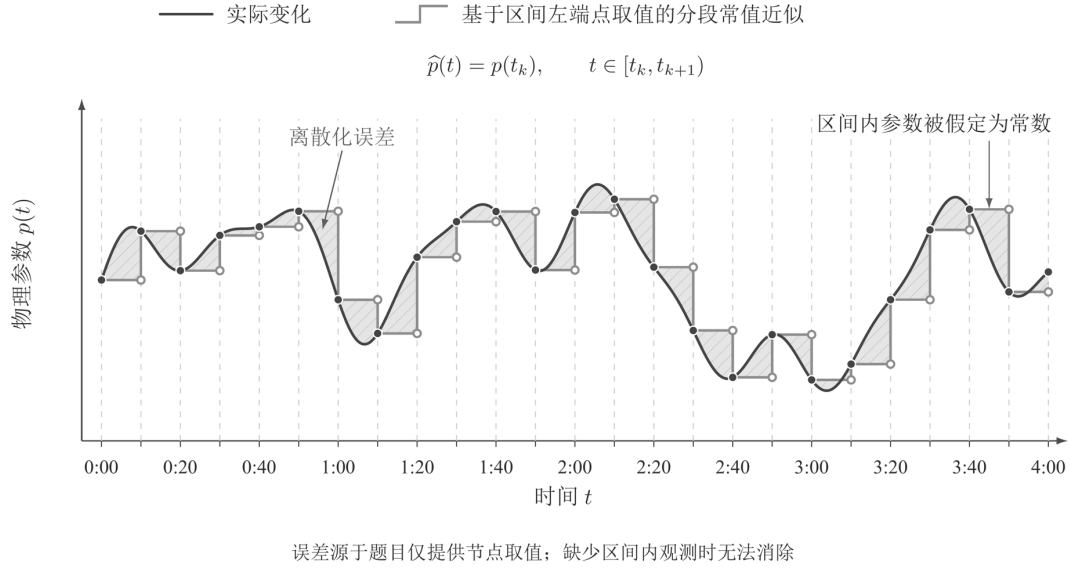


图2 十分钟离散化及区间左端点分段常值近似示意图

## 四、符号说明

每个时段的长度为  $\Delta t = 1/6 \text{ h}$ ； $L_k, V_k, N_k$  分别为负载、光伏和净负荷电量。 $p_k$  为真实结算电价， $\hat{p}_{i|r}$  为节点  $r$  对未来第  $i$  段的预测电价， $\tilde{p}_{i|r}$  为实际进入规划的电价，单位均为元/kWh。

$r$  始终表示包含日期的发布时间， $k_r$  为该时刻所在交接段的全局索引， $\nu(r)$  为其日内小时数。规划使用局部索引  $i = 1, \dots, 432$ ，对应全局段  $k = k_r + i$ ； $\mathcal{H}_r = \{1, \dots, 432\}$  表示节点  $r$  之后从下一段起的三日前瞻窗口。在单次规划中，省略部分变量的条件下标  $r$ ，并简记  $p_i = p_{k_r+i}$ ；Q1 的  $i$  则按该典型日的 144 段编号。时序图按物理时刻标记的量满足  $E(r) = E_{k_r}$ 、 $N_r = N_{k_r}$ ，图中 old、new 分别表示更新前后的合同。

$g_i$  为规划普通购电量， $G_k^0, G_k^{eff}$  为原合同量和最终生效合同量，执行公式简记  $G_k = G_k^{eff}$ ，相应规划原合同为  $g_i^0 = G_{k_r+i}^0$ 。 $c_i, b_i, u_i, \hat{h}_i$  为规划充电、放电、未利用及紧急购电量， $C_k, B_k, U_k, h_k$  为对应实际执行电量； $e_i, E_k, z_k, R_k$  分别为规划段末库存、实际段初库存、执行层参考段末库存和放电保护线。上述电量及库存单位均为 kWh； $q_{i|r}$  为进入规划的预测净需电量，单次规划简写  $q_i$ 。

$E^{min} = 1200 \text{ kWh}$ 、 $E^{max} = 10800 \text{ kWh}$ ， $\eta = 0.9$  为充、放电效率， $D = 5000/6 \text{ kWh}$  为单段充放电共用上限。 $a_i$  为合同费用辅助变量，单位元，与参考库存  $z_k$  区分； $\varepsilon = 10^{-6}$  元/kWh 为吞吐正则系数， $\lambda_r$  及  $\lambda_{Jan}$  为单位库存估值，单位元/kWh。 $(x)_+ = \max\{x, 0\}$  为正部算子。

$\tau_N, \tau_L$  为净负荷和负载的控制分位数， $\beta$  为库存保护系数， $\alpha$  为光伏残差修正收缩系数； $S$  为允许日内合同调整的小时数组成的集合。 $\mathcal{K}_{Jan}$  表示一月评分时段集合， $E_{end}$  为相应评分期末实际库存， $C_{\theta, S}^{Jan}$  为候选  $(\theta, S)$  的实际合同及紧急购电费用之和。

## 五、数据处理与基本分析

### 5.1 数据结构与时间对齐

附件 1 包含 144 个未标注具体日期的十分钟观测，用于 Q1；其日内电价按时槽映射至 Q2、Q3，不另行补设日期。附件 2 的负载、实际光伏及附件 4 的波动电价均为 365 行、144 个时槽的宽表，各含 52560 个观测；展开为长序列后，按物理区间起点对齐，并保留原始日期和标签。按本文左端点口径，日期  $d$  行从当天 00:10 覆盖至次日 00:10，末列 0:00+1 对应次日 00:00，因此该行不能直接视为一个自然日。附件 5 仅为输出模板，不参与训练或缺失率统计。

附件 3 每天在 00:00、06:00、12:00、18:00 发布未来 1–24 小时的整点光伏预报，共 1460 个版本、35040 个值。仅将日期列的 1095 个省略标签向下填充，预测数值不作填补。逐字段核对表明，JSON 与原始 Excel 一致，转换仅涉及时间序列化和上述日期补全。以  $(r, h)$  为唯一键保存发布时间  $r$ 、提前期  $h$  和目标时刻  $t_{r,h} = r + h \text{ h}$ ；同一目标时刻的不同发布版本分别保留，降尺度在各版本内部完成。

### 5.2 质量检查与电量换算

数值区未发现缺失值或非有限值。全年实际序列的时间戳唯一，相邻间隔均为 10 分钟。实际光伏中的 23540 个零值具有昼夜物理含义，予以保留；对峰值和谷值不作统一删除、缩尾或平滑处理。对区间  $I_k = [t_k, t_k + \Delta t)$ ，按左端点功率常值换算电量，即

$$\Delta t = \frac{1}{6} \text{ h}, \quad L_k = P_k^L \Delta t, \quad V_k = P_k^{PV} \Delta t, \quad N_k = L_k - V_k.$$

保留  $N_k$  的负值，由调度模型决定富余光伏的储存和未利用量。负载、光伏功率及电价的全年均值分别为 4626.03 kW、2311.79 kW 和 0.7662 元/kWh；10130 个时段光伏高于负载，占 19.27%，表明储能对富余光伏的吸收能力不可忽略。

### 5.3 周期结构与经验波动范围

图 3 按月对同一时槽取均值。负载呈现稳定的日内节律，夏季整体升高；光伏发电集中于白天，月均值在 8 月最高、12 月最低；电价兼有日内峰谷和月度差异。因此，三类序列均不宜作为平稳独立样本处理，预测中应保留其时间结构。

图 4 以同一时槽的中位数和 5%–95% 经验分位数描述日内曲线，保留偏态及光伏零值，无须正态分布假设。附件 1 的负载、光伏预测和电价各 144 个值均落在对应经验区间内，表明其幅度具有样本内典型性。由于未给定日期，这一结果不能说明其与某个季节匹配，也不能说明光伏预测准确。

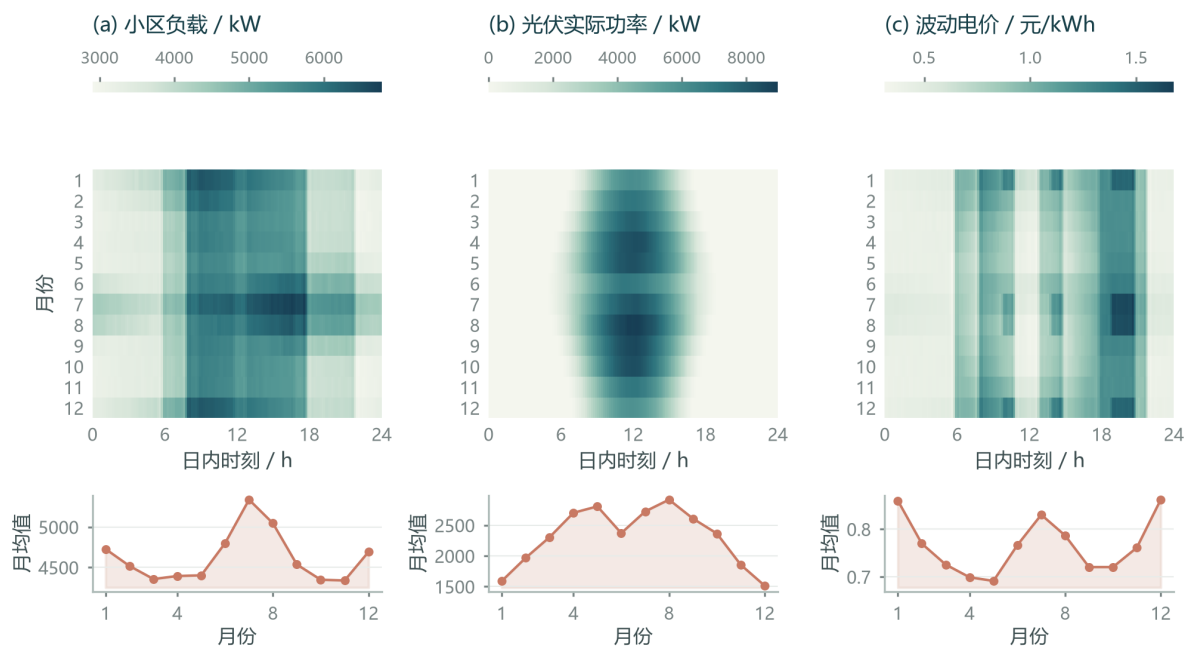


图3 负载、实际光伏与电价的月份及日内结构，热力图为月内同一时槽均值，下方为月均值

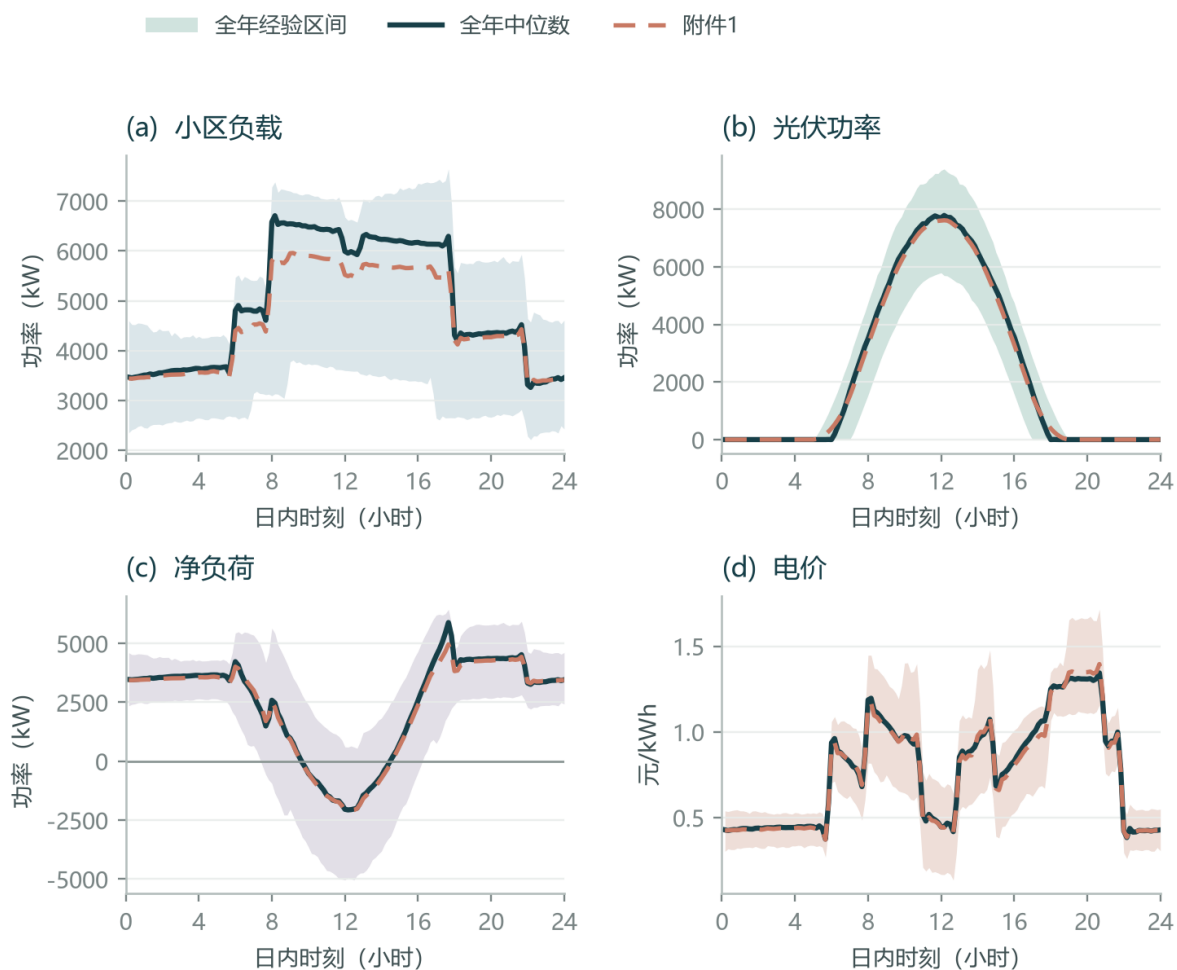


图4 日内中位数与5%-95%经验区间，虚线为附件1对应曲线；净负荷保留负值

对每个日期行的 144 个值计算总体日标准差  $s_d = [144^{-1} \sum_j (x_{d,j} - \bar{x}_d)^2]^{1/2}$ ，以其全年经验分布的  $T = Q_3 + 1.5(Q_3 - Q_1)$  筛查高波动日。负载、光伏、电价阈值分别为 2209.25 kW、4224.94 kW 和 0.4096 元/kWh，超阈值天数均为 0（图 5）。据此保留原始波动。该结果仅表示未检出高波动离群日，不能用作设备安全判据，也不能排除测量误差。

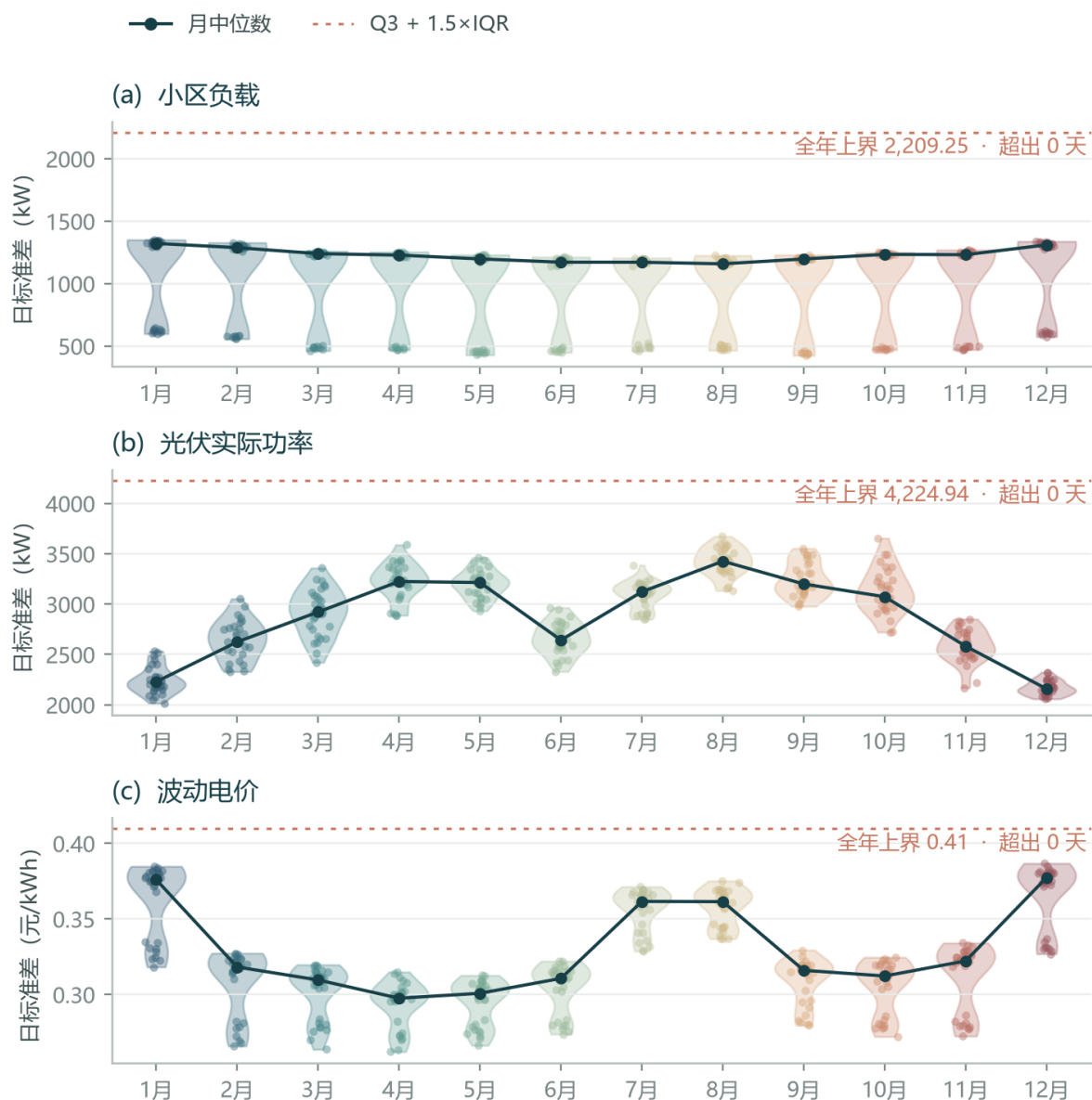


图 5 日标准差的月度分布，散点为日期行样本，实线连接月中位数，虚线为全年高波动筛查阈值

## 5.4 预报误差与信息边界

附件 3 按目标时刻与实际光伏整点值配对，获得 35004 组样本；末端 36 个跨年目标无对应真值，不计入误差。总体 MAE 为 186.61 kW、RMSE 为 438.56 kW；1–6、7–12、13–18 和 19–24 小时的 MAE 依次为 56.15、144.79、226.29 和 319.51 kW，表明较远提

前期的预测不确定性更高。夜间零误差会降低总体误差指标，但白天偏差仍需考虑；残差修正应保留提前期和目标时刻信息。

上述全年统计仅用于事后描述，不用于选择正式评价期的模型、设定阈值或删除样本。滚动预测的滞后值、过去 3 日和 7 日统计量及标准化参数均仅从节点前已成熟的数据构造；光伏预报只取发布时间不晚于节点的版本，未来实际值不进入预测与规划。当前交接段的运行结算仍遵循前述数量信息假设。候选筛选仅使用一月数据，二月至十二月保持方案冻结，新增观测成熟后才用于同类模型的更新。

## 六、统一物理模型与信息边界

### 6.1 储能约束

储能设备的最大充、放电功率均为 5000 kW。每个调度时段为 10 分钟，因此单段母线侧充、放电量的上限为

$$D = P^{\max} \Delta t = \frac{5000}{6} \text{ kWh.}$$

储能容量范围为 1200–10800 kWh，初始库存取 6000 kWh。规划阶段记第  $i$  个时段的充电量、放电量和期末库存分别为  $c_i$ 、 $b_i$  和  $e_i$ ，则

$$e_i = e_{i-1} + 0.9c_i - \frac{b_i}{0.9},$$

并满足

$$1200 \leq e_i \leq 10800, \quad c_i + b_i \leq \frac{5000}{6}.$$

实际运行时，储能库存按各时段的真实执行结果递推，即

$$E_{k+1} = E_k + 0.9C_k - \frac{B_k}{0.9},$$

其中  $C_k$  和  $B_k$  分别为第  $k$  个时段实际发生的充电量和放电电量。

购电量、充电量、放电电量、未利用电量以及模型中的紧急购电量均取非负值。规划库存  $e_i$  与实际库存  $E_k$  分别记录。前者是根据预测信息生成的参考轨迹，用于制定购电和储能计划；后者由实际负荷和光伏驱动的逐段执行得到，作为后续滚动决策的真实状态<sup>[4]</sup>。两类库存不能直接相互替代。

约束  $c_i + b_i \leq D$  表示充、放电共用单段时间预算，本身并不等价于  $c_i b_i = 0$ 。对 Q2–Q4 的规划，未利用电量  $u_i$  无上界，且目标含  $\varepsilon(c_i + b_i)$ 。若某最优解中  $c_i, b_i > 0$ ，令

$\delta = \min\{c_i, b_i/\eta^2\} > 0$ , 替换为

$$c'_i = c_i - \delta, \quad b'_i = b_i - \eta^2 \delta, \quad u'_i = u_i + (1 - \eta^2) \delta.$$

其余变量保持不变时, 能量平衡和库存递推仍成立, 非负与功率约束仍满足, 但目标减少  $\varepsilon(1 + \eta^2)\delta > 0$ , 与最优性矛盾。因此, 在上述条件下, 规划最优解不会出现同段充放电。Q1 另设弃光上界, 其同段充放电情况根据求解轨迹单独核验。实际执行层通过后文按合同余量正负划分的控制规则保证充放电互斥。

## 6.2 各分支的信息边界

四个问题共用上述物理约束。Q1 使用题设确定性负荷、光伏和电价, 并采用重复典型日边界; Q2、Q3 从旧合同交接后的真实库存重新规划; Q4 继承前问数量信息, 价格预测仅使用已完成区间的历史。

Q2、Q3 及 Q4 继承分支统一采用当前全段数量在段初可知的附加假设。在节点  $r$ , 先按旧合同执行当前交接区间, 再以该区间的真实期末库存作为下一轮规划初值, 即

$$e_0 = E_{k_r+1} = E_{k_r} + 0.9C_{k_r} - B_{k_r}/0.9.$$

当前交接区间的  $C_{k_r}, B_{k_r}$  由旧合同、当前真实净负荷和已有参考库存按储能反馈规则确定。新合同从  $r + \Delta t$  生效, 当前段继续执行旧合同。Q4 仅新增价格预测, 不另行估计交接库存。

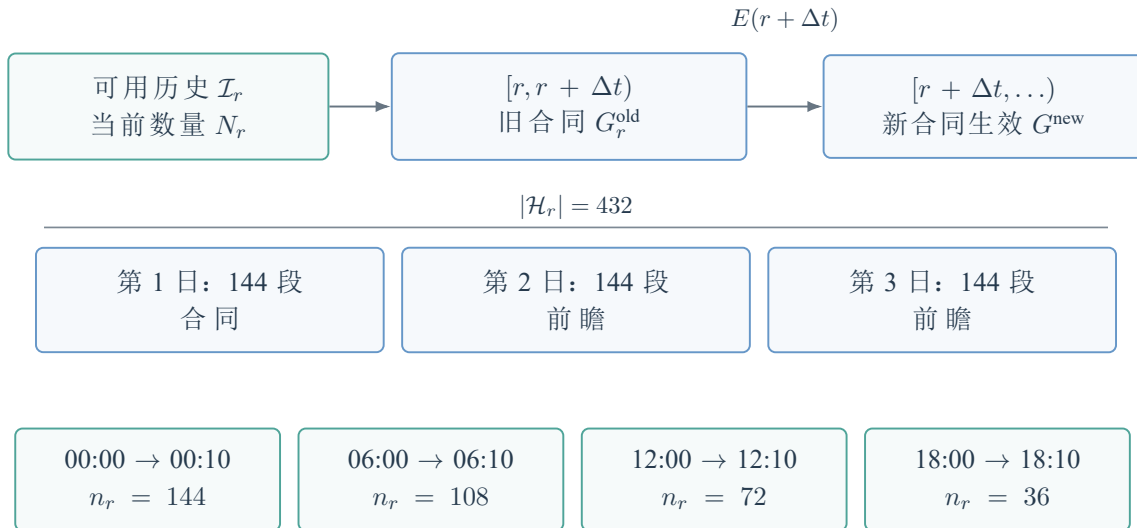


图 6 十分钟交接与滚动窗口

图 6 展示了上述时序关系。节点发布的新合同从下一个 10 分钟区间开始生效, 当前交接区间仍执行原计划, 因此预测、合同更新和实际结算在时间上保持严格的先后顺序。

### 6.3 初始状态

Q1 采用重复典型日边界，规定自然日 00:00 和 24:00 的储能库存均为 6000 kWh。当天 00:00 至 00:10 的交接区间执行上一典型日延续下来的合同，以保证日首、日末边界一致。

Q2、Q3 及 Q4 继承分支从 1 月 1 日 00:00 的 6000 kWh 初始库存开始连续预运行，使储能状态跨日连续传递。运行至 1 月 20 日 00:00 时，节点库存为 1214.351915 kWh，完成当前交接区间后降至 1200 kWh；至 2 月 1 日 00:00 时，库存为 10800 kWh。正式评价直接继承共同预运行形成的实际储能状态，不重新设定初值。共同预运行采用典型日非负净购电代理和  $\beta = 0$  的固定执行规则。一月各候选以共同预运行状态为起点，分别连续回放并评分；正式评价以共同预运行至 2 月 1 日的状态为起点。候选期末库存用于评分修正，不直接作为正式期初值，因此候选评分轨迹与共同预运行轨迹分别维护。

Q4 的初始化数量代理和物理预运行继承前问；首个预热区间缺失真实电价，不为其补造电费账单。初始化数量代理不作为真实训练标签，也不作为正式评价答案。

## 七、问题一 已知信息下的确定性调度

### 7.1 模型建立

问题一按自然日 00:00 至 24:00 的 144 个时段计费，记净负荷为  $N_i$ 。确定性线性规划为

$$\min \sum_{i=1}^{144} p_i g_i + 10^{-6} \sum_{i=1}^{144} (c_i + b_i),$$

约束为

$$\begin{aligned} g_i + b_i &= N_i + c_i + u_i, \\ e_i &= e_{i-1} + 0.9c_i - \frac{b_i}{0.9}, \\ e_0 = e_{144} &= 6000, \quad 1200 \leq e_i \leq 10800, \quad c_i + b_i \leq \frac{5000}{6}. \end{aligned}$$

Q1 将未利用电量定义为弃光量，并要求  $0 \leq u_i \leq V_i$ 。其中  $\varepsilon = 10^{-6}$  元/kWh 的微小吞吐正则项使模型倾向于选择低吞吐解，但该项不计入实际电费。设  $F$  为未加该项的购电目标、 $T = \sum_i (c_i + b_i)$ ， $x^*$  与  $x_\varepsilon$  分别为同一可行域上原目标与加项后目标的最优解，则由最优性有

$$0 \leq F(x_\varepsilon) - F(x^*) \leq \varepsilon [T(x^*) - T(x_\varepsilon)] \leq \varepsilon 144D = 0.12 \text{元}.$$

因此，该项对主目标的影响有明确上界，但使用有限系数并不严格等价于仅对同费用解作二次排序。模型先求解重复典型日的周期计划，再固定上一日延续至当前首段的购电

量并重新求解，以保持合同日与自然日的边界一致。目标函数和全部约束均为线性形式，采用 HiGHS 求解<sup>[5]</sup>。

## 7.2 结果

典型日总购电费用为 35126.85 元，普通购电量为 59482.70 kWh，充、放电量分别为 20740.67、16799.94 kWh，未利用电量为 0。库存始终满足容量约束，日末回到 6000 kWh，所得调度方案满足周期边界。

指定时段购电及全天费用、分时充放电及日初日末库存分别见附录一表附 1、表附 2。该典型日费用不能与其余各周的年度费用直接比较。

# 八、问题二 净负荷分位数预测与日前滚动调度

## 8.1 分位数预测

问题二直接预测未来净负荷。对分位水平  $\tau$ ，模型以 pinball 损失最小为目标<sup>[6]</sup>，即

$$\hat{f}_\tau \in \arg \min_f \sum_{(X,y) \in \mathcal{D}_\tau} \rho_\tau(y - f(X)),$$

$$\rho_\tau(v) = v(\tau - \mathbf{1}\{v < 0\}).$$

候选预测方法为历史同槽法、线性分位数回归和浅层分位数树，控制分位数为

$$\mathcal{Q}_N = \{0.50, 0.65, 0.80, 0.90, 0.95\},$$

库存保护系数为

$$\mathcal{B} = \{0, 0.5, 1\}.$$

五个分位数分别拟合后，采用等权保序校正消除分位数交叉<sup>[7]</sup>。所得结果为边际分位数，不能解释为五条联合随机路径。Q2 每天 0:00 重训，使用最近 30 个已完整成熟的午夜历史原点；仅当某个原点的 432 段目标全部成熟后，才将其纳入训练。

## 8.2 三日前瞻规划

每天 0:00 预测未来 432 段净负荷，并求解三日前瞻规划。前 144 段普通购电量用于签订合同，后 288 段用于估计跨日补能需求和期末库存价值。令进入规划的预测净需电量为  $q_i$ ，在 Q2 和 Q3 中规划电价等于已知电价，即  $\tilde{p}_{i|r} = p_i$ 。窗口期末库存价值系数取

$$\lambda_r = 0.9 \min_{i \in \mathcal{H}_r} \tilde{p}_{i|r}.$$

规划模型为

$$\min \left\{ \sum_{i \in \mathcal{H}_r} \tilde{p}_{i|r} g_i - \lambda_r e_{last} + \varepsilon \sum_{i \in \mathcal{H}_r} (c_i + b_i) \right\},$$

$$g_i + b_i = q_i + c_i + u_i,$$

并满足统一库存、容量、功率和非负约束。三日前瞻有助于缓解合同末端过度充满或放空的倾向，同时允许次日根据更新后的预测重新规划<sup>[8]</sup>。

### 8.3 实际执行与库存保护

设规划给出的参考期末库存为  $z_k$ ，库存保护线为

$$R_k = E^{min} + \beta(z_k - E^{min}).$$

令当前合同余量  $s_k = G_k - N_k$ 。当  $s_k \geq 0$  时，实际充电量为

$$C_k = \min \left\{ s_k, \frac{5000}{6}, \left( \frac{10800 - E_k}{0.9} \right)_+ \right\},$$

并取  $B_k = h_k = 0$ 、 $U_k = s_k - C_k$ 。当  $s_k < 0$  时，实际放电量为

$$B_k = \min \left\{ -s_k, \frac{5000}{6}, (0.9(E_k - R_k))_+ \right\},$$

并取  $C_k = U_k = 0$ 、 $h_k = -s_k - B_k$ 。实际能量平衡为

$$G_k + B_k + h_k = N_k + C_k + U_k.$$

紧急购电仅用于补足当期负荷，不用于储能充电。 $R_k$  是限制放电的阈值；若  $E_k < R_k$ ，本段不放电，但不强制购电补足至  $R_k$ ，实际库存硬下界仍为  $E^{min}$ 。

### 8.4 参数选择

三种预测方法、五个控制分位数与三个库存保护系数组成 45 个候选。各候选从相同初始库存出发，独立回放一月评价时段，评分为

$$J_{m,\tau,\beta} = \sum_{k \in \mathcal{K}_{Jan}} (p_k G_k + 5p_k h_k) - \lambda_{Jan} E_{end} + \varepsilon \sum_{k \in \mathcal{K}_{Jan}} (C_k + B_k).$$

将距最低主评分不超过  $10^{-6}$  元的候选视为并列；该评分容差与吞吐系数  $\varepsilon$  用途、量纲不同。并列时依次选择紧急购电量更少、控制分位数更低、库存保护系数更小、方法排

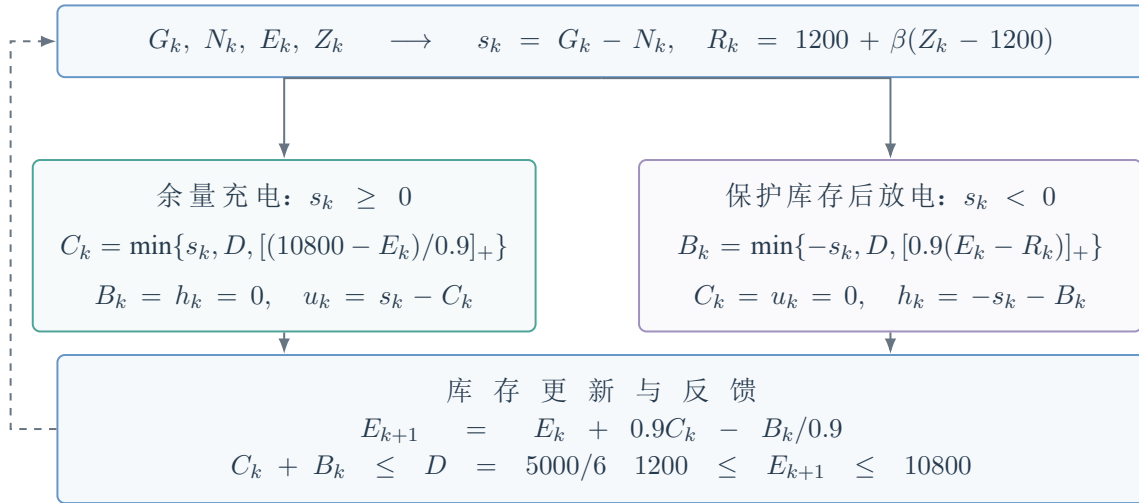


图7 库存保护与储能反馈

序更靠前的组合。最终选择浅层分位数树、 $\tau_N = 0.8$  和  $\beta = 0$ 。二月起固定模型类型和控制参数，每天仅用最新成熟历史重新拟合模型权重。

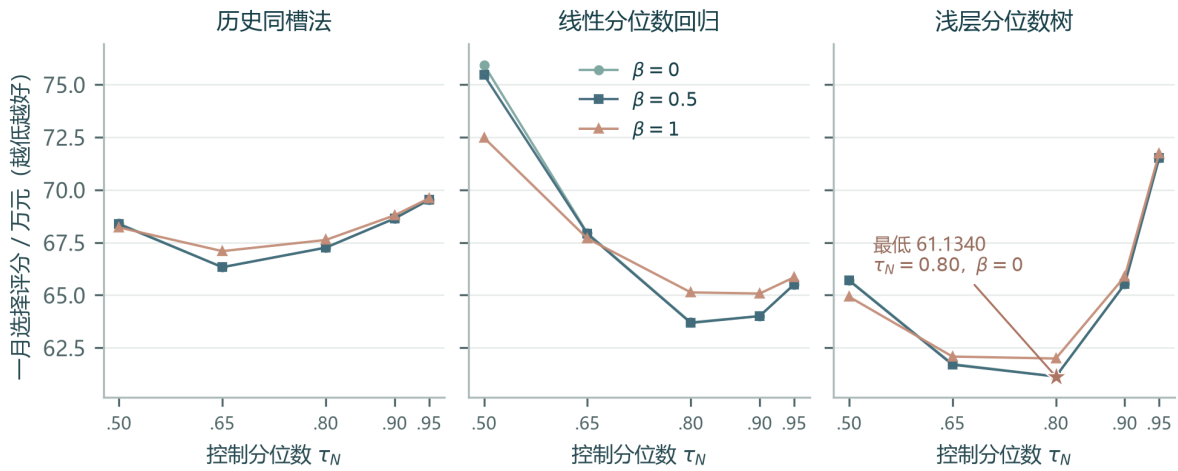


图8 Q2 一月 45 组候选评分，每个点对应一种参数组合，星号标记最低评分

如图 8 所示，浅层分位数树在  $\tau_N = 0.80$ 、 $\beta = 0$  时取得 45 组候选中的最低评分 611339.99 元，据此确定最终参数。

## 8.5 敏感性分析

为考察预测与控制参数的影响，补充选取 2025 年 6 月 1 日 00:00 至 7 月 1 日 00:00 的 4320 个十分钟时段进行单因素测试。按净负荷均值、标准差、95% 分位数及光伏电量占负载电量比进行跨月标准化，六月与二月至十二月总体特征的距离为 0.8741，在 11 个月中最小。各情景均继承六月初的实际库存 9270.8653 kWh、遗留合同和储能目标，测试窗起点不重置库存。

本项补充试验沿用前期 Q2 版本，采用浅层分位数树、30 日训练窗、 $\tau = 0.65$ 、 $\beta = 0.5$ ，每 7 日重训一次。该版本的参数、重训频率及时间对齐均与正式方案不同。因

此，以下结果仅描述该版本对参数变化的响应，不能直接验证当前每日重训方案的稳健性，也不能与 Q3、Q4 的基准费用直接比较。

以实际合同费用与紧急购电费用之和为  $C(\theta)$ ，费用变化率为

$$\Delta C(\theta) = \frac{C(\theta) - C_0}{C_0} \times 100\%, \quad C_0 = 1587928.6084 \text{元}.$$

测试结果见表 1。

**表 1 Q2 补充试验的六月单因素敏感性范围**

| 扰动因素     | 测试范围                     | 相对基准费用变化          |
|----------|--------------------------|-------------------|
| 预测模型与训练窗 | 树、线性模型分别取 14 日和 30 日     | -2.688% ~ 0.000%  |
| 控制分位数    | 0.50、0.65、0.80、0.90、0.95 | -2.558% ~ +6.187% |
| 库存保护系数   | 0、0.5、1                  | -0.962% ~ +1.786% |

模型与训练窗四组候选的费用极差为 42683.26 元，线性 14 日窗较基准降低 2.688%。费用随控制分位数提高而先降后升， $\tau = 0.80$  较基准减少 40615.05 元，而  $\tau = 0.50$  增加 98245.39 元。提高分位数可减少高价紧急购电，但分位数过高会增加未利用电量，因此费用并不随分位数单调下降。

当  $\beta$  从 0 提高到 1 时，本试验总费用下降，紧急购电量及发生时段数却上升，表明费用还取决于缺口发生时点和分时电价，不能仅凭紧急购电量评价库存保护策略。三档情景的期末库存均为 8970.19 kWh，费用比较不受期末库存差异影响。

补充试验报告显示，各情景均完成 4320 段，供需平衡与库存递推最大绝对残差分别不超过  $1.14 \times 10^{-13}$  kWh 和  $9.10 \times 10^{-13}$  kWh，无同时充放电，重复基准费用差为 0 元。在所设扰动范围内，控制分位数的费用跨度最大；本月结果不能证明一月选定参数在后续各月始终最优，也不足以作为修改正式方案的依据。若需重新冻结参数，应另设多月或全年滚动验证。

## 8.6 结果小结

采用浅层分位数树、0.8 控制分位数和 0 库存保护系数，结合三日前瞻规划逐日签约，二月至十二月总费用为 14046079.80 元，其中紧急购电费用为 566265.47 元。该配置从一月 45 组候选中选定，正式期保持冻结。

指定时段购电及全天费用、分时充放电及日初日末库存、紧急购电时段与电量分别见附录一表附 3、表附 4 及表附 5。

## 九、问题三 光伏预报驱动的日内合同调整

## 9.1 光伏预报降尺度

附件3只提供未来1–24小时的整点光伏预报。节点 $r$ 的第0个锚值取当前已知光伏功率 $A_{0|r} = P_{k_r}^{PV}$ ,  $A_{h|r}$  ( $h = 1, \dots, 24$ ) 取该节点发布的官方预报; 首个缺少观测的初始化时点沿用已标记的数量代理, 该代理不作为真实训练标签。设预测步 $m = 1, \dots, 144$ , 令 $h = \lfloor (m-1)/6 \rfloor$ 、 $v = m/6 - h$ , 则 $v \in \{1/6, 2/6, \dots, 1\}$ 。单个发布版本内省略条件下标 $r$ , 相邻整点锚值记为 $A_h, A_{h+1}$ , 线性插值为

$$\ell(v) = (1-v)A_h + vA_{h+1},$$

保持整点锚值不变的形状函数为

$$w(v) = 4v(1-v).$$

训练时只使用已成熟且 $w(v) > 0$ 的历史样本, 以归一化残差

$$y_{r,m}^{PV} = \frac{P_{k_r+m}^{PV} - \ell_{r,m}}{w(v_m)}$$

作为标签训练 LightGBM 模型<sup>[9]</sup>, 得到 $\hat{f}_{PV}(X)$ ;  $\ell_{r,m}$  及特征均依据该历史原点当时可得的信息构造。 $w(v) = 0$ 的整点不参与上述除法和残差拟合。预测时乘回 $w(v)$ , 完整修正曲线与收缩后的预测分别为

$$A(v) = \max\{0, \ell(v) + w(v)\hat{f}_{PV}(X)\},$$

$$\hat{P}_\alpha^{PV}(v) = (1-\alpha)\ell(v) + \alpha A(v).$$

候选收缩系数为 $\mathcal{A} = \{0, 0.25, 0.5, 0.75, 1\}$ 。 $\alpha$ 只改变光伏减项, 从而改变规划净需电量、合同、库存和紧急购电, 不进入负载模型训练, 因此需与其他控制参数联合选择。

## 9.2 负载分位数与光伏预测合成

Q3以真实负载 $L_i$ 为分位数模型的训练标签, 仅使用当时可得的负载历史和日历特征, 光伏预测不作为负载模型输入。将负载分位数减去独立光伏预测电量, 形成规划净需电量, 即

$$q_{i|r} = \hat{Q}_{\tau_L}^L(i | r) - \hat{V}_{\alpha,i|r}, \quad \hat{V}_{\alpha,i|r} = \hat{P}_{\alpha,i|r}^{PV} \Delta t.$$

五档 $\alpha$ 共用同一负载模型的预测,  $\alpha$ 只影响最终光伏减项。该合成量不保证是真实净负荷的严格条件分位数。

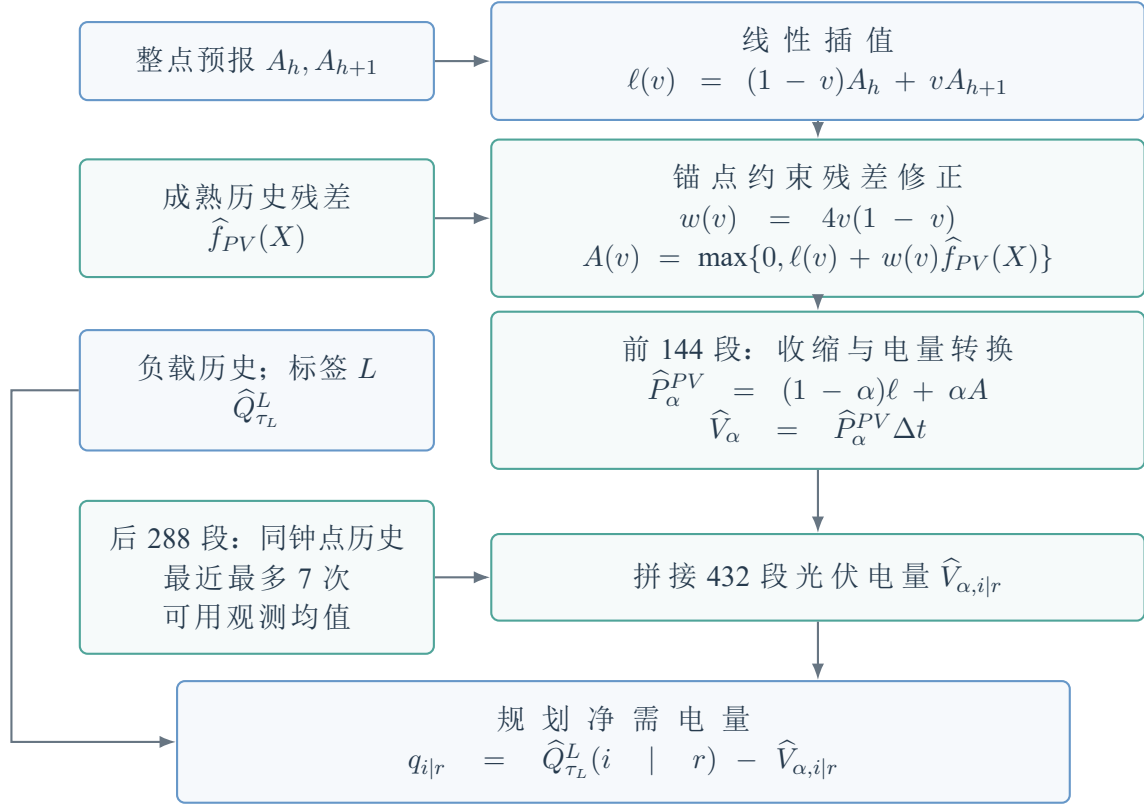


图 9 光伏降尺度与净需电量构造

前 24 小时使用官方光伏锚点和残差修正，第 25 至 72 小时使用节点当时可得的最近最多 7 个同钟点光伏观测均值。每个 0:00、6:00、12:00 和 18:00 发布节点均重新拟合光伏残差模型和负载模型；负载训练使用最近 30 自然日内、未来 432 段标签已全部成熟的历史原点。节点集合  $S$  只决定是否采用新合同，不限制预报更新。

### 9.3 合同调整费用

令  $g_i^0$  为当天 0:00 签订的原合同， $g_i$  为日内节点重新计算的合同。调减部分退还原费用的 50%，合同费用为

$$\Phi(g_i, g_i^0; p_i) = p_i g_i^0 + 1.5p_i(g_i - g_i^0)_+ - 0.5p_i(g_i^0 - g_i)_+.$$

在本文非负购电量及正规划电价范围内，两段斜率为  $0.5p_i$  和  $1.5p_i$ ，故该费用为两条仿射式的最大值。引入费用变量  $a_i$ ，可将分段函数线性化为

$$a_i \geq 0.5p_i(g_i + g_i^0),$$

$$a_i \geq 1.5p_i g_i - 0.5p_i g_i^0.$$

最小化  $a_i$  时有  $a_i = \Phi(g_i, g_i^0; p_i)$ 。各日内节点的调整费用始终以 0:00 原合同为基准计算，同一时段仅按最终生效合同结算一次。

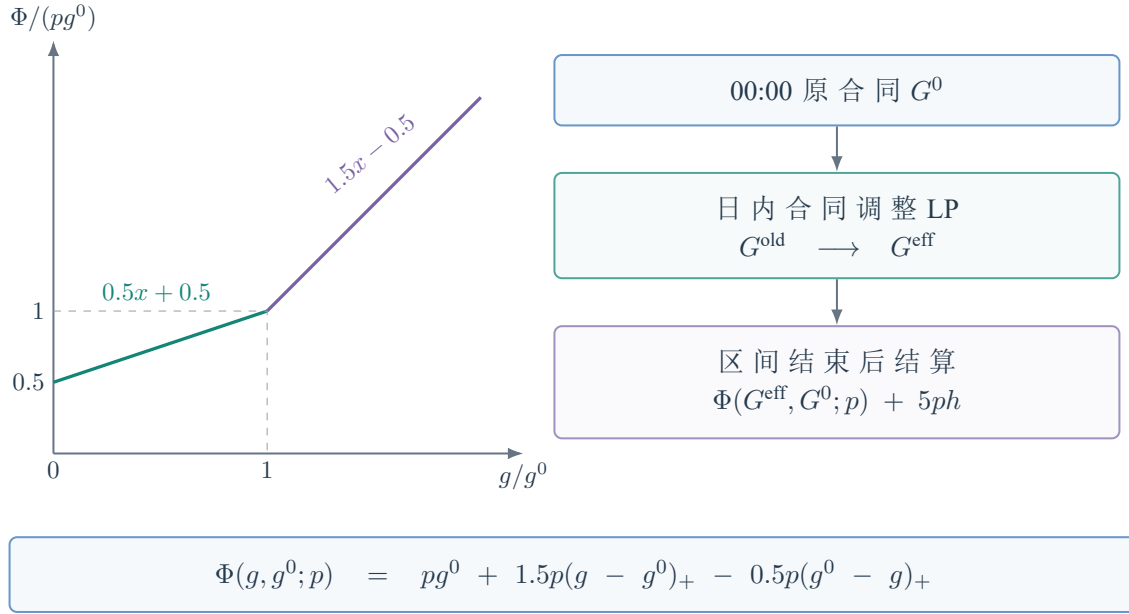


图 10 日内合同调整与结算

## 9.4 日内直接调整线性规划

0:00 先求解三日模型，再将前 144 段购电量写入原合同。日内节点  $r$  到达后 ( $\nu(r) \in \{6, 12, 18\}$ )，新计划从下一十分钟段生效，当天原合同剩余可调整段数为

$$n_r = 144 - 6\nu(r).$$

节点直接求解

$$\begin{aligned} \min \quad & \sum_{i=1}^{n_r} a_i + \sum_{i=n_r+1}^{432} p_i g_i + \sum_{i=1}^{432} 5p_i \hat{h}_i \\ & - \lambda_r e_{432} + \varepsilon \sum_{i=1}^{432} (c_i + b_i), \end{aligned}$$

满足

$$g_i + b_i + \hat{h}_i = q_i + c_i + u_i,$$

以及统一物理约束。求解后仅更新前  $n_r$  段合同，已经执行的区间、交接区间和远期模拟购电量均不修改。保持当前合同也是一个可行方案。固定剩余购电量，令  $c_i = b_i = 0$ ,

以  $\hat{h}_i = (q_i - g_i)_+$ 、 $u_i = (g_i - q_i)_+$  补足平衡，库存保持初值即可。因此，若调整无法降低当前预测目标，最优解可以保留当前合同。但这一性质不保证实际回放费用必然下降。

正规划电价下，普通增购边际费用至多为  $1.5p_i$ ，低于紧急购电的  $5p_i$ 。将任意正的  $\hat{h}_i$  等量替换为普通购电可降低目标，因此，最优解中的预测紧急购电量为零；实际执行的  $h_k$  仍可因预测误差或库存保护而为正。

## 9.5 联合选型

Q3 候选空间为

$$\Theta_3 = \mathcal{M}_L \times \mathcal{Q}_L \times \mathcal{B} \times \mathcal{A},$$

其中  $\mathcal{M}_L$  为历史同槽法、线性分位数回归和浅层分位数树；控制分位数取 0.50、0.65、0.80、0.90 和 0.95，共 225 组。日内节点集取  $\{6, 12, 18\}$  的全部八个子集，总计回放 1800 种策略。选择评分为

$$J_{\theta_3, S}^{Jan} = C_{\theta_3, S}^{Jan} - \lambda_{Jan} E_{end, \theta_3, S} + \varepsilon \sum_{k \in \mathcal{K}_{Jan}} (C_k + B_k).$$

评分在  $10^{-6}$  元容差内并列时，依次比较紧急购电量、节点数、控制分位数、库存保护系数、光伏收缩系数及预设方法和节点集合顺序，按上述顺序优先选择较小者。一月选定方案为浅层分位数树、 $\tau_L = 0.65$ 、 $\beta = 0.5$ 、 $\alpha = 0.75$ ，节点集合为  $S_3^* = \{6, 12, 18\}$ 。二月至十二月固定上述配置，只在各发布节点用成熟历史重新拟合模型。

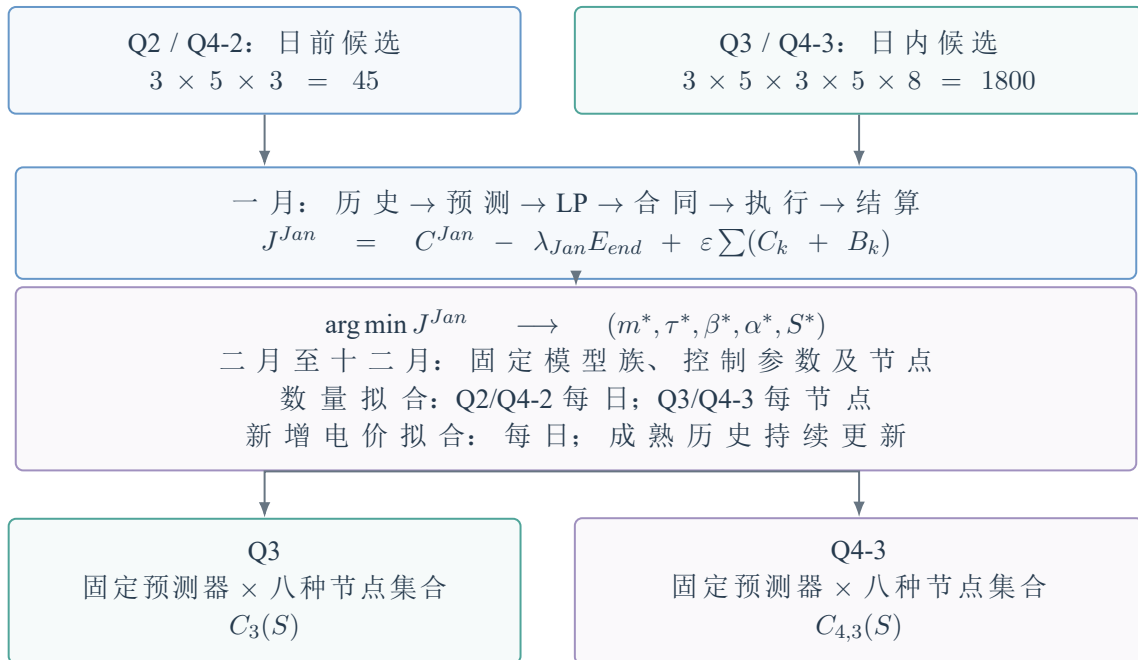


图 11 联合选参与冻结评价

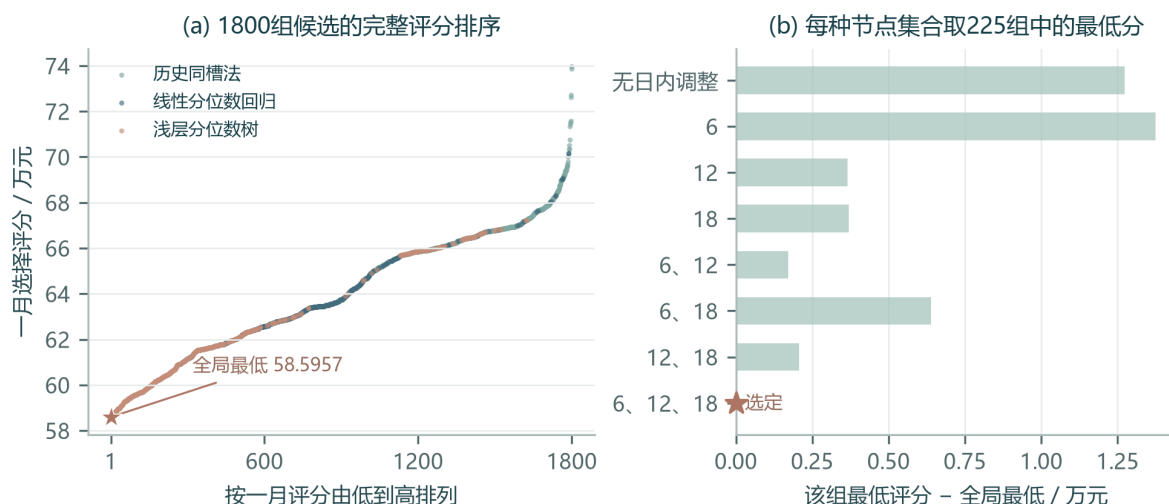


图 12 Q3 一月 1800 组联合候选评分与各节点集合的最低评分；评分包含末端库存修正

如图 12 所示，最低评分为 585956.92 元，对应浅层分位数树、 $\tau_L = 0.65$ 、 $\beta = 0.5$ 、 $\alpha = 0.75$  及 6:00、12:00、18:00 三个调整节点，据此冻结主方案。

## 9.6 敏感性分析

采用 2025 年 6 月 1 日 00:00 至 7 月 1 日 00:00 的 4320 个十分钟时段进行单因素敏感性回放。该窗口与 Q2 敏感性测试一致；按净负荷均值、标准差、95% 分位数及光伏电量占负载电量比进行标准化，在二月至十二月中，六月最接近正式期总体数量特征。各情景继承基准正式回放在六月初的实际库存和遗留合同，仅从该时点起改变待测参数。模型族、训练信息边界和逐节点重训规则保持不变。

基准取浅层分位数树、 $\tau_L = 0.65$ 、 $\beta = 0.5$ 、 $\alpha = 0.75$  及全部三个日内节点，六月总费用为 1,312,360.39 元。以  $\Delta C = [C(\theta) - C_0]/C_0$  衡量费用变化，结果如下。

表 2 Q3 六月单因素敏感性范围

| 扰动因素     | 测试范围                     | 相对基准的费用变化范围      |
|----------|--------------------------|------------------|
| 控制分位数    | 0.50、0.65、0.80、0.90、0.95 | -0.465%~+2.963%  |
| 库存保护系数   | 0、0.5、1                  | -0.058%~+6.834%  |
| 光伏残差收缩系数 | 0、0.25、0.50、0.75、1       | -0.236%~+0.487%  |
| 日内调整节点   | 6、12、18 点的八个子集           | +0.000%~+15.309% |

在所设范围内，关闭全部日内节点使费用增加 15.309%，三节点全开仍是八种集合中费用最低的方案；库存保护提高至 1 使费用增加 6.834%，表明过强的库存保护可能增加当期高价补电。五档光伏修正的最大绝对费用变化为 0.487%。六月最低费用分别出现在  $\tau_L = 0.80$  和  $\alpha = 0.50$ ，表明部分参数的选择受月份影响，一月选定值未必在每个月都最优。

本问共完成 18 个不同情景，六月基准轨迹与原正式回放逐值一致，重复运行的费用差为 0 元。供需平衡和库存递推最大残差分别不超过  $1.14 \times 10^{-13}$  kWh 及  $9.10 \times 10^{-13}$  kWh，未出现同时充放电或提前生效合同。按同分支统一末端库存估值扣除期末库存差后，所有非零费用变化的升降方向保持不变。上述范围包含基准情景。结论仅适用于本月和所设单因素扰动，不能替代全年回放，也不用于重选一月已冻结的参数。

## 9.7 结果小结

从一月 1800 种策略中选定浅层分位数树，控制分位数、库存保护系数和光伏收缩系数分别为 0.65、0.5、0.75，合同调整节点为 6:00、12:00、18:00。二月至十二月总费用为 13365464.69 元，较仅在 0:00 签约减少 1130235.69 元。

指定时段购电及全天费用、分时充放电及日初日末库存、紧急购电时段与电量分别见附录一表附 6、表附 7 及表附 8。

# 十、问题四 波动电价下的重新调度

## 10.1 电价预测

设  $\bar{p}_{j,r}^{(7)}$  为节点  $r$  之前最近最多 7 个完整历史日中相同日内位置  $j$  的真实电价均值。以该均值为基线，建立均值型残差预测模型，即

$$\hat{p}_{i|r} = \max \left\{ 10^{-6}, \bar{p}_{j(i),r}^{(7)} + \hat{f}_p(X_{i,r}) \right\}.$$

残差模型以平方损失最小为目标，即

$$\hat{f}_p \in \arg \min_f \sum_{(X,y) \in \mathcal{D}_r^p} (y - f(X))^2,$$

$$y = p_i - \bar{p}_{j(i),r_{old}}^{(7)}.$$

历史样本的基线按其原发布时间  $r_{old}$  重建，不使用当前节点或全年信息回填。电价模型每天 0:00 使用最多最近 30 天的成熟样本重训，日内节点仅更新信息边界内的历史特征及未来预测。对于节点信息  $\mathcal{I}_r$  下已确定的购电量，电费满足

$$\mathbb{E}[p_{k_r+i} g_i | \mathcal{I}_r] = g_i \mathbb{E}[p_{k_r+i} | \mathcal{I}_r],$$

因此本文以条件均值作为当前确定性规划的价格系数。实际紧急购电及后续调整会随新信息变化，上述等式并未消除价量相关性；本方法的总体经济效果仍通过冻结方案后的回放评价。

## Q4 规划使用

$$\hat{p}_{i|r} = \hat{p}_{i|r}, \quad \lambda_r = 0.9 \min_{i \in \mathcal{H}_r} \hat{p}_{i|r}.$$

上述  $\lambda_r$  沿用三日前瞻规划的末端库存估值方式，不能视为由条件均价精确推导出的随机终端价值。实际区间结束后仍按  $p_k$  结算，预测价格不直接进入账单。

## 10.2 Q4-2 重算问题二

Q4-2 保持净负荷分位数预测、每天 0:00 签约、三日前瞻规划和库存保护结构，只把规划中的已知电价替换为预测电价<sup>[3]</sup>。三个净负荷预测方法、五个控制分位数和三个库存保护系数组成 45 个候选，各候选在一月严格按时间顺序完成预测、规划和结算，再依据

$$J_{\theta_{4,2}}^{Jan} = \sum_{k \in \mathcal{K}_{Jan}} (p_k G_k + 5p_k h_k) - \lambda_{Jan} E_{end} + \varepsilon \sum_{k \in \mathcal{K}_{Jan}} (C_k + B_k)$$

选择方案。选定配置为浅层分位数树、 $\tau_{N,4} = 0.8$  和  $\beta_{N,4} = 0$ 。正式评价期普通购电费用为 14065543.09 元，紧急购电费用为 601420.54 元，总费用为 14666963.63 元。

## 10.3 Q4-3 重算问题三

Q4-3 完整继承 Q3 的负载预测、光伏预报、逐节点拟合、真实交接库存，以及通过线性规划直接调整日内合同的机制。先预测负载分位数，再减去光伏点预测，构造

$$q_{i|r} = \hat{Q}_{\tau_L}^L(i | r) - \hat{V}_{i|r}.$$

该量是基于负载分位数的规划净需电量，不保证是真实净负荷的  $\tau_L$  条件分位数。负载模型不使用光伏预测作为输入，光伏收缩系数仅影响合成净需电量时的光伏减项。前 144 段光伏使用官方锚点与残差修正，后 288 段使用最近最多 7 个可用同钟点观测的均值。

节点规划采用预测价格计算合同调整费用，即

$$\hat{\Phi}(g_i, g_i^0; \hat{p}_{i|r}) = \hat{p}_{i|r} g_i^0 + 1.5 \hat{p}_{i|r} (g_i - g_i^0)_+ - 0.5 \hat{p}_{i|r} (g_i^0 - g_i)_+.$$

Q4-3 比较 3 种负载预测方法、5 个控制分位数、3 个库存保护系数、5 个光伏收缩系数和 8 种节点集合，共 1800 种策略。一月选定配置为浅层分位数树、 $\tau_{L,4} = 0.65$ 、 $\beta_{4,3} = 0$ 、 $\alpha_{4,3} = 0.75$  和  $S_{4,3}^* = \{6, 12, 18\}$ 。正式评价期合同费用为 13573293.40 元，紧急购电费用为 477380.52 元，总费用为 14050673.91 元。

## 10.4 电价预测误差

按二月至十二月各发布节点的 432 个预测步统计，仅保留目标时刻落在正式评价自然日期间的预测与真值对。同一目标在不同发布原点的预测分别计数，年末缺少真值的跨年目标不纳入误差统计。

四个发布时间的 MAE 为 0.038101 至 0.038754 元/kWh，RMSE 为 0.052377 至 0.053568 元/kWh，各节点误差尺度接近。

Q4-2 和 Q4-3 使用同一价格预测器，在相同节点生成的价格预测逐值一致。因此，二者的费用差异来自各自的负荷或净负荷预测、合同机制和控制策略。

## 10.5 敏感性分析

沿用相同的六月窗口和单因素方法，Q4-2 与 Q4-3 各情景分别继承本分支基准在六月初的库存及遗留合同。基准参数分别为  $(\tau, \beta) = (0.80, 0)$ ，以及  $(\tau, \beta, \alpha) = (0.65, 0, 0.75)$ 、三节点全开；基准费用分别为 1,563,237.80 元和 1,386,583.05 元。各费用变化百分比均以对应分支的基准费用为参照。

表 3 Q4 两个分支的六月单因素敏感性范围

| 扰动因素     | Q4-2 费用变化范围      | Q4-3 费用变化范围      |
|----------|------------------|------------------|
| 控制分位数    | +0.000%~+12.497% | -0.625%~+2.013%  |
| 库存保护系数   | -0.666%~+0.000%  | +0.000%~+7.119%  |
| 光伏残差收缩系数 | 不适用              | -0.205%~+0.403%  |
| 日内调整节点   | 不适用              | +0.000%~+15.096% |
| 电价残差修正强度 | +0.000%~+0.124%  | -0.025%~+0.234%  |
| 预测电价时段差异 | -0.010%~+0.072%  | -0.026%~+0.085%  |

数量预测与控制参数采用 Q3 的相同候选范围。价格侧设置残差系数 0、0.5、1、1.5，分别取消、减弱、保留和增强原价格模型相对近 7 日同槽均价的修正；另将每个 144 段预测块内偏离预测均价的幅度缩放为 0.8 倍和 1.2 倍。两类扰动分别回放，仅改变规划价格及相应的规划末端库存价值；实际账单始终按原真实电价结算。

Q4-2 在五档分位数中仍以 0.80 费用最低，降至 0.50 时费用增加 12.497%。Q4-3 保留全部日内节点时费用最低，取消日内调整使费用增加 15.096%，提高库存保护至 1 则增加 7.119%。两个分支在本次价格预测扰动下的费用变化均较小，但该结果不能推广至真实电价冲击、预测时点错位或突发尖峰。

本问共完成 35 个不同情景，六月基准轨迹与原正式回放逐值一致，重复运行的费用差为 0 元。供需平衡和库存递推最大残差分别不超过  $1.14 \times 10^{-13}$  kWh 及  $9.10 \times 10^{-13}$  kWh，未出现同时充放电或提前生效合同。按同分支统一末端库存估值扣除期末库存差

后，所有非零费用变化的升降方向保持不变。上述范围包含基准情景。结论仅适用于本月和所设单因素扰动，不能替代全年回放，也不用于重选一月已冻结的参数。

## 10.6 结果小结

在波动电价下重算问题二，选用浅层分位数树、0.8 控制分位数和 0 库存保护系数。二月至十二月总费用为 14666963.63 元，其中普通购电费用为 14065543.09 元、紧急购电费用为 601420.54 元；预测价格用于规划，真实价格用于结算。

指定时段购电及全天费用、分时充放电及日初日末库存、紧急购电时段与电量分别见附录一表附 9、表附 10 及表附 11。

重算问题三采用浅层分位数树，分位数、库存保护和光伏收缩参数分别为 0.65、0、0.75，在 6:00、12:00、18:00 调整合同。二月至十二月总费用为 14050673.91 元，其中合同费用为 13573293.40 元、紧急购电费用为 477380.52 元。

指定时段购电及全天费用、分时充放电及日初日末库存、紧急购电时段与电量分别见附录一表附 12、表附 13 及表附 14。

# 十一、结果分析

## 11.1 各分支主结果

正式期总费用：Q2 为 14046079.80 元，Q3 为 13365464.69 元，Q4-2 为 14666963.63 元，Q4-3 为 14050673.91 元。题目指定四日的购电、储能及紧急购电结果，分别见附录表附 3—附 5、表附 6—附 8、表附 9—附 11 和表附 12—附 14。填写口径及模型参数见附录。

Q2 的一月选型结果为浅层分位数树、 $\tau = 0.8$  和  $\beta = 0$ 。Q1 是单个典型日的确定性结果，Q2、Q3、Q4-2 和 Q4-3 是二月至十二月的顺序回放结果，Q1 与年度结果不能按绝对费用直接横向比较。Q4-2 和 Q4-3 分别继承 Q2 和 Q3 的数量预测、信息假设、初始化、真实库存衔接及拟合频率，新增电价预测并在波动电价下一月重新选参。Q3 与 Q4-3 的费用差包含电价环境、价格预测及选参变化的共同影响，不能直接解释为电价预测误差损失。

## 11.2 Q3 节点价值

在固定 Q3 胜出模型族、控制分位数、库存保护系数和光伏收缩系数后，对八种节点集合分别回放正式期。相对节省以仅 0:00 签约为基准。

一月选出的主方案为 6:00、12:00 和 18:00 调整，正式费用为 13365464.69 元。固定同一预测器的八种节点对照中，该节点集合的费用同样最低，相对仅 0:00 签约节省

1130235.69 元。正式期对照不回写一月已经冻结的主方案。对照还显示，增加日内节点通常提高合同费用，同时降低紧急购电费用；是否值得调整取决于两项费用的净变化。18:00 单节点的节省明显高于 6:00 和 12:00 单节点，说明该时点的更新信息在本次回放中对降低晚间供电缺口更有价值。

### 11.3 Q4-3 节点价值

Q4-3 一月选择全部三个日内节点，固定模型的正式八组对照也以全开方案费用最低。相对仅 0:00 签约，正式期总费用减少 1192010.75 元。日内调整将紧急购电费用从 2078673.30 元降至 477380.52 元，下降额超过合同费用增加额。全开主方案实际调整 1002 次；部分其他节点安排会出现不改约，例如仅 18:00 的方案调整 332 次。

## 十二、数值核验

规划与实际执行分别按各自的信息口径核验。对每个规划时段，复算

$$\begin{aligned} g_i + b_i + \hat{h}_i - q_i - c_i - u_i &= 0, \\ e_i - e_{i-1} - 0.9c_i + b_i/0.9 &= 0. \end{aligned}$$

Q1 取  $q_i = N_i$ 、 $\hat{h}_i = 0$ ；Q2 的规划不含紧急购电变量，按  $\hat{h}_i = 0$  核验。Q3 及 Q4 按各分支的规划模型检查预测变量，不将未来真实净负荷代入规划平衡式。对每个真实执行时段，另复算

$$\begin{aligned} G_k + B_k + h_k - N_k - C_k - U_k &= 0, \\ E_{k+1} - E_k - 0.9C_k + B_k/0.9 &= 0. \end{aligned}$$

同时检查  $E_k \in [1200, 10800]$ 、充放电功率不超过 5000 kW、同一真实时段不同时充放电、紧急购电不用于充电、合同只修改尚未执行区间、三日前瞻始终包含 432 段，以及预测特征不越过各分支的信息截止时刻。全部费用均依据逐段合同量及其最终生效版本、紧急购电量和真实结算电价独立复算。

此外，通过扰动未来真实值检查因果边界。各分支统一按段初数量可知假设处理当前数量；扰动节点之后才可获得的数量或尚未结算的真实价格，不得改变该节点的预测和合同。当前已知数量的变化可以影响当前节点特征、真实交接库存及后续决策。该检验分别考察计划的因果性与执行反馈中的状态传递。

依据最终方案的逐段独立复算记录，Q1、Q2、Q3、Q4-2 和 Q4-3 均通过各自的物理约束与费用核验；对应核验记录分别包含 16、79、12、13 和 13 项检查，全部通过。各记录的检查范围不同，不能依据检查项数比较方案优劣。能量平衡、库存递推与连续性、容量及功率边界、合同时点均满足相应数值容差；逐段费用汇总与正文报告的五项总费用一致（保留两位小数）。其中，Q1 能量平衡与库存递推的最大残差分别为  $3.41 \times 10^{-13}$

和  $8.31 \times 10^{-13}$  kWh; Q2 正式期 48096 段实际执行的对应最大残差分别为  $1.14 \times 10^{-13}$  和  $9.10 \times 10^{-13}$  kWh。上述核验通过结论仅说明, 在本文数据、信息假设和冻结方案下, 数值执行与结算保持一致, 不能据此认为预测无误差或策略在跨年份情景中最优。

## 十三、模型评价与结论

### 13.1 目标完成情况

本文完成了题目要求的五个求解分支。Q1 在期末库存回到 6000 kWh 的周期边界下, 得到 35126.85 元的典型日费用; Q2 通过净负荷分位数预测与日前滚动签约, 正式期费用为 14046079.80 元; Q3 利用光伏预报进行日内合同调整, 费用为 13365464.69 元。在波动电价下重新选型并调度后, Q4-2 和 Q4-3 的费用分别为 14666963.63 元和 14050673.91 元。Q1 为单个典型日, 其余为二月至十二月顺序回放, 不能直接比较绝对费用。

### 13.2 模型有效性与优点

固定已选预测器和控制参数开展节点对照时, Q3 费用由仅 0:00 签约的 14495700.38 元降至三节点全开的 13365464.69 元, 节省 1130235.69 元 (7.80%), 紧急购电费用下降 77.68%。Q4-3 相应由 15242684.67 元降至 14050673.91 元, 按未舍入值计算节省 1192010.75 元 (7.82%), 紧急购电费用下降 77.03%。日内更新主要通过减少供电缺口降低本次回放费用, 对照结果不用于修改一月冻结方案。Q2 尚缺少同口径基准, 现有结果能够支持滚动调度的可执行性, 但不能单独说明其相对改进幅度。

模型以统一的能量平衡和库存递推描述四问, 分别记录预测、规划、合同版本和执行结算。分位数预测及库存保护考虑供需偏差的不对称代价, 最终依据调度费用选参; 三日前瞻考虑跨日库存价值, 日内 LP 保留原合同作为可行方案。一月选型后冻结结构, 仅用成熟历史更新权重。各分支通过能量平衡、容量、功率及合同时点核验, 未出现同时充放电或紧急购电充电, 所报告的经济结果均在上述约束满足的条件下取得。

### 13.3 敏感性分析对模型的支持程度

敏感性分析支持模型的调度机制及其在有限范围内的稳定性, 但不能证明全部冻结参数始终最优。首先, 在六月共同初始库存与遗留合同的对照中, 关闭全部日内节点使 Q3 和 Q4-3 费用分别增加 15.309% 和 15.096%, 三节点全开仍为各自八种集合中费用最低的方案。这一变化方向与正式期节点对照一致, 支持利用新预报调整尚未执行合同的经济作用; 期末库存估值修正后, 费用差异的方向保持不变。

其次, 五档光伏修正强度下, Q3 和 Q4-3 的最大绝对费用变化分别为 0.487% 和 0.403%; 两种价格预测扰动下, Q4-2 和 Q4-3 的最大绝对费用变化分别为 0.124% 和

0.234%。这说明在本月、所设扰动范围及其余条件固定时，这些预测修正引起的总费用变化较小。53 个情景均满足能量平衡、库存、功率和合同延迟约束，表明执行层在本次参数扰动下保持物理可行。这些结果不能替代预测精度评价，也不能证明模型对实际系统的全部假设均成立。

参数稳定性的证据仍有明确边界。Q3 和 Q4-3 将库存保护系数提高至 1 时，费用分别增加 6.834% 和 7.119%；二者在六月的分位数及光伏修正最低费用点也与一月冻结值不同。因此，模型仍需合理校准参数，不能笼统认为其对所有参数均不敏感。Q4-2 的 0.80 分位数在本月五档中仍为最低费用点，仅能说明该项选择在本月仍适用。Q2 补充试验采用前期版本，其重训频率、参数及时间对齐不同，不能直接证明本文当前 Q2 方案的稳健性。

现有证据支持统一物理约束下滚动调度的可执行性、日内信息更新的经济价值，以及费用对部分预测修正参数的局部稳定性，但尚不足以证明方案在跨月份、跨年份或连续参数空间中普遍最优。单月、单因素试验未覆盖参数交互与极端价格冲击，因此保留一月冻结方案，不根据事后结果重新选参。

### 13.4 适用边界与改进方向

Q2、Q3 及其 Q4 分支共同补充假设全段数量在段初可知；该假设不能仅由时间标签推出。十分钟尺度的执行不能代表逐秒控制；若储能动作须在段初锁定，则需重建执行层。负载分位数减光伏点预测未描述误差联合分布，不能视为严格的净负荷条件分位数；电价条件均值预测也未显式刻画尾部风险。

参数仅在一月的有限网格内选取，1800 组候选中的最低评分不代表跨年份或连续参数空间中的全局最优。结果限于 2025 年及题设规则，未计售电、电池退化<sup>[10]</sup>、需求响应与实时波动。

后续可在遵循现有信息截止时刻、十分钟合同延迟和物理约束的条件下，开展多月、多因素及跨年份回放，检验一月冻结配置在不同季节、价格波动和预测误差下的适用性。数量预测可进一步刻画负载与光伏预测误差的联合分布，价格预测可增加对尾部风险的描述，并据此比较库存保护与合同调整规则。若扩展至实际运行场景，还可纳入售电、电池退化、需求响应和更短时间尺度的控制约束。训练样本与预测特征仍应严格限定为决策时刻可得的信息，并按独立结算口径复算费用及约束满足情况。

## AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于题意与建模讨论、代码编写与调试、数值实验与结果核验辅助、论文与图表整理及语言润色，详细使用情况见支撑材料。

## 参 考 文 献

- [1] García Vera Y E, Dufo-López R, Bernal-Agustín J L. Energy management in microgrids with renewable energy sources: a literature review[J/OL]. Applied Sciences, 2019, 9 (18): 3854. DOI:10.3390/app9183854.
- [2] Faustine A, Pereira L. FPSeq2Q: fully parameterized sequence to quantile regression for net-load forecasting with uncertainty estimates[J/OL]. IEEE Transactions on Smart Grid, 2022, 13 (3): 2440-2451. DOI:10.1109/TSG.2022.3148699.
- [3] Du Yan, Pei Wei, Chen Naishi, et al. Real-time microgrid economic dispatch based on model predictive control strategy[J/OL]. Journal of Modern Power Systems and Clean Energy, 2017, 5 (5): 787-796. DOI:10.1007/s40565-017-0265-4.
- [4] Sigalo M B, Pillai A C, Das S, et al. An energy management system for the control of battery storage in a grid-connected microgrid using mixed integer linear programming [J/OL]. Energies, 2021, 14 (19): 6212. DOI:10.3390/en14196212.
- [5] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J/OL]. Mathematical Programming Computation, 2018, 10 (1): 119-142. DOI:10.1007/s12532-017-0130-5.
- [6] Koenker R, Bassett G. Regression quantiles[J/OL]. Econometrica, 1978, 46 (1): 33-50. DOI:10.2307/1913643.
- [7] Barlow R E, Brunk H D. The isotonic regression problem and its dual[J/OL]. Journal of the American Statistical Association, 1972, 67 (337): 140-147. DOI:10.1080/01621459.1972.10481216.
- [8] Palma-Behnke R, Benavides C, Lanas F, et al. A microgrid energy management system based on the rolling horizon strategy[J/OL]. IEEE Transactions on Smart Grid, 2013, 4 (2): 996-1006. DOI:10.1109/TSG.2012.2231440.
- [9] Ke Guolin, Meng Qi, Finley T, et al. LightGBM: a highly efficient gradient boosting decision tree[C]//Advances in neural information processing systems 30. 2017.
- [10] Shamarova N, Suslov K, Ilyushin P, et al. Review of battery energy storage systems modeling in microgrids with renewables considering battery degradation[J/OL]. Energies, 2022, 15 (19): 6967. DOI:10.3390/en15196967.

附录一 结果表

1.1 问题一

表附 1 和表附 2 分别列示指定时段购电量与全天费用、分时充放电量与日初日末储电量。

表 附 1 问题一指定时段购电量与全天费用（电量单位：kWh）

| 时间段         | 购电量      | 时间段         | 购电量    | 时间段         | 购电量      |
|-------------|----------|-------------|--------|-------------|----------|
| 10:00–10:10 | 0.00     | 12:00–12:10 | 486.40 | 14:00–14:10 | 0.00     |
| 16:00–16:10 | 394.93   | 18:00–18:10 | 636.98 | 20:00–20:10 | 0.00     |
| 全天购电量       | 59482.70 | 全天购电费/元     |        |             | 35126.85 |

表 附 2 问题一分时充放电量与日初、日末储电量（单位：kWh）

| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |
|-------------|---------|---------|-------------|---------|---------|
| 0:00–4:00   | 4500.00 | 0.00    | 4:00–8:00   | 833.33  | 5947.42 |
| 8:00–12:00  | 3954.63 | 2121.42 | 12:00–16:00 | 6119.37 | 91.10   |
| 16:00–20:00 | 0.00    | 5068.69 | 20:00–24:00 | 5333.33 | 3571.31 |
| 0:00 储电量    |         | 6000.00 | 24:00 储电量   |         | 6000.00 |

1.2 问题二

表附 3、表附 4 和表附 5 依次列示指定时段购电量与全天费用、分时充放电量与日初日末储电量、紧急购电时段与购电量。

表 附 3 问题二指定时段购电量与全天费用

| 2025-03-20  |          |             |        |             |          |
|-------------|----------|-------------|--------|-------------|----------|
| 时间段         | 购电量      | 时间段         | 购电量    | 时间段         | 购电量      |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 784.27 | 14:00–14:10 | 0.00     |
| 16:00–16:10 | 476.33   | 18:00–18:10 | 736.87 | 20:00–20:10 | 0.00     |
| 全天购电量       | 68231.80 | 全天购电费/元     |        |             | 41901.88 |
| 2025-06-21  |          |             |        |             |          |
| 时间段         | 购电量      | 时间段         | 购电量    | 时间段         | 购电量      |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 0.00   | 14:00–14:10 | 0.00     |
| 16:00–16:10 | 189.22   | 18:00–18:10 | 515.02 | 20:00–20:10 | 0.00     |
| 全天购电量       | 42890.57 | 全天购电费/元     |        |             | 24735.71 |
| 2025-09-23  |          |             |        |             |          |
| 时间段         | 购电量      | 时间段         | 购电量    | 时间段         | 购电量      |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 546.38 | 14:00–14:10 | 0.00     |
| 16:00–16:10 | 448.65   | 18:00–18:10 | 856.07 | 20:00–20:10 | 0.48     |
| 全天购电量       | 67916.58 | 全天购电费/元     |        |             | 44112.47 |
| 2025-12-21  |          |             |        |             |          |
| 时间段         | 购电量      | 时间段         | 购电量    | 时间段         | 购电量      |

表 附 3（续）

| 时间段         | 购电量      | 时间段         | 购电量     | 时间段         | 购电量      |
|-------------|----------|-------------|---------|-------------|----------|
| 10:00–10:10 | 0.00     | 12:00–12:10 | 1026.27 | 14:00–14:10 | 0.00     |
| 16:00–16:10 | 821.95   | 18:00–18:10 | 748.20  | 20:00–20:10 | 0.00     |
| 全天购电量       | 97354.89 | 全天购电费/元     |         |             | 62606.84 |

表 附 4 问题二分时充放电量与日初、日末储电量

| 2025-03-20  |         |         |             |         |         |
|-------------|---------|---------|-------------|---------|---------|
| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |
| 0:00–4:00   | 1458.47 | 92.35   | 4:00–8:00   | 1099.39 | 4898.65 |
| 8:00–12:00  | 5458.12 | 3265.56 | 12:00–16:00 | 4441.94 | 266.53  |
| 16:00–20:00 | 199.35  | 5170.61 | 20:00–24:00 | 5085.28 | 3661.75 |
| 0:00 储电量    |         | 9050.47 | 24:00 储电量   |         | 5734.93 |
| 2025-06-21  |         |         |             |         |         |
| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |
| 0:00–4:00   | 760.47  | 61.79   | 4:00–8:00   | 330.27  | 2430.63 |
| 8:00–12:00  | 3970.06 | 0.00    | 12:00–16:00 | 0.00    | 0.00    |
| 16:00–20:00 | 175.40  | 5092.98 | 20:00–24:00 | 8503.17 | 3031.63 |
| 0:00 储电量    |         | 9014.63 | 24:00 储电量   |         | 9583.37 |
| 2025-09-23  |         |         |             |         |         |
| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |
| 0:00–4:00   | 1972.72 | 0.00    | 4:00–8:00   | 378.29  | 5925.55 |
| 8:00–12:00  | 3898.01 | 2387.94 | 12:00–16:00 | 6506.90 | 552.32  |
| 16:00–20:00 | 220.26  | 5044.69 | 20:00–24:00 | 8335.94 | 3496.75 |
| 0:00 储电量    |         | 8840.23 | 24:00 储电量   |         | 8679.76 |
| 2025-12-21  |         |         |             |         |         |
| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |
| 0:00–4:00   | 1212.13 | 89.05   | 4:00–8:00   | 1061.25 | 670.01  |
| 8:00–12:00  | 4790.44 | 8098.54 | 12:00–16:00 | 7948.63 | 2220.11 |
| 16:00–20:00 | 13.97   | 4927.61 | 20:00–24:00 | 8408.88 | 3626.91 |
| 0:00 储电量    |         | 9597.35 | 24:00 储电量   |         | 8875.55 |

表 附 5 问题二指定日期紧急购电时段与购电量

| 2025-03-20  |       | 2025-06-21 |      | 2025-09-23  |        | 2025-12-21  |      |
|-------------|-------|------------|------|-------------|--------|-------------|------|
| 时间段         | 购电量   | 时间段        | 购电量  | 时间段         | 购电量    | 时间段         | 购电量  |
| 20:40–20:50 | 32.47 | 无          | 0.00 | 20:30–20:50 | 237.16 | 08:00–08:10 | 5.97 |
| –           | –     | –          | –    | 21:00–21:20 | 37.23  | –           | –    |
| 合计          | 32.47 | 合计         | 0.00 | 合计          | 274.39 | 合计          | 5.97 |

1.3 问题三

表附 6、表附 7 和表附 8 依次列示指定时段购电量与全天费用、分时充放电量与日初日末储电量、紧急购电时段与购电量。

表 附 6    问题三指定时段购电量与全天费用

| 2025-03-20  |          |             |        |             |      |
|-------------|----------|-------------|--------|-------------|------|
| 时间段         | 购电量      | 时间段         | 购电量    | 时间段         | 购电量  |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 438.07 | 14:00–14:10 | 0.00 |
| 16:00–16:10 | 475.86   | 18:00–18:10 | 727.36 | 20:00–20:10 | 0.00 |
| 全天购电量       | 62342.32 | 全天购电费/元     |        | 41784.69    |      |
| 2025-06-21  |          |             |        |             |      |
| 时间段         | 购电量      | 时间段         | 购电量    | 时间段         | 购电量  |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 0.00   | 14:00–14:10 | 0.00 |
| 16:00–16:10 | 58.88    | 18:00–18:10 | 441.11 | 20:00–20:10 | 0.00 |
| 全天购电量       | 40768.04 | 全天购电费/元     |        | 22202.79    |      |
| 2025-09-23  |          |             |        |             |      |
| 时间段         | 购电量      | 时间段         | 购电量    | 时间段         | 购电量  |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 367.38 | 14:00–14:10 | 0.00 |
| 16:00–16:10 | 470.88   | 18:00–18:10 | 788.82 | 20:00–20:10 | 0.00 |
| 全天购电量       | 66607.56 | 全天购电费/元     |        | 44439.85    |      |
| 2025-12-21  |          |             |        |             |      |
| 时间段         | 购电量      | 时间段         | 购电量    | 时间段         | 购电量  |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 852.65 | 14:00–14:10 | 0.00 |
| 16:00–16:10 | 819.14   | 18:00–18:10 | 720.00 | 20:00–20:10 | 0.00 |
| 全天购电量       | 94390.75 | 全天购电费/元     |        | 63443.41    |      |

表 附 7    问题三分时充放电量与日初、日末储电量

| 2025-03-20  |         |         |             |         |         |
|-------------|---------|---------|-------------|---------|---------|
| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |
| 0:00–4:00   | 1566.81 | 181.29  | 4:00–8:00   | 965.44  | 5859.92 |
| 8:00–12:00  | 3483.04 | 2879.22 | 12:00–16:00 | 7296.75 | 266.89  |
| 16:00–20:00 | 199.25  | 5007.78 | 20:00–24:00 | 4391.76 | 3692.64 |
| 0:00 储电量    |         | 8832.56 | 24:00 储电量   |         | 5070.04 |
| 2025-06-21  |         |         |             |         |         |
| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |
| 0:00–4:00   | 342.80  | 109.85  | 4:00–8:00   | 1291.84 | 1720.00 |
| 8:00–12:00  | 9547.89 | 0.00    | 12:00–16:00 | 0.00    | 0.00    |
| 16:00–20:00 | 49.35   | 5548.31 | 20:00–24:00 | 8302.15 | 3109.31 |
| 0:00 储电量    |         | 2768.89 | 24:00 储电量   |         | 8696.77 |
| 2025-09-23  |         |         |             |         |         |
| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |
| 0:00–4:00   | 1960.04 | 8.30    | 4:00–8:00   | 541.89  | 6495.58 |
| 8:00–12:00  | 4316.35 | 2194.87 | 12:00–16:00 | 6795.86 | 403.62  |
| 16:00–20:00 | 60.62   | 5214.10 | 20:00–24:00 | 8191.73 | 3436.77 |
| 0:00 储电量    |         | 8613.54 | 24:00 储电量   |         | 8567.55 |
| 2025-12-21  |         |         |             |         |         |
| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |

表 附 7（续）

| 时间段         | 充电量     | 放电量     | 时间段         | 充电量     | 放电量     |
|-------------|---------|---------|-------------|---------|---------|
| 0:00–4:00   | 1528.31 | 238.39  | 4:00–8:00   | 1593.18 | 703.48  |
| 8:00–12:00  | 4332.74 | 8570.22 | 12:00–16:00 | 8843.34 | 2102.40 |
| 16:00–20:00 | 21.12   | 5063.65 | 20:00–24:00 | 8291.07 | 3595.55 |
| 0:00 储电量    |         | 9037.18 | 24:00 储电量   |         | 8659.65 |

表 附 8 问题三指定日期紧急购电时段与购电量

| 2025-03-20  |        | 2025-06-21 |      | 2025-09-23  |        | 2025-12-21  |        |
|-------------|--------|------------|------|-------------|--------|-------------|--------|
| 时间段         | 购电量    | 时间段        | 购电量  | 时间段         | 购电量    | 时间段         | 购电量    |
| 09:10–10:10 | 386.34 | 无          | 0.00 | 09:00–10:00 | 189.68 | 08:00–08:20 | 49.73  |
| 20:40–20:50 | 38.23  | –          | –    | 10:10–10:20 | 3.39   | 10:20–11:00 | 395.43 |
| –           | –      | –          | –    | 20:30–20:50 | 140.35 | 20:40–20:50 | 29.33  |
| –           | –      | –          | –    | 21:00–21:20 | 96.62  | –           | –      |
| –           | –      | –          | –    | 21:30–22:00 | 32.96  | –           | –      |
| 合计          | 424.57 | 合计         | 0.00 | 合计          | 463.00 | 合计          | 474.49 |

1.4 问题四：重算问题二

表附 9、表附 10 和表附 11 依次列示指定时段购电量与全天费用、分时充放电量与日初日末储电量、紧急购电时段与购电量。

表 附 9 问题四：重算问题二指定时段购电量与全天费用

| 2025-03-20  |          |             |         |             |        |
|-------------|----------|-------------|---------|-------------|--------|
| 时间段         | 购电量      | 时间段         | 购电量     | 时间段         | 购电量    |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 784.27  | 14:00–14:10 | 0.00   |
| 16:00–16:10 | 476.33   | 18:00–18:10 | 0.00    | 20:00–20:10 | 0.00   |
| 全天购电量       | 64786.66 | 全天购电费/元     |         | 41816.22    |        |
| 2025-06-21  |          |             |         |             |        |
| 时间段         | 购电量      | 时间段         | 购电量     | 时间段         | 购电量    |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 0.00    | 14:00–14:10 | 0.00   |
| 16:00–16:10 | 189.22   | 18:00–18:10 | 515.02  | 20:00–20:10 | 0.00   |
| 全天购电量       | 51054.84 | 全天购电费/元     |         | 24136.71    |        |
| 2025-09-23  |          |             |         |             |        |
| 时间段         | 购电量      | 时间段         | 购电量     | 时间段         | 购电量    |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 546.38  | 14:00–14:10 | 0.00   |
| 16:00–16:10 | 448.65   | 18:00–18:10 | 22.73   | 20:00–20:10 | 833.81 |
| 全天购电量       | 66703.66 | 全天购电费/元     |         | 45368.28    |        |
| 2025-12-21  |          |             |         |             |        |
| 时间段         | 购电量      | 时间段         | 购电量     | 时间段         | 购电量    |
| 10:00–10:10 | 0.00     | 12:00–12:10 | 1026.27 | 14:00–14:10 | 0.00   |
| 16:00–16:10 | 821.95   | 18:00–18:10 | 748.20  | 20:00–20:10 | 0.00   |
| 全天购电量       | 95072.29 | 全天购电费/元     |         | 72606.92    |        |

表 附 10   问题四：重算问题二分时充放电量与日初、日末储电量

| 2025-03-20  |         |          |             |          |          |
|-------------|---------|----------|-------------|----------|----------|
| 时间段         | 充电量     | 放电量      | 时间段         | 充电量      | 放电量      |
| 0:00–4:00   | 2869.76 | 76.76    | 4:00–8:00   | 1162.68  | 4582.52  |
| 8:00–12:00  | 5458.12 | 3265.56  | 12:00–16:00 | 4016.03  | 266.53   |
| 16:00–20:00 | 87.41   | 6713.83  | 20:00–24:00 | 420.57   | 2027.86  |
| 0:00 储电量    |         | 7738.07  | 24:00 储电量   |          | 1536.69  |
| 2025-06-21  |         |          |             |          |          |
| 时间段         | 充电量     | 放电量      | 时间段         | 充电量      | 放电量      |
| 0:00–4:00   | 760.47  | 61.79    | 4:00–8:00   | 4427.36  | 1386.37  |
| 8:00–12:00  | 5167.04 | 0.00     | 12:00–16:00 | 0.00     | 0.00     |
| 16:00–20:00 | 0.00    | 5125.99  | 20:00–24:00 | 10053.69 | 3017.50  |
| 0:00 储电量    |         | 3089.69  | 24:00 储电量   |          | 10800.00 |
| 2025-09-23  |         |          |             |          |          |
| 时间段         | 充电量     | 放电量      | 时间段         | 充电量      | 放电量      |
| 0:00–4:00   | 4757.42 | 0.00     | 4:00–8:00   | 2880.67  | 5925.55  |
| 8:00–12:00  | 3865.92 | 2387.94  | 12:00–16:00 | 6502.38  | 515.71   |
| 16:00–20:00 | 117.00  | 6204.41  | 20:00–24:00 | 2064.50  | 2318.11  |
| 0:00 储电量    |         | 4081.87  | 24:00 储电量   |          | 2971.28  |
| 2025-12-21  |         |          |             |          |          |
| 时间段         | 充电量     | 放电量      | 时间段         | 充电量      | 放电量      |
| 0:00–4:00   | 32.22   | 89.05    | 4:00–8:00   | 85.12    | 6.00     |
| 8:00–12:00  | 4840.87 | 8148.97  | 12:00–16:00 | 7960.46  | 2220.11  |
| 16:00–20:00 | 13.97   | 4927.61  | 20:00–24:00 | 7590.28  | 3626.91  |
| 0:00 储电量    |         | 10800.00 | 24:00 储电量   |          | 8138.81  |

表 附 11   问题四：重算问题二指定日期紧急购电时段与购电量

| 2025-03-20  |       | 2025-06-21 |      | 2025-09-23  |        | 2025-12-21  |      |
|-------------|-------|------------|------|-------------|--------|-------------|------|
| 时间段         | 购电量   | 时间段        | 购电量  | 时间段         | 购电量    | 时间段         | 购电量  |
| 20:40–20:50 | 11.20 | 无          | 0.00 | 20:20–20:30 | 10.23  | 08:00–08:10 | 5.97 |
| –           | –     | –          | –    | 20:40–20:50 | 204.51 | –           | –    |
| –           | –     | –          | –    | 21:00–21:20 | 37.23  | –           | –    |
| 合计          | 11.20 | 合计         | 0.00 | 合计          | 251.97 | 合计          | 5.97 |

1.5   问题四：重算问题三

表附 12、表附 13 和表附 14 依次列示指定时段购电量与全天费用、分时充放电量与日初日末储电量、紧急购电时段与购电量。

表 附 12   问题四：重算问题三指定时段购电量与全天费用

| 2025-03-20 |     |     |     |     |     |
|------------|-----|-----|-----|-----|-----|
| 时间段        | 购电量 | 时间段 | 购电量 | 时间段 | 购电量 |

表附 12 (续)

| 时间段               | 购电量      | 时间段         | 购电量    | 时间段         | 购电量      |
|-------------------|----------|-------------|--------|-------------|----------|
| 10:00–10:10       | 0.00     | 12:00–12:10 | 438.07 | 14:00–14:10 | 0.00     |
| 16:00–16:10       | 475.86   | 18:00–18:10 | 0.00   | 20:00–20:10 | 0.00     |
| 全天购电量             | 59722.83 | 全天购电费/元     |        |             | 42054.08 |
| <b>2025-06-21</b> |          |             |        |             |          |
| 时间段               | 购电量      | 时间段         | 购电量    | 时间段         | 购电量      |
| 10:00–10:10       | 0.00     | 12:00–12:10 | 0.00   | 14:00–14:10 | 0.00     |
| 16:00–16:10       | 58.88    | 18:00–18:10 | 441.11 | 20:00–20:10 | 0.00     |
| 全天购电量             | 44115.91 | 全天购电费/元     |        |             | 20701.97 |
| <b>2025-09-23</b> |          |             |        |             |          |
| 时间段               | 购电量      | 时间段         | 购电量    | 时间段         | 购电量      |
| 10:00–10:10       | 0.00     | 12:00–12:10 | 367.38 | 14:00–14:10 | 0.00     |
| 16:00–16:10       | 470.88   | 18:00–18:10 | 0.00   | 20:00–20:10 | 807.84   |
| 全天购电量             | 65770.94 | 全天购电费/元     |        |             | 46203.46 |
| <b>2025-12-21</b> |          |             |        |             |          |
| 时间段               | 购电量      | 时间段         | 购电量    | 时间段         | 购电量      |
| 10:00–10:10       | 0.00     | 12:00–12:10 | 852.65 | 14:00–14:10 | 0.00     |
| 16:00–16:10       | 819.14   | 18:00–18:10 | 720.00 | 20:00–20:10 | 0.00     |
| 全天购电量             | 91764.74 | 全天购电费/元     |        |             | 73023.50 |

表附 13 问题四：重算问题三分时充放电量与日初、日末储电量

| <b>2025-03-20</b> |         |          |             |          |          |
|-------------------|---------|----------|-------------|----------|----------|
| 时间段               | 充电量     | 放电量      | 时间段         | 充电量      | 放电量      |
| 0:00–4:00         | 3065.51 | 142.69   | 4:00–8:00   | 965.44   | 5849.40  |
| 8:00–12:00        | 3483.04 | 2879.22  | 12:00–16:00 | 7296.75  | 266.89   |
| 16:00–20:00       | 127.08  | 6561.38  | 20:00–24:00 | 322.31   | 2080.59  |
| 0:00 储电量          |         | 7429.15  | 24:00 储电量   |          | 1407.54  |
| <b>2025-06-21</b> |         |          |             |          |          |
| 时间段               | 充电量     | 放电量      | 时间段         | 充电量      | 放电量      |
| 0:00–4:00         | 326.11  | 105.97   | 4:00–8:00   | 2441.27  | 1382.96  |
| 8:00–12:00        | 9605.18 | 0.00     | 12:00–16:00 | 0.00     | 0.00     |
| 16:00–20:00       | 49.35   | 5548.31  | 20:00–24:00 | 10085.90 | 3101.31  |
| 0:00 储电量          |         | 1319.06  | 24:00 储电量   |          | 10311.03 |
| <b>2025-09-23</b> |         |          |             |          |          |
| 时间段               | 充电量     | 放电量      | 时间段         | 充电量      | 放电量      |
| 0:00–4:00         | 5084.06 | 6.07     | 4:00–8:00   | 3071.28  | 6488.93  |
| 8:00–12:00        | 4316.35 | 2198.26  | 12:00–16:00 | 6800.04  | 403.62   |
| 16:00–20:00       | 57.90   | 6296.84  | 20:00–24:00 | 1923.37  | 2395.22  |
| 0:00 储电量          |         | 3515.61  | 24:00 储电量   |          | 2877.81  |
| <b>2025-12-21</b> |         |          |             |          |          |
| 时间段               | 充电量     | 放电量      | 时间段         | 充电量      | 放电量      |
| 0:00–4:00         | 274.95  | 238.39   | 4:00–8:00   | 206.22   | 33.95    |
| 8:00–12:00        | 4332.74 | 8640.00  | 12:00–16:00 | 9131.71  | 2266.20  |
| 16:00–20:00       | 21.12   | 5063.65  | 20:00–24:00 | 7467.29  | 3601.81  |
| 0:00 储电量          |         | 10669.54 | 24:00 储电量   |          | 7911.28  |

表 附 14    问题四：重算问题三指定日期紧急购电时段与购电量

| 2025-03-20  |        | 2025-06-21 |      | 2025-09-23  |        | 2025-12-21  |        |
|-------------|--------|------------|------|-------------|--------|-------------|--------|
| 时间段         | 购电量    | 时间段        | 购电量  | 时间段         | 购电量    | 时间段         | 购电量    |
| 09:10–10:10 | 386.34 | 无          | 0.00 | 09:10–10:00 | 189.68 | 08:00–08:20 | 49.73  |
| 20:40–20:50 | 31.43  | –          | –    | 20:20–20:30 | 10.23  | 10:40–11:00 | 279.36 |
| –           | –      | –          | –    | 20:40–20:50 | 134.77 | 20:40–20:50 | 29.33  |
| –           | –      | –          | –    | 21:00–21:20 | 96.62  | –           | –      |
| –           | –      | –          | –    | 21:30–22:10 | 68.19  | –           | –      |
| 合计          | 417.77 | 合计         | 0.00 | 合计          | 499.49 | 合计          | 358.43 |

附录二    模型参数与求解设置

浅层分位数树采用 60 轮、7 个叶节点、最小叶样本 30、L2 正则系数 1 和随机种子 20260911。线性分位数回归的正则系数为 0.002。光伏残差 LightGBM 采用绝对误差目标、240 棵树、学习率 0.035、31 个叶节点、最小叶样本 50、L1 正则系数 0.1、L2 正则系数 1 和随机种子 2026；成熟残差不足 2000 行时回退到线性插值。电价残差树采用平方误差目标、60 轮、7 个叶节点、最小叶样本 30、L2 正则系数 1 和随机种子 20260911。

Q2、Q3 和 Q4 的一月评分区间为 1 月 20 日 00:10 至 1 月 31 日 00:10，共 1584 段。正式评价期为 2 月 1 日 00:00 至次年 1 月 1 日 00:00，共 48096 段。Q3 和 Q4-3 各比较 1800 种策略，Q4-2 比较 45 种策略。所有候选使用相同评价区间和初始条件，库存按候选独立传递。上述区间右端均不包含在评价中。

Q2 与 Q3 一月评分的公共末端价值系数为 0.33417；Q4 采用 1 月 31 日 0:00 公共评价节点发布的 432 段预测价格最低值乘 0.9，本轮为 0.2487249171689285。该公共系数用于候选评分，区别于各滚动节点自己的规划末端系数，均不计入实际账单。Q4-2 继承 Q2 最近 30 个成熟午夜原点的数量训练窗口；Q4-3 继承 Q3 最近 30 自然日内成熟原点的训练窗口。

## 附录三 支撑材料文件列表

本附录列出压缩包“支撑材料 \_ 最小复现.rar”中的全部 39 个文件。以下路径均以包内“支撑材料 \_ 最小复现”目录为起点；五份题目结果表位于“结果表”目录，26 个 Python 源文件的完整代码见附录四。

| 序号 | 文件相对路径                                        |
|----|-----------------------------------------------|
| 1  | 00_ 文件列表.txt                                  |
| 2  | AI 使用说明/AI 工具使用详情.pdf                         |
| 3  | requirements.txt                              |
| 4  | 源码/Q1_ 第 4 节独立代码/第 2 至 4 节 _Q1 数据与时间对齐.py     |
| 5  | 源码/Q1_ 第 4 节独立代码/第 4 节 _Q1 确定性储能优化.py         |
| 6  | 源码/Q1_ 第 4 节独立代码/第 4 节 _Q1 运行与结果导出.py         |
| 7  | 源码/Q2_ 第 5 节独立代码/第 5 节 _Q2 分位数预测与滚动调度.py      |
| 8  | 源码/Q2_ 第 5 节独立代码/第 5 节 _Q2 四十五组方案对比.py        |
| 9  | 源码/Q2_ 第 5 节独立代码/第 5 节 _Q2 运行与结果导出.py         |
| 10 | 源码/Q2 前期敏感性/Q2 敏感性测试.py                       |
| 11 | 源码/Q2 前期敏感性/核心代码/02_q2_forecast_controller.py |
| 12 | 源码/Q2 前期敏感性/核心代码/battery_controller.py        |
| 13 | 源码/Q2 前期敏感性/核心代码/local_bootstrap.py           |
| 14 | 源码/Q2 前期敏感性/核心代码/q2_clock_features.npy        |
| 15 | 源码/Q2 前期敏感性/核心代码/q2_reproducibility.py        |
| 16 | 源码/Q3_Q4 敏感性/Q3_Q4 敏感性测试.py                   |
| 17 | 源码/代码/第 2 至 3 节 _ 历史数据与储能基础.py                |
| 18 | 源码/代码/第 4 至 7 节 _ 合同优化与储能执行.py                |
| 19 | 源码/代码/第 5 至 7 节 _ 联合选型与正式回放.py                |
| 20 | 源码/代码/第 6 节 _Q3 逐节点负载与光伏预测.py                 |
| 21 | 源码/代码/第 7 节 _Q4 每日负载与电价预测.py                  |
| 22 | 源码/代码/第 9 节 _ 独立核验.py                         |
| 23 | 源码/正文绘图/compact_plot_source.py                |
| 24 | 源码/正文绘图/selection_figures.py                  |
| 25 | 源码/辅助运行/第 6 至 7 节 _ 固定模型的八节点价值对照.py           |
| 26 | 源码/输出工具/第 10 节 _ 官方结果填表.py                    |
| 27 | 源码/输出工具/第 10 节 _ 提取官方填表数据.py                  |
| 28 | 生成结果表与论文数据.py                                 |
| 29 | 结果表/result1.xlsx                              |
| 30 | 结果表/result2.xlsx                              |
| 31 | 结果表/result3.xlsx                              |
| 32 | 结果表/result4-2.xlsx                            |
| 33 | 结果表/result4-3.xlsx                            |
| 34 | 请先阅读.md                                       |
| 35 | 输入转换.py                                       |
| 36 | 输入配置.json                                     |
| 37 | 运行论文结果.py                                     |
| 38 | 重要参考资料/六张机制图.pdf                              |
| 39 | 重要参考资料/论文参考文献.bib                             |

## 附录四 程序代码

本附录按附录三的文件顺序完整收录 26 个 Python 源文件，共 3164 行；文件标题表明其相对路径。另附运行依赖和输入配置，便于与压缩包逐项对应。代码保留原有注释、缩进和空行，长行仅作排版折行，行号按各文件重新计数。

### 4.1 第 2 至 4 节 \_\_Q1 数据与时间对齐.py

源码/Q1\_第 4 节独立代码/第 2 至 4 节 \_\_Q1 数据与时间对齐.py

```

1  """ 只读取 Q1 附件 1，检查原始单元格并转换为自然日左端点时段。"""
2  from pathlib import Path
3  import sys
4  import hashlib
5  import json
6  import math
7
8  ROOT = Path(__file__).resolve().parent
9  # 附带 Excel 读取库，避免原工程的 Q2/Q3 参数和公共入口影响独立运行。
10 sys.path.insert(0, str(ROOT / '依赖'))
11 import openpyxl
12
13
14 def file_sha256(path):
15     """ 文件指纹用于冻结检查；它不参与优化计算。"""
16     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
17
18
19 def check_frozen_files():
20     """ 冻结后若程序、数据或说明被改动，先报错，避免误称为原冻结版本。"""
21     record = ROOT / '冻结记录.json'
22     if not record.exists():
23         return # 首次整理并核验通过后，才生成冻结记录。
24     frozen = json.loads(record.read_text(encoding='utf-8'))
25     for name, expected in frozen['protected_files'].items():
26         path = ROOT / name
27         if not path.is_file() or file_sha256(path) != expected:
28             raise ValueError(f'冻结文件已改变或缺失：{name}；不能继续称为原冻结版本。')
29
30
31 def minute_of(value):
32     """ 附件中 00:10 至次日 00:00 的标签，转换为 10 至 1440 分钟。"""
33     if hasattr(value, 'hour'):
34         return value.hour * 60 + value.minute
35     label = str(value).strip()
36     if label in ('0:00+1', '00:00+1', '24:00'):
37         return 1440
38     hour, minute = label.split(':')[2]
39     return int(hour) * 60 + int(minute)
40
41
42 def read_records():
43     """ 返回 144 个自然日时段及原始 Excel 来源；全部数值从 Excel 读取。"""
44     path = ROOT / '数据' / '附件 1.xlsx'
45     book = openpyxl.load_workbook(path, read_only=True, data_only=True)
46     try:
47         sheet = book.worksheets[0]
48         sheet_name = sheet.title
49         rows = list(sheet.values)[1:] # 首行为表头。
50     finally:
51         book.close()
52     if len(rows) != 144 or [minute_of(r[0]) for r in rows] != list(range(10, 1441, 10)):
53         raise ValueError('附件 1 必须连续包含 00:10 至次日 00:00 的 144 个标签。')
54     raw = []
55     for i, row in enumerate(rows):
56         values = row[1:4] # 原文件 B/C/D 列依次为电价、负荷功率、光伏功率。
57         if any(isinstance(v, (str, bool)) or v is None for v in values):
58             raise ValueError(f'附件 1 第 {i+2} 行存在空值或非数值。')

```

```

59     price, load, pv = map(float, values)
60     if not all(map(math.isfinite, (price, load, pv))) or price <= 0 or min(load, pv) < 0:
61         raise ValueError(f'附件 1 第 {i+2} 行数值范围不合法。')
62     raw.append(dict(t=i, label_minute=(i+1)*10, price=price, load_kw=load,
63                   pv_kw=pv, source_sheet=sheet_name, source_row=i+2))
64     # 已冻结的左端点口径: 最后“次日 00:00”行循环接到本日 00:00—00:10。
65     # 电价、负荷、光伏三项一起重排; 这不是只改显示标签。
66     records = [dict(r, t=j, start_minute=j*10, end_minute=(j+1)*10,
67                    source_day_offset=-1 if j == 0 else 0)
68               for j, r in enumerate(raw[-1:] + raw[:-1])]
69     return records, dict(file=path.name, sheet=sheet_name, sha256=file_sha256(path),
70                          time_mapping='natural day, left endpoint, cyclic typical day')

```

## 4.2 第 4 节 \_\_Q1 确定性储能优化.py

源码/Q1\_第 4 节独立代码/第 4 节 \_\_Q1 确定性储能优化.py

```

1  """Q1 第 4 节 __Q1 确定性储能优化: 给定典型日负荷、光伏和电价, 求购电费用与微小充放电惩罚之和最小。
2
3  已冻结口径: 144 个十分钟时段; 左端点标签; 充、放电效率各 0.9;
4  储电量范围 1200—10800 kWh, 日初与日末均 6000 kWh; 最大功率 5000 kW。
5  本文件只负责建立、求解和核对线性规划, 读取与导出请运行“第 4 节 __Q1 运行与结果导出.py”。
6  2026-09-13 按用户要求加入 1e-6 充放电惩罚; 物理约束及两次求解边界不变。
7  """
8
9  from __future__ import annotations
10
11
12  import numpy as np
13  import scipy
14  from scipy.optimize import linprog
15  from scipy.sparse import lil_matrix
16
17
18  # 固定物理参数: 功率 kW 乘以时长 h 得到电量 kWh; 不得把二者混用。
19  N = 144
20  DT = 1 / 6
21  PMAX = 5000.0
22  QMAX = PMAX * DT
23  EMIN, EMAX, E0 = 1200.0, 10800.0, 6000.0
24  ETA_C = ETA_D = 0.9
25  THROUGHPUT_EPS = 1e-6 # 与最终整合稿一致, 仅用于优化, 不计入实际电费。
26
27
28  def solve(records: list[dict], previous_first_kwh=None) -> dict:
29      # records 必须已排成自然日 00:00—24:00; 本函数不偷偷移动时间标签。
30      # previous_first_kwh 为上一典型日遗留的 00:00—00:10 购电量。
31      # 先不传入该值求循环典型日, 再将所得首段量固定后重新求解。
32      price = np.array([r["price"] for r in records], dtype=float)
33      load = np.array([r["load_kw"] * DT for r in records], dtype=float)
34      pv = np.array([r["pv_kw"] * DT for r in records], dtype=float)
35
36      if len(records) != N or [(r["start_minute"], r["end_minute"]) for r in records] !=
37         ↳ [(10*t, 10*(t+1)) for t in range(N)]:
38          raise ValueError("Q1 records must use complete natural-day intervals")
39      # 决策向量共 721 维: 购电 g、充电 c、放电 d、弃光 s 各 144 维, 库存 E 为 145 维。
40      # g/c/d/s 的单位都是 kWh。E[t] 为段初库存, E[t+1] 为该段执行后的库存。
41      g0, c0, d0, s0, e0 = 0, N, 2 * N, 3 * N, 4 * N
42      nv = 5 * N + 1
43
44      # 目标为购电费 +THROUGHPUT_EPS* 总充放电量。Q1 不含紧急购电、日内违约费用,
45      # 也不加 Q2/Q3 的末端库存估值: 这里已用日初 = 日末的硬约束封闭库存。
46      objective = np.zeros(nv)
47      objective[g0:g0 + N] = price
48      objective[c0:c0 + N] = THROUGHPUT_EPS
49      objective[d0:d0 + N] = THROUGHPUT_EPS
50
51      # 变量边界: 购电、充放电、弃光非负; 弃光不超过本段光伏;
52      # 各段库存遵守上下限, 并将首尾两个库存状态都固定为 6000 kWh。

```

```

53     lower = np.zeros(nv)
54     upper = np.full(nv, np.inf)
55     upper[c0:c0 + N] = QMAX
56     upper[d0:d0 + N] = QMAX
57     upper[s0:s0 + N] = pv
58     lower[e0:e0 + N + 1] = EMIN
59     upper[e0:e0 + N + 1] = EMAX
60     lower[e0] = upper[e0] = E0
61     lower[e0 + N] = upper[e0 + N] = E0
62
63     # 当前 00:00—00:10 属于昨天已经锁定的合同, 不能被当天新合同覆盖。
64     if previous_first_kwh is not None:
65         if not np.isfinite(previous_first_kwh) or previous_first_kwh < 0:
66             raise ValueError("Invalid inherited purchase")
67         lower[0] = upper[0] = float(previous_first_kwh)
68     # 两组等式, 每组 144 行:
69     # 1)  $g + pv - s + d = load + c$ : 供给与用电守恒。
70     # 2)  $E[t+1] = E[t] + \text{ETA\_C} * c - d / \text{ETA\_D}$ : 电池内部库存递推。
71     # 放电  $d$  是在负荷侧收到的电量, 所以电池内部消耗量必须除以放电效率。
72     aeq = lil_matrix((2 * N, nv))
73     beq = np.zeros(2 * N)
74     for t in range(N):
75         aeq[t, g0 + t] = 1
76         aeq[t, c0 + t] = -1
77         aeq[t, d0 + t] = 1
78         aeq[t, s0 + t] = -1
79         beq[t] = load[t] - pv[t]
80
81         row = N + t
82         aeq[row, e0 + t + 1] = 1
83         aeq[row, e0 + t] = -1
84         aeq[row, c0 + t] = -ETA_C
85         aeq[row, d0 + t] = 1 / ETA_D
86
87     # 同一十分钟内充、放电共用时间:  $c/QMAX + d/QMAX \leq 1$ 。
88     # 等价于  $c+d \leq 5000/6$  kWh; 两项不能各自占满十分钟。
89     # 这是区间内分时充放电约束, 允许顺序充放, 不强行加入  $c*d=0$  的非线性式。
90     aub = lil_matrix((N, nv))
91     bub = np.ones(N)
92     for t in range(N):
93         aub[t, c0 + t] = 1 / QMAX
94         aub[t, d0 + t] = 1 / QMAX
95
96     # 保持原 HiGHS 求解器、变量次序和预处理选项, 以复现已验收结果。
97     result = linprog(
98         objective,
99         A_ub=aub.tocsr(),
100         b_ub=bub,
101         A_eq=aeq.tocsr(),
102         b_eq=beq,
103         bounds=list(zip(lower, upper)),
104         method="highs",
105         options={"presolve": True},
106     )
107     if not result.success:
108         raise RuntimeError(result.message)
109
110     # 按原变量块拆出解, 并计算充电、放电、空闲的最少时间份额。
111     x = result.x
112     g = x[g0:g0 + N]
113     c = x[c0:c0 + N]
114     d = x[d0:d0 + N]
115     s = x[s0:s0 + N]
116     energy = x[e0:e0 + N + 1]
117     alpha_c = c / QMAX
118     alpha_d = d / QMAX
119     alpha_idle = 1 - alpha_c - alpha_d
120
121     # 从解重新计算守恒、功率、库存和费用残差。
122     # simultaneous_interval_count 只统计是否出现同段先充后放, 不能把它当成违约。
123     tol = 1e-7
124     checks = {
125         "balance_max_kwh": float(np.max(np.abs(g + pv - s + d - load - c))),

```

```

126     "energy_recursion_max_kwh": float(
127         np.max(np.abs(energy[1:] - energy[:-1] - ETA_C * c + d / ETA_D))
128     ),
129     "time_share_violation": float(max(0.0, np.max(alpha_c + alpha_d - 1))),
130     "energy_bound_violation_kwh": float(max(0.0, EMIN - np.min(energy), np.max(energy) -
131         ↳ EMAX)),
132     "power_bound_violation_kw": float(max(0.0, np.max(c) / DT - PMAX, np.max(d) / DT -
133         ↳ PMAX)),
134     "negative_flow_violation_kwh": float(max(0.0, -np.min(g), -np.min(c), -np.min(d),
135         ↳ -np.min(s))),
136     "curtail_bound_violation_kwh": float(max(0.0, np.max(s - pv))),
137     "terminal_error_kwh": float(max(abs(energy[0] - E0), abs(energy[-1] - E0))),
138     "objective_recompute_error_yuan": float(abs(price @ g + THROUGHPUT_EPS * np.sum(c + d) -
139         ↳ result.fun)),
140     "simultaneous_interval_count": int(np.sum((c > 1e-6) & (d > 1e-6))),
141 }
142 for key, value in checks.items():
143     if key != "simultaneous_interval_count" and value > tol:
144         raise AssertionError((key, value))
145
146 # 从 6000 kWh 开始逐段执行求得的充放电量，再独立递推库存。
147 # 规划库存仅用来核对；不能在每段开始把实际库存重置为规划给定值。
148 executed_energy = [E0]
149 for t in range(N):
150     executed_energy.append(executed_energy[-1] + ETA_C * c[t] - d[t] / ETA_D)
151 checks['sequential_execution_error_kwh'] =
152     ↳ float(np.max(np.abs(np.array(executed_energy) - energy)))
153 assert checks['sequential_execution_error_kwh'] < 1e-7
154 # 保留每段输入、动作、库存和费用，便于在 CSV 中逐行检查。
155 schedule = []
156 for t, source in enumerate(records):
157     if c[t] > 1e-6 and d[t] > 1e-6:
158         mode = " 分时充放"
159     elif c[t] > 1e-6:
160         mode = " 充电"
161     elif d[t] > 1e-6:
162         mode = " 放电"
163     else:
164         mode = " 空闲"
165     schedule.append(
166         {
167             **source,
168             "t": t + 1,
169             "load_kwh": float(load[t]),
170             "pv_kwh": float(pv[t]),
171             "buy_kwh": float(g[t]),
172             "charge_kwh": float(c[t]),
173             "discharge_kwh": float(d[t]),
174             "curtail_kwh": float(s[t]),
175             "e_start_kwh": float(executed_energy[t]),
176             "e_end_kwh": float(executed_energy[t + 1]),
177             "state_before_slot": t,
178             "state_after_slot": t + 1,
179             "execution_rule": "apply solved commands, then calculate next inventory",
180             "charge_share": float(alpha_c[t]),
181             "discharge_share": float(alpha_d[t]),
182             "idle_share": float(alpha_idle[t]),
183             "mode": mode,
184             "cost_yuan": float(price[t] * g[t]),
185         }
186     )
187
188 # 汇总数据与 144 段明细一起返回，便于与冻结前结果逐项比较。
189 return {
190     "method": "LP with within-interval time-sharing exclusivity",
191     "solver": "scipy.optimize.linprog(method='highs')",
192     "scipy_version": scipy.__version__,
193     "status": int(result.status),
194     "message": result.message,
195     "objective_yuan": float(price @ g), # 保留历史接口：实际购电费。
196     "optimization_objective_yuan": float(result.fun),
197     "throughput_penalty_yuan": float(THROUGHPUT_EPS * np.sum(c + d)),
198     "throughput_penalty_coefficient": THROUGHPUT_EPS,

```

```

194         "buy_kwh": float(np.sum(g)),
195         "charge_kwh": float(np.sum(c)),
196         "discharge_kwh": float(np.sum(d)),
197         "curtail_kwh": float(np.sum(s)),
198         "min_energy_kwh": float(np.min(energy)),
199         "max_energy_kwh": float(np.max(energy)),
200         "final_energy_kwh": float(energy[-1]),
201         "checks": checks,
202         "schedule": schedule,
203     }
204

```

### 4.3 第 4 节 \_Q1 运行与结果导出.py

源码/Q1\_第 4 节独立代码/第 4 节 \_Q1 运行与结果导出.py

```

1  """Q1 唯一运行入口：读取附件 1、求解、输出明细、自动核验。
2
3  直接运行本文件即可。默认写入新的“复跑结果/时间戳”目录，不覆盖冻结结果。
4  本版已按最终整合稿加入 1e-6 充放电惩罚；购电费用与惩罚分开导出。
5  """
6  from pathlib import Path
7  import argparse
8  import csv
9  import datetime
10 import json
11 import sys
12 import numpy as np
13 from 第 2 至 4 节 _Q1 数据与时间对齐 import ROOT, read_records, check_frozen_files
14 from 第 4 节 _Q1 确定性储能优化 import solve
15
16
17 def write_json(path, data):
18     Path(path).write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False),
19                             ↪ encoding='utf-8')
20
21 def write_csv(path, rows):
22     """UTF-8 带签名，便于 Windows Excel 直接打开中文表格。"""
23     with Path(path).open('w', encoding='utf-8-sig', newline='') as f:
24         writer = csv.DictWriter(f, fieldnames=list(rows[0]))
25         writer.writeheader()
26         writer.writerows(rows)
27
28
29 def run(output=None):
30     check_frozen_files()
31     records, source = read_records()
32     out = Path(output) if output is not None else
33     ↪ ROOT/'复跑结果'/datetime.datetime.now().strftime('%Y%m%d_%H%M%S_%f')
34     if not out.is_absolute():
35         out = ROOT/out
36     out.mkdir(parents=True, exist_ok=False)
37
38     # 第一次求解重复典型日，取得上一日合同遗留到 00:00—00:10 的购电量。
39     # 第二次把这一数量固定，完全沿用已验收 Q1 的两次求解顺序。
40     periodic = solve(records)
41     inherited = float(periodic['schedule'][0]['buy_kwh'])
42     result = solve(records, previous_first_kwh=inherited)
43     result['inherited_first_kwh'] = inherited
44     result['first_interval_contract_origin'] = 'previous day 00:00, repeated typical-day
45     ↪ contract'
46     result['new_contract_effective_from'] = '00:10'
47     for j, row in enumerate(result['schedule']):
48         row['issue_day_offset'] = -1 if j == 0 else 0
49         row['issue_time'] = '00:00'
50         row['effective_from'] = 'previous day 00:10' if j == 0 else '00:10'
51
52     # 自然日费用表按 00:00—24:00 排列；新合同按 00:10—次日 00:10 排列。
53     # 典型日周期重复，因此合同最后一格使用循环日的 00:00 首段购电量。

```

```

52 contract = result['schedule'][1:] + [result['schedule'][0]]
53 issued = [dict(contract_slot=i, interval_start_minute=(i+1)*10, interval_end_minute=(i+2)*10,
54               buy_kwh=row['buy_kwh']) for i, row in enumerate(contract)]
55 write_csv(out/'日内明细.csv', result['schedule'])
56 write_csv(out/'日前购电合同.csv', issued)
57 write_json(out/'完整结果.json', result)
58 summary = {k:v for k,v in result.items() if k != 'schedule'}
59 price = np.array([r['price'] for r in records])
60 reference = np.array([(r['load_kw']-r['pv_kw'])/6 for r in records])
61 summary['no_battery_cost_yuan'] = float(price @ np.maximum(reference, 0))
62 summary['input_source'] = source
63 write_json(out/'结果汇总.json', summary)
64
65 # 每次运行后独立复算物理、电费和惩罚，并记录与旧 144 段结果的差异。
66 verification = verify(out)
67 write_json(out/'第 9 节_Q1 结果校验.json', verification)
68 print(f'Q1 运行与校验完成：购电费 {summary["objective_yuan"]:.9f} 元；结果位于 {out}')
69 return out
70
71
72 def main():
73     parser = argparse.ArgumentParser(description='运行含 1e-6 充放电惩罚的 Q1 典型日调度')
74     parser.add_argument('--output', type=Path, help='新输出目录；省略则自动新建复跑目录')
75     args = parser.parse_args()
76     run(args.output)
77
78
79 def verify(output):
80     check_frozen_files()
81     output = Path(output)
82     new = json.loads((output/'完整结果.json').read_text(encoding='utf-8'))
83     rows = new['schedule']; records, _ = read_records()
84     checks = []
85
86     def check(name, passed, **details):
87         checks.append(dict(name=name, passed=bool(passed), **details))
88         if not passed:
89             raise AssertionError(f'Q1 校验未通过：{name}; {details}')
90
91     from 第 4 节_Q1 确定性储能优化 import THROUGHPUT_EPS
92     check('惩罚系数与最终整合稿一致', THROUGHPUT_EPS == 1e-6)
93     check('自然日144个左端点时段完整', len(rows) == 144 and [(r['start_minute'], r['end_minute'])
94     ↪ for r in rows] == [(10*k, 10*(k+1)) for k in range(144)])
95     for field in ['price', 'load_kw', 'pv_kw']:
96         check('Excel输入逐段一致_'+field, np.array_equal([r[field] for r in rows], [r[field] for
97         ↪ r in records]))
98
99     # 能量均为 kWh；按原始功率/6 独立恢复负荷、光伏。
100     g, c, d, s, e0, e1, p, load, pv = np.array([r[k] for r in rows] for k in ['buy_kwh', 'charge_kwh', 'd',
101     ↪ 'ischarge_kwh', 'curtail_kwh', 'e_start_kwh', 'e_end_kwh', 'price', 'load_kw', 'pv_kw'])
102     balance = float(np.max(abs(g+pv/6-s-d-load/6-c)))
103     inventory = float(np.max(abs(e1-e0-.9*c+d/.9)))
104     check('全段能量平衡', balance < 1e-7, maximum_error=balance)
105     check('全段库存递推', inventory < 1e-7, maximum_error=inventory)
106     check('库存连续且首尾6000', np.max(abs(e1[:-1]-e0[1:]))<1e-7 and abs(e0[0]-6000)<1e-7 and
107     ↪ abs(e1[-1]-6000)<1e-7)
108     check('全部库存满足容量上下限', min(e0.min(), e1.min())>=1200-1e-7 and
109     ↪ max(e0.max(), e1.max())<=10800+1e-7)
110     check('充放电共用功率上限', np.max(c+d)<=5000/6+1e-7)
111     check('购电充放电弃光非负且弃光不超过光伏', min(g.min(), c.min(), d.min(), s.min())>=-1e-7 and
112     ↪ np.max(s-pv/6)<1e-7)
113     check('全段及总费用独立复算', np.max(abs(p*g-np.array([r['cost_yuan'] for r in rows])))<1e-8
114     ↪ and abs(float(p@g)-new['objective_yuan'])<1e-8)
115     penalty = 1e-6*float(np.sum(c+d))
116     check('吞吐惩罚与优化目标独立复算', abs(new['throughput_penalty_yuan']-penalty)<1e-9 and
117     ↪ abs(new['optimization_objective_yuan']-(float(p@g)+penalty))<1e-8)
118     check('首段继承原合同', abs(g[0]-new['inherited_first_kwh'])<1e-8 and
119     ↪ rows[0]['issue_day_offset']==-1 and all(r['issue_day_offset']==0 for r in rows[1:]))
120
121     with (output/'日前购电合同.csv').open(encoding='utf-8-sig', newline='') as f:
122         contract = list(csv.DictReader(f))
123     check('新合同从00:10开始连续144段', len(contract)==144 and [int(r['interval_start_minute'])
124     ↪ for r in contract]==list(range(10, 1441, 10)))

```

```

115     check('合同购电量与自然日循环一致', np.array_equal(np.array([float(r['buy_kwh']) for r in
    ↪ contract]),np.r_[g[1:],g[0]]))
116     return dict(passed=True, count=len(checks), checks=checks,
    ↪ objective_yuan=new['objective_yuan'],
117                   historical_version_comparison="not part of minimal computation",
118                   scope='加入1e-6吞吐惩罚的新Q1; 独立复算物理、实际电费及优化目标, 不要求沿用旧等价计划')
119
120
121
122 if __name__ == '__main__':
123     if hasattr(sys.stdout, 'reconfigure'):
124         sys.stdout.reconfigure(encoding='utf-8')
125     main()

```

#### 4.4 第 5 节 \_Q2 分位数预测与滚动调度.py

源码/Q2\_第 5 节独立代码/第 5 节 \_Q2 分位数预测与滚动调度.py

```

1  """Q2 预测与第 5 节 _Q2 分位数预测与滚动调度: 历史信息前缀、每日分位数预测、三日前瞻与即时储能。
2
3  正式采用一月 45 组中按同分规则选出的树模型、分位数 0.8、保护系数 0。
4  本文件保留已验收的数值计算; 运行入口、45 组比较和第 9 节 _Q2 版本完整性检查由其他文件负责。
5  """
6  import os,sys,json,hashlib,datetime,time
7  from pathlib import Path
8  from dataclasses import dataclass
9  os.environ.setdefault('OMP_NUM_THREADS','1')
10 os.environ.setdefault('OPENBLAS_NUM_THREADS','1')
11 ROOT=Path(__file__).resolve().parent
12
13 import numpy as np
14 from scipy.optimize import linprog
15 from scipy.sparse import coo_matrix
16 from sklearn.ensemble import HistGradientBoostingRegressor
17 from sklearn.linear_model import QuantileRegressor
18 from sklearn.preprocessing import StandardScaler
19 from threadpoolctl import threadpool_limits
20 import joblib
21 # 五个预测分位数与三个保护系数构成原先固定的候选集合, 不在测试期扩展。
22 TAUS=np.array([.5,.65,.8,.9,.95]); BETAS=[0.,.5,1.]
23 EMIN=1200.;EMAX=10800.;ETA=.9;CAP=5000/6;EPS=1e-6
24 JAN_END=31*144;FORMAL_END=365*144
25 START=datetime.datetime(2025,1,1)
26
27 def dump(path,obj):
28     p=Path(path);p.parent.mkdir(parents=True,exist_ok=True)
29     p.write_text(json.dumps(obj,ensure_ascii=False,indent=2,default=lambda x:x.item() if
    ↪ isinstance(x,np.generic) else str(x),allow_nan=False),encoding='utf-8')
30
31 def digest(a):return hashlib.sha256(np.ascontiguousarray(a).tobytes()).hexdigest()
32
33 # 每次模型只能收到截止指定时刻的只读前缀, 无法索引后续真实值。
34 @dataclass(frozen=True)
35 class Prefix:
36     net:np.ndarray
37     pv:np.ndarray
38     cutoff:int
39     def labels(self,indices):
40         ix=np.asarray(indices, dtype=int)
41         if ix.min()<1 or ix.max()>self.cutoff:raise ValueError('Training target outside observed
    ↪ prefix / initialization proxy')
42         return self.net[ix]
43
44 # Excel 与 JSON 的对应关系在独立核验中逐值检查; 首段典型日代理不作为训练答案。
45 class Data:
46     def __init__(self):
47         p=ROOT/'输入/inputs.json'
48         self.raw=json.loads(p.read_text(encoding='utf-8'))
49         for source in self.raw['sources'].values():
50             local=ROOT/'输入'/Path(source['path']).name

```

```

51         if not local.exists() or
           ↪ hashlib.sha256(local.read_bytes()).hexdigest()!=source['sha256']:
52             raise ValueError('Copied Excel changed or missing: regenerate audited inputs')
53         rows=self.raw['q1_records'];order=rows[-1:]+rows[:-1]
54         self.price=np.array([r['price'] for r in order])
55         self.reference=np.array([(r['load_kw']-r['pv_kw'])/6 for r in order])
56         rawpv=np.array(self.raw['pv_kw']).ravel()
57         self.net=np.r_[self.reference[0],(np.array(self.raw['load_kw']).ravel()-rawpv)/6]
58         self.pv=np.r_[rows[-1]['pv_kw'],rawpv]
59
60     def prefix(self,cutoff):
61         if not 0<=cutoff<len(self.net):raise ValueError('Observation cutoff outside supplied
           ↪ history')
62         n=self.net[:cutoff+1].copy();p=self.pv[:cutoff+1].copy()
63         n.flags.writeable=False;p.flags.writeable=False
64         return Prefix(n,p,int(cutoff))
65
66
67 # 输入特征按原历史预测时刻构造; 同钟点统计、滚动均值不能包含之后的真实量。
68 def net_features(view,origin,targets,pv_pred=None,pv_bias=0.):
69     if view.cutoff!=origin:raise ValueError('Feature view must end at historical decision time')
70     t=np.asarray(targets);j=t%144;lead=(t-origin-1)//144
71     n=len(t);clock=2*np.pi*j/144
72     # Latest same-clock observations, including the current slot only when already known.
73     latest=origin-((origin-j)%144)
74     rows=np.maximum(latest[None,:]-144*np.arange(7)[: ,None],0)
75     h=view.net[rows]
76     dow=((t//144)+2)%7
77     x=np.column_stack([np.sin(clock),np.cos(clock),lead,*[(dow==d).astype(float) for d in
           ↪ range(6)],
78                     h[0]/100,h[:3].mean(0)/100,h.mean(0)/100,
79                     np.full(n,view.net[max(1,origin-143):origin+1].mean()/100),
80                     np.full(n,view.net[max(1,origin-431):origin+1].mean()/100)])
81     if not np.isfinite(x).all():raise ValueError('Non-finite historical features')
82     return x
83
84 # 对同一目标的五个分位数做等权 PAVA 单调投影, 不把它们当成五条联合路径。
85 def rearrange(q):
86     # Equal-weight isotonic least-squares projection (PAVA), not sorting values.
87     a=np.asarray(q).copy()
88     for row in a:
89         values=[];weights=[];sizes=[]
90         for z in row:
91             values.append(z);weights.append(1);sizes.append(1)
92             while len(values)>1 and values[-2]>values[-1]:
93                 w=weights[-2]+weights[-1];z=(weights[-2]*values[-2]+weights[-1]*values[-1])/w
94                 s=sizes[-2]+sizes[-1];values[-2:]=[z];weights[-2:]=[w];sizes[-2:]=[s]
95         row[:]=np.repeat(values,sizes)
96     return a
97
98 # 三种预测法共用成熟标签; 标准化仅拟合当次训练集, 目标缩放后再还原 kwh。
99 def fit_predict(kind,X,y,future,artifact=None):
100     if kind=='history':
101         q=rearrange(np.quantile(y.reshape(-1,432),TAUS,axis=0).T)
102         if artifact is not None:joblib.dump(dict(kind=kind,q=q),artifact,compress=3)
103         return q
104     scale=StandardScaler().fit(X);a=scale.transform(X);b=scale.transform(future)
105     def one(tau):
106         if kind=='linear':m=QuantileRegressor(quantile=float(tau),alpha=.002,solver='highs')
107         else:m=HistGradientBoostingRegressor(loss='quantile',quantile=float(tau),max_iter=60,max_
           ↪ _leaf_nodes=7,
108             min_samples_leaf=30,l2_regularization=1.,early_stopping=False,random_state=20260911)
109         m.fit(a,y/100);return m.predict(b)*100,m
110     with threadpool_limits(limits=1):
111         from concurrent.futures import ThreadPoolExecutor
112         with ThreadPoolExecutor(max_workers=3) as pool:preds=list(pool.map(one,TAUS))
113     if artifact is not None:joblib.dump(dict(kind=kind,scaler=scale,models=[z[1] for z in
           ↪ preds],target_scale=100.,taus=TAUS),artifact,compress=3)
114     return rearrange(np.column_stack([z[0] for z in preds]))
115
116 # 每天 00:00 重训; 仅选最后 432 个目标已全部发生的历史起点, 最多 30 个。
117 def q2_origin(data,day,kind,features=None,jan_only=False,artifact=None):
118     o=day*144;cut=o if not jan_only else min(o,JAN_END-1)

```

```

119 latest=(cut-432)//144
120 origins=list(range(max(7,latest-29),latest+1))
121 if not origins:raise ValueError('No mature Q2 origin')
122 view=data.prefix(cut);xs=[];ys=[]
123 for d in origins:
124     oo=d*144;t=np.arange(oo+1,oo+433)
125     xs.append(features[d] if features is not None else
126             ↪ net_features(data.prefix(oo),oo,t));ys.append(view.labels(t))
127 current=net_features(data.prefix(o),o,np.arange(o+1,o+433))
128 q=fit_predict(kind,np.vstack(xs),np.concatenate(ys),current,artifact=artifact)
129 return q,dict(origin=o,first_origin=origins[0]*144,last_origin=origins[-1]*144,
130             ↪ max_training_target=origins[-1]*144+432,training_cutoff=cut,rows=len(origins)*
131             ↪ 432,
132             ↪ predictor_kind=kind,features_sha=digest(current),training_features_sha=digest(
133             ↪ np.vstack(xs)),training_labels_sha=digest(np.concatenate(ys)),target_first
134             ↪ =o+1,target_last=o+432)
135
136 # Q2 已锁定的普通合同不可日内修改，真实费用只含普通购电与 5 倍紧急购电。
137 def fee(g,g0,p):
138     if not np.array_equal(np.asarray(g),np.asarray(g0)):raise ValueError('Q2 cannot change an
139     ↪ existing contract')
140     return p*g
141
142 # 每内部 kWh 放电后可提供 ETA kWh，因此末端估值为最低普通电价乘放电效率。
143 def terminal_value(p):
144     return float(np.min(p))*ETA
145
146 # LP 变量块依次为购电、充电、放电、未利用电量、末端库存，每块 432 个。
147 # 等式约束：g+b=q+c+u；E_next=E+ETA*c-b/ETA。
148 # 功率约束：c+b<=5000/6；目标：购电费-p_min*ETA*E_end+ 微小充放电惩罚。
149 def plan(q,p,E):
150     q=np.asarray(q);p=np.asarray(p);n=len(q);nv=5*n;terminal=terminal_value(p)
151     i=np.arange(n);rr=[i,i,i,i,i+n,i+n,i+n,(i+n)[1:]]
152     cc=[i,i+n,i+2*n,i+3*n,i+4*n,i+n,i+2*n,(i+4*n)[-1]]
153     vv=[np.ones(n),-np.ones(n),np.ones(n),-np.ones(n),np.ones(n),-ETA*np.ones(n),np.ones(n)/ETA,
154     ↪ -np.ones(n-1)]
155     a=coo_matrix((np.concatenate(vv),(np.concatenate(rr),np.concatenate(cc))),shape=(2*n,nv)).to
156     ↪ csr()
157     rhs=np.r_[q,np.zeros(n)];rhs[n]=E
158     ur=[i,i];uc=[i+n,i+2*n];uv=[np.ones(n),np.ones(n)];ub=[np.full(n,CAP)]
159     obj=np.r_[p,np.full(2*n,EPS),np.zeros(2*n)];obj[5*n-1]=-terminal
160     u=coo_matrix((np.concatenate(uv),(np.concatenate(ur),np.concatenate(uc))),shape=(n,nv)).tocs
161     ↪ r()
162     bounds=np.zeros((nv,2));bounds[:,1]=np.inf;bounds[n:3*n,1]=CAP;bounds[4*n:5*n]=[EMIN,EMAX]
163     res=linprog(obj,A_ub=u,b_ub=np.concatenate(ub),A_eq=a,b_eq=rhs,bounds=bounds,method='highs')
164     if not res.success:raise RuntimeError(res.message)
165     z=res.x[:5*n].reshape(5,n)
166     if np.max(abs(a@res.x-rhs))>1e-6:raise ValueError('LP physical failure')
167     return z
168
169 # 当前段净余量为正则充电，为负则放电，不同时充放。
170 # 正式 beta=0 时保护线等于 1200kWh 硬下限，不再额外追踪规划参考库存。
171 def control(net,g,z,E,beta):
172     r=EMIN+beta*(np.asarray(z)-EMIN);delta=np.asarray(g)-np.asarray(net)
173     c=np.minimum(np.maximum(delta,0),np.minimum(CAP,np.maximum((EMAX-E)/ETA,0)))
174     b=np.minimum(np.maximum(-delta,0),np.minimum(CAP,np.maximum(ETA*(E-r),0)))
175     h=np.maximum(-delta,0)-b;unused=np.maximum(delta,0)-c
176     end=E+ETA*c-b/ETA
177     return c,b,h,unused,end,r
178
179 # 从 1 月 1 日 6000kWh 连续执行固定典型日购电策略，避免 1 月 20 日或 2 月 1 日重置库存。
180 def warmup(data,until=JAN_END):
181     E=6000.;trace=[]
182     for t in range(until):
183         g=max(data.reference[t%144],0);c,b,h,u,end,r=control(data.net[t],g,EMIN,E,0.)
184         trace.append([t,data.net[t],g,g,c,b,h,u,E,end,data.price[t%144]*g,5*data.price[t%144]*h])
185     E=float(end)
186     return E,max(data.reference[0],0),EMIN,np.asarray(trace)
187
188 # 逐段复算物理和账单；末端估值只用于比较，不从实际账单退款。
189 def summary(trace,price):
190     a=np.asarray(trace);t=a[:,0].astype(int);p=price[t%144]

```

```

184     n,g,g0,c,b,h,u,e0,e1,pc,ec=a[:,1:].T
185     errors={'balance':float(np.max(abs(g+b+h-n-c-u))), 'inventory':float(np.max(abs(e1-e0-ETA*c+b_j
    ↪ /ETA))),
186         'continuity':float(np.max(abs(e1[:-1]-e0[1:]))) , 'fee':float(np.max(abs(pc-fee(g,g0,p)))) ,
187         'emergency_fee':float(np.max(abs(ec-5*p*h)))}
188     if max(errors.values())>1e-6 or e0.min()<EMIN-1e-6 or e1.max()>EMAX+1e-6 or
    ↪ (c+b).max()>CAP+1e-6:
189         raise ValueError(str(errors))
190     cost=float(np.sum(pc+ec));end=float(e1[-1])
191     return dict(cost_yuan=cost,contract_cost_yuan=float(pc.sum()),emergency_cost_yuan=float(ec.s_j
    ↪ um()),
192         emergency_kwh=float(h.sum()),unused_kwh=float(u.sum()),start_energy=float(e0[0]),end_energ_j
    ↪ y=end,
193         inventory_adjusted_cost_yuan=cost-terminal_value(price)*end,slots=len(a),checks=errors,
194         monthly=[{'month':m, 'cost':float((pc+ec)[np.array([(START+datetime.timedelta(minutes=int(k_j
    ↪ )*10)).month==m for k in t])).sum()}) for m in
    ↪ range(1,13)])
195
196 # 每日先结算当前遗留段,再用其末库存规划下一段起生效的 144 段合同。
197 # 年末仍完整规划 432 段,真实费用只统计至次年 1 月 1 日 00:00 之前。
198 def q2_play(data,forecasts,tau_index,beta,jan=False):
199     """Every origin plans 432 slots, including late December; only 144 are locked."""
200     trace=[];plans=[]
201     start=19*144+1 if jan else JAN_END
202     end=30*144+1 if jan else FORMAL_END
203     E,carry,cz,warm=warmup(data,until=start)
204     garr=np.full(end+145,np.nan);zarr=np.full(end+145,np.nan)
205     if not jan:garr[start]=carry;zarr[start]=cz
206     def decision(origin,energy):
207         q=forecasts[origin//144];targets=origin+np.arange(1,433)
208         if q.shape!=(432,5) or not np.isfinite(q).all():raise ValueError('Missing full forecast')
209         p=data.price[targets%144];a=plan(q[:,tau_index],p,energy)
210         garr[origin+1:origin+145]=a[0,:144];zarr[origin+1:origin+145]=a[4,:144]
211         plans.append(dict(origin=origin,initial_energy=energy,targets=targets,forecast=q.copy(),j
    ↪ values=a,terminal_value=terminal_value(p)))
212     if jan:decision(start-1,E)
213     for t in range(start,end):
214         g=garr[t];z=zarr[t]
215         if not np.isfinite(g+z):raise ValueError('Uncommitted purchase or reference inventory')
216         c,b,h,u,ee,r=control(data.net[t],g,z,E,beta);p=data.price[t%144]
217         trace.append([t,data.net[t],g,g,c,b,h,u,E,ee,p*g,5*p*h]);E=float(ee)
218         if t%144==0 and t+1<end:decision(t,E)
219     a=np.asarray(trace);s=summary(a,data.price)
220     s.update(tau=float(TAUS[tau_index]),beta=float(beta),terminal_coefficient=terminal_value(dat_j
    ↪ a.price),horizon=432,
221         selection_score=s['inventory_adjusted_cost_yuan']+EPS*float(a[:,4:6].sum()))
222     return a,s,plans

```

## 4.5 第 5 节 \_Q2 四十五组方案对比.py

源码/Q2\_第 5 节独立代码/第 5 节 \_Q2 四十五组方案对比.py

```

1     """ 一月 45 组联合选型及正式期对照;正式期结果绝不反向改变选择。
2
3     选型在主入口的一月阶段调用。对照在正式主轨迹生成之后调用,使用同批
4     每日重训预测,分别重新规划和连续传递库存;对照费用仅作结果解释。
5     """
6     from 第 5 节 _Q2 分位数预测与滚动调度 import *
7     KINDS=['history','linear','tree']
8     TIE_TOL=1e-6
9
10    def select_main(candidates):
11        """ 评分容差内按原顺序破同分:应急量、分位数、保护系数、模型顺序。"""
12        minimum=min(c['selection_score'] for c in candidates)
13        tied=[c for c in candidates if c['selection_score']<=minimum+TIE_TOL]
14        return min(tied,key=lambda c:(c['emergency_kwh'],c['tau'],c['beta'],KINDS.index(c['kind'])))
15
16    def compare_schemes(out,data,qfull,candidates,selected):
17        # 延后导入输出函数,避免主入口与比较模块循环导入。

```

```

18 from 第5节_Q2运行与结果导出 import save_play, csv_write
19 out=Path(out); folder=out/'第5节_Q2 四十五组方案对比'; folder.mkdir()
20 main=json.loads((out/'summary.json').read_text(encoding='utf-8'))
21 protocol=dict(main_configuration=dict(kind=selected['kind'], tau=selected['tau'], beta=selected
    ↳ d['beta']),
22             selection_data='January only; original deterministic tie rule',
23             formal_role='Descriptive comparison only; cannot alter selected configuration',
24             alternatives=[dict(directory='原方案_065_05', tau=.65, beta=.5), dict(directory='同分对照_080
    ↳ _05', tau=.8, beta=.5)])
25 dump(folder/'对比规则.json', protocol)
26 alternatives=[]
27 for spec in protocol['alternatives']:
28     # 复用预测不复用控制轨迹：每个组合有自己连续的库存和每日锁定合同。
29     trace, s, plans=q2_play(data, qfull, list(TAUS).index(spec['tau']), spec['beta'])
30     save_play(folder/spec['directory'], trace, s, plans, data.price, spec['beta'])
31     alternatives.append(dict(**spec, summary=s))
32 baseline=alternatives[0]['summary']; saving=baseline['cost_yuan']-main['cost_yuan']
33 result=dict(main_configuration=protocol['main_configuration'], main_summary=main, alternatives
    ↳ =alternatives,
34             saving_vs_old_yuan=saving, saving_vs_old_percent=100*saving/baseline['cost_yuan'])
35 dump(folder/'对比结果.json', result)
36 monthly=[]
37 for m in range(2, 13):
38     mc=next(x['cost'] for x in main['monthly'] if x['month']==m)
39     bc=next(x['cost'] for x in baseline['monthly'] if x['month']==m)
40     monthly.append(dict(month=m, main_080_beta0=mc, old_065_beta05=bc, saving=bc-mc))
41 csv_write(folder/'月度对比.csv', monthly)
42 lines=['# Q2: 45 组选型与原第5节_Q2 四十五组方案对比', '',
43        '** 正式采用：浅层分位数树、 $\tau=0.8$ 、 $\beta=0$ 。 ** 这是用户确认采用的一月 45 组最低评分候选。',
44        '', '一月  $\tau=0.8$  的  $\beta=0$  与  $\beta=0.5$  并列最低，按原同分规则取  $\beta=0$ 。0.8 来自既定有限网格的一月回放，
    ↳ 不是理论推导出的普适最优分位数。用户是在查看已有对照后确认采用本方案，
    ↳ 因此本表属于已有数据上的事后对照报告，不能称为未被查看过的独立测试。代码的训练、
    ↳ 特征和一月评分仍严格遵守当时的信息边界。',
45        '', '## 正式期对照（2025 年 2 月 1 日至 12 月 31 日）', '',
46        '| 方案 | 实际费用/元 | 应急购电费用/元 | 期末库存/kWh |', '|---|---|---|---|']
47 for name, s in [(('现正式方案：0.8、 $\beta=0$ ', main), ('原方案：0.65、 $\beta=0.5$ ', baseline), ('一月同分对照：0.8、
    ↳  $\beta=0.5$ ', alternatives[1]['summary']))]:
48     lines.append(f"| {name} | {s['cost_yuan']:.6f} | {s['emergency_cost_yuan']:.6f} |
    ↳ {s['end_energy']:.6f} |")
49 lines+=['', f'现正式方案比原方案节省 **{saving:.2f}元 ({100*saving/baseline["cost_yuan"]:.4f}%)
    ↳ **；11个月中有{sum(r["saving"]>0 for r in monthly)}个月费用更低。',
50        '', ' $\tau=0.8$ 、 $\beta=0.5$  的正式期费用略低于  $\beta=0$ ，但它不改变一月破同分结果。所有方案使用相同初始库存、价格、
    ↳ 432段前瞻、 $p_n$  末端价值和同批每日重训预测。实际账单不扣末端库存价值；
    ↳ 一月选型评分扣除库存价值并加入微小充放电项。',
51        '', '## 一月 45 组完整结果', '',
52        '评价共1584段（1月20日00:10至1月31日00:10），每组独立传递库存。下表按原始评分排序；
    ↳ 顺序号不代表容差内的优劣，同分处理以正文规则为准。',
53        '', '| 顺序 | 模型 |  $\tau$  |  $\beta$  | 一月实际费用/元 | 一月选型评分/元
    ↳ |', '|---|---|---|---|---|---|']
54 for rank, c in enumerate(sorted(candidates, key=lambda c: c['selection_score']), 1):
55     lines.append(f"| {rank} | {c['kind']} | {c['tau']:.6f} | {c['beta']:.6f} |
    ↳ {c['cost_yuan']:.6f} | {c['selection_score']:.6f} |")
56 lines+=['', '机器可读结果见 `q2_candidate_summary.csv` 和
    ↳ `第5节_Q2四十五组方案对比/对比结果.json`；月度对比见 `第5节_Q2四十五组方案对比/月度对比.csv`。
    ↳ ', '']
57 (out/'45 组对比报告.md').write_text('\n'.join(lines), encoding='utf-8')
58 print('45 组比较已整合；与原方案正式期对照完成', saving, flush=True)
59 return result

```

## 4.6 第5节\_Q2运行与结果导出.py

源码/Q2\_第5节独立代码/第5节\_Q2运行与结果导出.py

```

1 """ 第5节_Q2运行与结果导出成品：一月45组比较 → 冻结0.8方案 → 每日重训 → 全期调度 → 对比与验收。
2
3 默认每次新建复跑目录，不覆盖冻结结果。全部比较、检查均由本入口自动调用。
4 """
5 import os, sys, csv, argparse, platform
6 os.environ['OMP_NUM_THREADS']='1'; os.environ['OPENBLAS_NUM_THREADS']='1'

```

```

7 sys.stdout.reconfigure(encoding='utf-8')
8 from 第5节_Q2_分位数预测与滚动调度 import *
9 from 第5节_Q2_四十五组方案对比 import select_main, compare_schemes
10
11 KINDS=['history','linear','tree'];TIE_TOL=1e-6
12
13 def sha(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()
14 def clock(k):return (START+datetime.timedelta(minutes=int(k)*10)).isoformat(sep=' ')
15 def csv_write(path,rows):
16     rows=list(rows)
17     with Path(path).open('w',encoding='utf-8-sig',newline='') as f:
18         writer=csv.DictWriter(f,fieldnames=list(rows[0]));writer.writeheader();writer.writerows(
19             ↪ rows)
20
21 # 在看正式期费用前记录候选集合、模型参数、时标及数据/代码指纹。
22 def configuration():
23     return dict(schema='Q2_FROZEN_FULLL432_PETA_080_BETA0_v1',tau=.8,beta=0.,selection_scope='all
24         ↪ 45 January candidates',taus=TAUS.tolist(),betas=BETAS,
25         kinds=KINDS,horizon=432,locked_slots=144,eta=ETA,capacity=[EMIN,EMAX],power_kw=5000,
26         terminal='p_min * eta',terminal_at_year_end='same coefficient and full 432-slot forecast',
27         initial_energy=6000.,formal_start=JAN_END,formal_end_exclusive=FORMAL_END,
28         training='daily; at most 30 fully mature origins; historical features use original origin
29         ↪ prefix',
30         linear_alpha=.002,target_scale=100.,feature_standardization='training only',
31         tree=dict(max_iter=60,max_leaf_nodes=7,min_samples_leaf=30,l2_regularization=1.,early_stop
32             ↪ ping=False,random_state=20260911),
33         numeric_tie_tolerance=TIE_TOL,epsilon=EPS,input_sha256=sha(ROOT/'输入/inputs.json'),
34         source_sha256={p.name:sha(p) for p in [ROOT/'第5节_Q2分位数预测与滚动调度.py',ROOT/'第5节_Q2
35             ↪ 四十五组方案对比.py',Path(__file__)]},
36         libraries={name:__import__(name).__version__ for name in
37             ↪ ['numpy','scipy','sklearn','joblib']},python=platform.python_version())
38
39 # 缓存必须匹配数据、参数、代码及模型文件指纹;主流程仍每次从头拟合。
40 def load_prediction(path,expected):
41     """Used by resumptions and audit probes; stale data/config/code is rejected."""
42     meta=json.loads(Path(path).with_suffix('.json').read_text(encoding='utf-8'))
43     if meta['configuration']!=expected:raise ValueError('Cached input/configuration/code
44         ↪ fingerprint differs')
45     if sha(path)!=meta['prediction_file_sha256']:raise ValueError('Prediction cache checksum
46         ↪ differs')
47     if sha(Path(path).with_suffix('.joblib'))!=meta['model_sha256']:raise ValueError('Model
48         ↪ checksum differs')
49     return np.load(path)['q'],meta['audit']
50
51 # 保存当日模型权重、预测结果及训练标签截止,方便重建每一个历史决策。
52 def fresh_prediction(data,day,kind,folder,cfg,features):
53     path=folder/f'{kind}_{day:03d}.npz';artifact=path.with_suffix('.joblib')
54     q,a=q2_origin(data,day,kind,features=features,artifact=artifact)
55     np.savez_compressed(path,q=q)
56     dump(path.with_suffix('.json'),dict(configuration=cfg,audit=a,prediction_file_sha256=sha(pat
57         ↪ h),model_sha256=sha(artifact)))
58     return q,a
59
60 # 输出全部 432 段规划、144 段锁定标记及实际执行账单,二者分开保存。
61 def save_play(out,trace,summary,plans,price,beta):
62     out.mkdir(parents=True,exist_ok=True);np.savez_compressed(out/'trace.npz',trace=trace);dump(
63         ↪ out/'summary.json',summary)
64     np.savez_compressed(out/'plans.npz',origins=np.array([p['origin'] for p in plans]),
65         initial=np.array([p['initial_energy'] for p in plans]),targets=np.stack([p['targets']
66             ↪ for p in plans]),
67         forecasts=np.stack([p['forecast'] for p in plans]),values=np.stack([p['values'] for p in
68             ↪ plans]))
69     reference={}
70     for p in plans:
71         for i in range(144):reference[int(p['targets'][i])]=float(p['values'][4,i])
72     execution=[]
73     for row in trace:
74         k=int(row[0]);z=reference.get(k,EMIN)
75         execution.append(dict(interval_start=clock(k),global_slot=k,net_kwh=row[1],locked_purcha
76             ↪ se_kwh=row[2],
77             charge_in_current_interval=row[4],discharge_in_current_interval=row[5],emergency_kwh
78             ↪ =row[6],unused_kwh=row[7],

```

```

64         inventory_at_start=row[8],inventory_at_end=row[9],reference_at_end=z,reserve_at_end=
        ↪ EMIN+beta*(z-EMIN),
65         ordinary_price=price[k%144],ordinary_cost=row[10],emergency_cost=row[11]))
66 csv_write(out/'q2_execution.csv',execution)
67 daily=[]
68 for d in sorted(set(trace[:,0].astype(int)//144)):
69     a=trace[(trace[:,0].astype(int)//144)==d];cost=float(a[:,10:12].sum());v=terminal_value(
        ↪ price)
70     daily.append(dict(date=clock(d*144)[10],slots=len(a),ordinary_cost=float(a[:,10].sum()),
71                     emergency_cost=float(a[:,11].sum()),actual_cost=cost,inventory_at_start=a[0,8],inven
        ↪ tory_at_end=a[-1,9],
72                     inventory_value_change=v*(a[0,8]-a[-1,9]),inventory_adjusted_cost=cost+v*(a[0,8]-a[-
        ↪ 1,9])))
73 csv_write(out/'q2_daily_cost.csv',daily)
74 def plan_rows():
75     for p in plans:
76         a=p['values'];q=p['forecast']
77         for i,k in enumerate(p['targets']):
78             yield dict(issue_time=clock(p['origin']),interval_start=clock(k),lead_block=i//1
        ↪ 44,lead_slot=i+1,
79                     q50=q[i,0],q65=q[i,1],q80=q[i,2],q90=q[i,3],q95=q[i,4],
80                     planned_purchase=a[0,i],planned_charge=a[1,i],planned_discharge=a[2,i],plann
        ↪ ed_unused=a[3,i],
81                     planned_inventory_at_start=p['initial_energy'] if i==0 else
        ↪ a[4,i-1],planned_inventory_at_end=a[4,i],
82                     locked=int(i<144),price=price[k%144],terminal_coefficient=p['terminal_value']
        ↪ ])
83 csv_write(out/'q2_daily_plan.csv',plan_rows())
84
85 def run(output=None):
86     out=Path(output) if output is not None else
        ↪ ROOT/'复跑结果'/datetime.datetime.now().strftime('%Y%m%d_%H%M%S_%f')
87     if not out.is_absolute():out=ROOT/out
88     out.mkdir(parents=True,exist_ok=False);cfg=configuration();dump(out/'q2_protocol.json',cfg)
89     for folder in ['一月模型','每日模型','候选轨迹']: (out/folder).mkdir()
90     started=time.perf_counter();data=Data();jan=Data();jan.net=jan.net[:JAN_END].copy();jan.pv=j
        ↪ an.pv[:JAN_END].copy()
91     # Features are archived at each original origin; they never contain later observations.
92     jan_features={d:net_features(jan.prefix(d*144),d*144,d*144+np.arange(1,433)) for d in
        ↪ range(7,31)}
93     qjan={kind:np.full((31,432,5),np.nan) for kind in KINDS};audits=[]
94     for day in range(19,30):
95         for kind in KINDS:
96             q,a=fresh_prediction(jan,day,kind,out/'一月模型',cfg,jan_features);qjan[kind][day]=q;
        ↪ audits.append(a)
97     print('January fresh fits',day+1,round(time.perf_counter()-started,1),flush=True)
98     dump(out/'q2_january_training_audit.json',audits)
99     # 45 组只使用一月真实量回放,各自从同一个实际库存连续运行。
100     candidates=[]
101     for kind in KINDS:
102         for qi,tau in enumerate(TAUS):
103             for beta in BETAS:
104                 trace,s,plans=q2_play(jan,qjan[kind],qi,beta,jan=True)
105                 s.update(kind=kind,main_eligible=True);candidates.append(s)
106                 np.savez_compressed(out/f'候选轨迹/{kind}_{tau:g}_{beta:g}.npz',trace=trace)
107     # 不读取正式期费用选型;同分依次比较紧急购电量、分位数、保护系数、模型次序。
108     best=select_main(candidates)
109     if (best['kind'],best['tau'],best['beta'])!=('tree',.8,0.):raise ValueError('January
        ↪ selection differs from adopted frozen configuration; do not silently replace it')
110     selected=dict(kind=best['kind'],tau=best['tau'],tau_index=2,beta=best['beta'],selection_scor
        ↪ e=best['selection_score'],
111                 final_selection_before=clock(JAN_END),configuration=cfg,adoption='User adopted the
        ↪ January 45-candidate winner: tree, tau=0.8, beta=0; tie rule unchanged')
112     dump(out/'q2_selected_config.json',selected)
113     for rank,c in enumerate(sorted(candidates,key=lambda
        ↪ s:s['selection_score']),1):c['diagnostic_rank']=rank
114     csv_write(out/'q2_candidate_summary.csv',[{k:c[k] for k in
        ↪ ['kind','tau','beta','main_eligible','diagnostic_rank','cost_yuan','emergency_kwh','unus
        ↪ ed_kwh','start_energy','end_energy','selection_score']} for c in
        ↪ candidates])
115     metrics=[]
116     for kind in KINDS:

```

```

117         for qi,tau in enumerate(TAUS):
118             for lead in range(3):
119                 errors=np.concatenate([jan.net[d*144+np.arange(1+lead*144,145+lead*144)]-qjan[ki]
120                                     ↪ nd][d,lead*144:(lead+1)*144,qi] for d in
121                                     ↪ range(19,28)])
122                 metrics.append(dict(kind=kind,tau=tau,lead_block=lead,records=len(errors),pinbal
123                                     ↪ l=float(np.mean(np.maximum(tau*errors,(tau-1)*errors))))))
124             csv_write(out/'q2_forecast_metrics.csv',metrics)
125             fresh_prediction(jan,30,best['kind'],out/'一月模型',cfg,jan_features)
126             print('January frozen',selected['kind'],selected['tau'],selected['beta'],selected['selection']
127                 ↪ _score'],flush=True)
128             del jan_features
129             # 特征可预计算,但每个历史时点都独立截取自己的历史前缀。
130             features={d:net_features(data.prefix(d*144),d*144,d*144+np.arange(1,433)) for d in
131                 ↪ range(7,365)}
132             qfull=np.full((365,432,5),np.nan);formal_audit=[]
133             # 正式期每天重新拟合;初始配置冻结后不再重选,允许使用当时已发生的后续月份历史。
134             for day in range(31,365):
135                 q,a=fresh_prediction(data,day,best['kind'],out/'每日模型',cfg,features);qfull[day]=q;form
136                 ↪ al_audit.append(a)
137                 if day%15==0:print('Formal daily fresh
138                 ↪ fits',clock(day*144)[:10],round(time.perf_counter()-started,1),flush=True)
139             dump(out/'q2_formal_training_audit.json',formal_audit);np.savez_compressed(out/'q2_daily_for
140             ↪ ecasts.npz',q=qfull)
141             trace,s,plans=q2_play(data,qfull,selected['tau_index'],best['beta']);save_play(out,trace,s,p
142             ↪ lans,data.price,best['beta'])
143             e,g,z,warm=warmup(data);np.savez_compressed(out/'january_startup_trace.npz',trace=warm)
144             jan20=warm[19*144]
145             dump(out/'startup_and_bridge.json',dict(January20_0000_inventory=jan20[8],January20_0010_inv
146             ↪ entory=jan20[9],
147                 February1_0000_inventory=e,February1_current_purchase=g,February1_reference=z,
148                 next_year_locked_purchase=float(plans[-1]['values'][0,143]),next_year_reference=float(pl
149                 ↪ ans[-1]['values'][4,143]),
150                 next_year_slot_excluded_from_formal_cost=FORMAL_END))
151             dump(out/'DONE.json',dict(complete=True,seconds=time.perf_counter()-started,summary=s,config
152             ↪ uration=cfg))
153             # 比较代码正式接入,不再需要另开脚本手动执行。
154             compare_schemes(out,data,qfull,candidates,selected)
155             print('Q2 COMPUTATION
156             ↪ COMPLETE',s['cost_yuan'],s['slots'],time.perf_counter()-started,flush=True)
157             return out
158
159 def main():
160     parser=argparse.ArgumentParser(description='从头运行并验收 Q2 冻结方案')
161     parser.add_argument('--output',help='新的输出目录;默认自动创建复跑目录')
162     args=parser.parse_args();run(args.output)
163
164 if __name__=='__main__':main()

```

## 4.7 Q2 敏感性测试.py

源码/Q2 前期敏感性/Q2 敏感性测试.py

```

1  """Q2 独立敏感性分析。
2
3  只读取正式 Q2 主模型和 inputs.json,不读取任何已有敏感性脚本或结果。
4  程序按因果顺序从 1 月连续运行到 6 月末,只统计 2025-06-01 至 2025-06-30。
5  """
6
7  from __future__ import annotations
8
9  import hashlib
10 import json
11 import os
12 import platform
13 import sys
14 import time
15 from pathlib import Path
16

```

```

17 os.environ.setdefault("OMP_NUM_THREADS", "4")
18 os.environ.setdefault("OPENBLAS_NUM_THREADS", "4")
19
20 import numpy as np
21 from sklearn.linear_model import QuantileRegressor
22 from sklearn.preprocessing import StandardScaler
23
24
25 ROOT = Path(__file__).resolve().parent
26 CORE = ROOT / " 核心代码"
27 INPUT = ROOT / " 输入" / "inputs.json"
28 sys.path.insert(0, str(CORE))
29
30 import local_bootstrap # noqa: E402,F401
31 import battery_controller as battery # noqa: E402
32 import q2_forecast_controller as q2 # noqa: E402
33 import scipy # noqa: E402
34 import sklearn # noqa: E402
35
36
37 START_DAY = 31
38 WINDOW_START = 151 # 2025-06-01, zero based
39 WINDOW_END = 181 # 2025-07-01, exclusive
40 BASELINE = {"kind": "tree", "window": 30, "tau": 0.65, "beta": 0.5}
41
42
43 def sha256(path: Path) -> str:
44     return hashlib.sha256(path.read_bytes()).hexdigest()
45
46
47 def load_arrays():
48     raw = json.loads(INPUT.read_text(encoding="utf-8"))
49     if raw.get("schema") != "Q1-Q3-STARTPOINT-DELAY10-v3":
50         raise ValueError("inputs.json schema mismatch")
51     load = np.asarray(raw["load_kw"], dtype=float)
52     pv = np.asarray(raw["pv_kw"], dtype=float)
53     if load.shape != (365, 144) or pv.shape != (365, 144):
54         raise ValueError("annual load/PV matrix must be 365 x 144")
55     if not np.isfinite(load).all() or not np.isfinite(pv).all() or min(load.min(), pv.min()) < 0:
56         raise ValueError("invalid load/PV values")
57     rows = raw["q1_records"][-1:] + raw["q1_records"][:-1]
58     price = np.asarray([r["price"] for r in rows], dtype=float)
59     reference = np.asarray([(r["load_kw"] - r["pv_kw"]) / 6 for r in rows], dtype=float)
60     net_raw = (load - pv).ravel() / 6
61     net = np.r_[reference[0], net_raw[:-1]].reshape(365, 144)
62     return raw, load, pv, net, price, reference
63
64
65 def representative_month(load: np.ndarray, pv: np.ndarray):
66     """Use four scale-normalized descriptors; lower distance is more representative."""
67     net = (load - pv) / 6
68     formal = np.arange(31, 365)
69     months = np.asarray([int(x[5:7]) for x in
70 ↪ json.loads(INPUT.read_text(encoding="utf-8"))["dates"]])
71
72     def describe(ix):
73         return np.asarray([
74             net[ix].mean(),
75             net[ix].std(),
76             np.quantile(net[ix], 0.95),
77             pv[ix].sum() / load[ix].sum(),
78         ])
79
80     overall = describe(formal)
81     vectors = np.vstack([describe(np.flatnonzero(months == month)) for month in range(2, 13)])
82     scale = vectors.std(axis=0, ddof=1)
83     distances = np.linalg.norm((vectors - overall) / scale, axis=1)
84     ranking = [
85         {"month": month, "distance": float(distance)}
86         for month, distance in zip(range(2, 13), distances)
87     ]
88     ranking.sort(key=lambda x: x["distance"])
89     if ranking[0]["month"] != 6:

```

```

89         raise AssertionError(f"representative month changed: {ranking[0]}")
90     return ranking, overall.tolist()
91
92
93 def build_forecasts(
94     net: np.ndarray, kind: str, window: int, *,
95     output_start: int, first_fit: int, end_day: int = WINDOW_END,
96     only_tau: float | None = None,
97 ):
98     outputs = {}
99     model = None
100     audits = []
101     for day in range(first_fit, end_day):
102         if (day - first_fit) % 7 == 0:
103             if only_tau is None:
104                 model = q2.Forecast(kind, window).fit(net[:day])
105                 audit = dict(model.audit)
106             else:
107                 origins = range(max(7, day - window), day - 2)
108                 x = np.vstack([q2.features(net[:origin], origin) for origin in origins])
109                 y = np.concatenate([net[origin:origin + 3].reshape(-1) / 100 for origin in
110                                     ↪ origins])
111                 scale = StandardScaler().fit(x)
112                 model = QuantileRegressor(quantile=only_tau, alpha=.002, solver="highs")
113                 model.fit(scale.transform(x), y)
114                 audit = {
115                     "cutoff_day": day, "latest_label_day": day - 1,
116                     "first_origin": origins.start, "last_origin": origins.stop - 1,
117                     "samples": len(y), "kind": kind, "window": window,
118                     "only_tau": only_tau,
119                 }
120             audits.append(audit)
121             if only_tau is None:
122                 pred = model.predict(net[:day])
123             else:
124                 q65 = model.predict(scale.transform(q2.features(net[:day], day))) * 100
125                 pred = np.repeat(q65.reshape(3, 144, 1), 5, axis=2)
126                 if pred.shape != (3, 144, 5) or not np.isfinite(pred).all():
127                     raise ValueError("invalid Q2 forecast")
128                 if day >= output_start:
129                     outputs[day] = pred
130             if max(a["latest_label_day"] for a in audits) >= WINDOW_END:
131                 raise AssertionError("future labels entered training")
132     return outputs, audits
133
134 def advance_to_window(net, price, reference, forecasts):
135     """Run the common baseline policy through 2025-05-31 once."""
136     warm = q2.startup(net[:31], price, reference)
137     energy = float(warm[-1, 5])
138     carry_g = float(max(reference[0], 0))
139     carry_z = q2.EMIN
140     for day in range(START_DAY, WINDOW_START):
141         q = forecasts[day].reshape(-1, len(q2.TAUS))[:, 1]
142         horizon = min(3 * 144, 365 * 144 + 1 - day * 144)
143         p = np.resize(price, horizon)
144         plan = q2.plan_delayed(
145             q[:horizon], p, energy, carry_g, carry_z, 0.5,
146             day * 144 + horizon < 365 * 144 + 1,
147             known_net=float(net[day, 0]),
148         )
149         committed = plan[0, :144].copy()
150         reserve = plan[4, :144].copy()
151         committed[0], reserve[0] = carry_g, carry_z
152         run, energy = q2.execute_day(
153             net[day], committed, reserve, 0.5, price, start_slot=day * 144
154         )
155         if abs(plan[4, 0] - run[0, 5]) > 1e-7:
156             raise AssertionError("locked bridge inventory mismatch")
157         carry_g, carry_z = float(plan[0, 144]), float(plan[4, 144])
158     return float(energy), carry_g, carry_z
159
160

```

```

161 def simulate(net, price, forecasts, initial, tau: float, beta: float):
162     tau_index = int(np.flatnonzero(np.isclose(q2.TAUS, tau))[0])
163     energy, carry_g, carry_z = map(float, initial)
164     selected = []
165
166     for day in range(WINDOW_START, WINDOW_END):
167         q = forecasts[day].reshape(-1, len(q2.TAUS))[:, tau_index]
168         horizon = min(3 * 144, 365 * 144 + 1 - day * 144)
169         p = np.resize(price, horizon)
170         plan = q2.plan_delayed(
171             q[:horizon], p, energy, carry_g, carry_z, beta,
172             day * 144 + horizon < 365 * 144 + 1,
173             known_net=float(net[day, 0]),
174         )
175         committed = plan[0, :144].copy()
176         reserve = plan[4, :144].copy()
177         committed[0] = carry_g
178         reserve[0] = carry_z
179         run, energy = q2.execute_day(
180             net[day], committed, reserve, energy, beta, price, start_slot=day * 144
181         )
182         if abs(plan[4, 0] - run[0, 5]) > 1e-7:
183             raise AssertionError("locked bridge inventory mismatch")
184         selected.append(run)
185         carry_g = float(plan[0, 144])
186         carry_z = float(plan[4, 144])
187
188     trace = np.vstack(selected)
189     checks = q2.physical(trace, net[WINDOW_START:WINDOW_END], price)
190     metrics = q2.metrics(trace)
191     metrics.update(
192         tau=float(tau), beta=float(beta),
193         max_balance_residual=checks["errors"]["balance"],
194         max_state_residual=checks["errors"]["state"],
195         simultaneous_slots=checks["simultaneous"],
196     )
197     return metrics
198
199
200 def main():
201     began = time.perf_counter()
202     raw, load, pv, net, price, reference = load_arrays()
203     ranking, overall = representative_month(load, pv)
204
205     specs = [("tree", 30), ("tree", 14), ("linear", 30), ("linear", 14)]
206     forecast_sets = {}
207     fit_audits = {}
208     print("fit Q2 forecasts: tree_30, common history and June", flush=True)
209     forecast_sets[("tree", 30)], fit_audits["tree_30"] = build_forecasts(
210         net, "tree", 30, output_start=START_DAY, first_fit=START_DAY
211     )
212     initial = advance_to_window(net, price, reference, forecast_sets[("tree", 30)])
213     for kind, window in specs[1:]:
214         print(f"fit Q2 June forecasts: {kind}_{window}", flush=True)
215         forecast_sets[(kind, window)], fit_audits[f"{kind}_{window}"] = build_forecasts(
216             net, kind, window, output_start=WINDOW_START, first_fit=WINDOW_START - 1,
217             only_tau=0.65 if kind == "linear" else None,
218         )
219
220     scenarios = []
221
222     def add(group, label, kind="tree", window=30, tau=0.65, beta=0.5):
223         result = simulate(net, price, forecast_sets[(kind, window)], initial, tau, beta)
224         result.update(group=group, label=label, kind=kind, training_window_days=window)
225         scenarios.append(result)
226         print(group, label, round(result["cost_yuan"], 3), flush=True)
227
228     for kind, window in specs:
229         add("forecast_model", f"{kind}_{window}", kind=kind, window=window)
230     for tau in q2.TAUS:
231         if not np.isclose(tau, BASELINE["tau"]):
232             add("tau", f"{tau:.2f}", tau=float(tau))
233     for beta in (0.0, 1.0):

```

```

234         add("beta", f"{beta:.1f}", beta=beta)
235
236     baseline = next(x for x in scenarios if x["group"] == "forecast_model" and x["label"] ==
    ↪ "tree_30")
237     for row in scenarios:
238         row["delta_cost_yuan"] = row["cost_yuan"] - baseline["cost_yuan"]
239         row["delta_cost_percent"] = 100 * row["delta_cost_yuan"] / baseline["cost_yuan"]
240
241     # A second identical baseline run catches accidental state or RNG leakage.
242     repeated = simulate(net, price, forecast_sets[("tree", 30)], initial, 0.65, 0.5)
243     repeat_error = abs(repeated["cost_yuan"] - baseline["cost_yuan"])
244     if repeat_error > 1e-8:
245         raise AssertionError(f"baseline repeat mismatch: {repeat_error}")
246
247     result = {
248         "analysis": "Q2 independent sensitivity analysis",
249         "input_sha256": sha256(INPUT),
250         "source_model_sha256": sha256(CORE / "02_q2_forecast_controller.py"),
251         "window": ["2025-06-01 00:00", "2025-07-01 00:00"],
252         "selection_method": "minimum standardized distance on mean/std/p95 net load and
    ↪ PV-to-load ratio",
253         "representative_month_ranking": ranking,
254         "formal_period_descriptor": overall,
255         "baseline": BASELINE,
256         "common_state_rule": "all scenarios share the baseline state and inherited contract at
    ↪ 2025-06-01 00:00",
257         "scenarios": scenarios,
258         "baseline_repeat_absolute_error_yuan": repeat_error,
259         "fit_audits": fit_audits,
260         "runtime": {
261             "python": platform.python_version(),
262             "numpy": np.__version__,
263             "scipy": scipy.__version__,
264             "sklearn": sklearn.__version__,
265             "seconds": time.perf_counter() - began,
266         },
267     }
268     print("RESULT_JSON=" + json.dumps(result, ensure_ascii=False, allow_nan=False), flush=True)
269
270
271 if __name__ == "__main__":
272     main()

```

## 4.8 02\_q2\_forecast\_controller.py

源码/Q2 前期敏感性/核心代码/02\_q2\_forecast\_controller.py

```

1 import local_bootstrap
2 """New Q2 model; no imports or caches from previous Q2 policies."""
3 import os
4 os.environ.setdefault('OMP_NUM_THREADS','4')
5 os.environ.setdefault('OPENBLAS_NUM_THREADS','4')
6 import datetime as dt
7 import numpy as np
8 from scipy.optimize import linprog
9 from scipy.sparse import lil_matrix
10 from sklearn.ensemble import HistGradientBoostingRegressor
11 from sklearn.linear_model import QuantileRegressor
12 from sklearn.preprocessing import StandardScaler
13 from threadpoolctl import threadpool_limits
14 from q2_reproducibility import clock_features, validate_runtime
15
16 H=144; CAP=5000/6; EMIN=1200.; EMAX=10800.; ETA=.9
17 TAUS=np.array([.50,.65,.80,.90,.95])
18
19 def features(past,day):
20     """past is physically truncated at day; all leads use same observed history."""
21     assert len(past)==day and day>=7
22     clock=clock_features()
23     last=past[-1];mean3=past[-3:].mean(0);mean7=past[-7:].mean(0)

```

```

24     sd7=past[-7:].std(0);week=past[-7]
25     features=[]
26     for lead in range(3):
27         dow=(dt.date(2025,1,1)+dt.timedelta(days=day+lead)).weekday()
28         x=np.column_stack([clock,
29                             np.full(H,lead),*[np.full(H,float(dow==j)) for j in range(6)],
30                             last/100,mean3/100,mean7/100,sd7/100,week/100,
31                             np.full(H,past[-1].mean()/100),np.full(H,past[-3:].mean()/100)])
32         features.append(x)
33     return np.vstack(features)
34
35 class Forecast:
36     def __init__(self,kind>window):self.kind=kind;self.window=window
37     def fit(self,past):
38         validate_runtime()
39         cutoff=len(past)
40         origins=range(max(7,cutoff-self.window),cutoff-2)
41         xs=[];ys=[]
42         for d in origins:
43             xs.append(features(past[:d],d));ys.append(past[d:d+3].reshape(-1)/100)
44         X=np.vstack(xs);y=np.concatenate(ys)
45         self.scale=StandardScaler().fit(X);X=self.scale.transform(X)
46         self.models=[]
47         with threadpool_limits(limits=4):
48             for tau in TAUS:
49                 if self.kind=='linear':m=QuantileRegressor(quantile=float(tau),alpha=.002,solver_
↳ 'highs')
50                 else:m=HistGradientBoostingRegressor(loss='quantile',quantile=float(tau),max_ite_
↳ r=60,max_leaf_nodes=7,min_samples_leaf=30,l2_regularization=1.,early_stoppin_
↳ g=False,random_state=20260911)
51                 m.fit(X,y);self.models.append(m)
52         self.audit={'cutoff_day':cutoff,'latest_label_day':cutoff-1,'first_origin':origins.start_
↳ ', 'last_origin':origins.stop-1,'samples':len(y),'kind':self.kind,'window':self.window}
53         assert self.audit['last_origin']+2<cutoff
54         return self
55     def predict(self,past):
56         assert self.audit['cutoff_day']<=len(past)
57         X=self.scale.transform(features(past,len(past)))
58         with threadpool_limits(limits=4):q=np.column_stack([m.predict(X)*100 for m in
↳ self.models])
59         # Rearrangement removes crossing without treating quantile columns as paths.
60         return np.sort(q,axis=1).reshape(3,H,len(TAUS))
61
62 def plan(q,price,E,terminal=True):
63     q=np.asarray(q).reshape(-1);n=len(q);p=np.resize(price,n)
64     # g,c,b,u,E_end blocks; exact bus balance and storage recursion.
65     A=lil_matrix((2*n,5*n));rhs=np.r_[q,np.zeros(n)];rhs[n]=E
66     for t in range(n):
67         A[t,t]=1;A[t,n+t]=-1;A[t,2*n+t]=1;A[t,3*n+t]=-1
68         A[n+t,4*n+t]=1;A[n+t,n+t]=-ETA;A[n+t,2*n+t]=1/ETA
69         if t:A[n+t,4*n+t-1]=-1
70     U=lil_matrix((n,5*n))
71     for t in range(n):U[t,n+t]=1;U[t,2*n+t]=1
72     obj=np.r_[p,np.full(2*n,1e-6),np.zeros(n),np.zeros(n)]
73     if terminal:obj[-1]=-float(price.min())/ETA
74     bounds=[(0,None)]*n+[(0,CAP)]*(2*n)+[(0,None)]*n+[(EMIN,EMAX)]*n
75     r=linprog(obj,A_ub=U.tocsr(),b_ub=np.full(n,CAP),A_eq=A.tocsr(),b_eq=rhs,bounds=bounds,method='highs')
76     if not r.success:raise RuntimeError(r.message)
77     a=r.x.reshape(5,n)
78     assert np.max(abs(A@r.x-rhs))<1e-6
79     assert not np.any((a[1]>1e-6)&(a[2]>1e-6))
80     return a
81
82 FIELDS=['plan_kwh','emergency_kwh','charge_kwh','discharge_kwh','e_start_kwh','e_end_kwh','unuse_
↳ d_kwh','plan_cost_yuan','emergency_cost_yuan','reserve_kwh']
83
84 def plan_delayed(q, price, E, inherited_g, inherited_z, beta=.5, terminal=True, *, known_net):
85     """Use only the known current net to project the locked ten-minute bridge."""
86     from battery_controller import project_current
87     q=np.asarray(q).reshape(-1);p=np.asarray(price)
88     action,projected=project_current(known_net,inherited_g,inherited_z,E,beta)
89     rest=plan(q[1:],p[1:],projected,terminal)

```

```

90     first=np.array([inherited_g,action.charge,action.discharge,action.unused,projected])
91     return np.column_stack([first,rest])
92
93 def execute_day(actual,g,planned_end,E,beta,price,*,start_slot=0):
94     from battery_controller import BatteryState,CurrentObservation,choose_action,advance
95     if not (len(actual)==len(g)==len(planned_end)==len(price)):
96         raise ValueError('Controller arrays must use identical target intervals')
97     state=BatteryState(start_slot,float(E));out=[]
98     for t in range(len(actual)):
99         observation=CurrentObservation(state.slot,float(actual[t]))
100         action=choose_action(state,observation,float(g[t]),float(planned_end[t]),beta)
101         after=advance(state,action)
102         out.append([g[t],action.emergency,action.charge,action.discharge,
103                   state.energy,after.energy,action.unused,price[t]*g[t],
104                   5*price[t]*action.emergency,action.reserve])
105     state=after
106     return np.array(out),state.energy
107
108 def startup(net,price,reference):
109     E=6000.;out=[];g=np.maximum(reference,0)
110     for d in range(31):
111         a,E=execute_day(net[d],g,np.full(H,EMIN),E,0.,price);out.append(a)
112     return np.vstack(out)
113
114 def physical(a,actual,price):
115     n=np.asarray(actual).reshape(-1);p=np.resize(price,len(n))
116     g,h,c,b,e0,e1,u,pc,ec,_=a.T
117     errors={'balance':float(np.max(abs(g+h+b-n-c-u))), 'state':float(np.max(abs(e1-e0-ETA*c+b/ETA
↪ ))),
118            'continuity':float(np.max(abs(e1[:-1]-e0[1:]))), 'plan_cost':float(np.max(abs(pc-p*g))),
119            'emergency_cost':float(np.max(abs(ec-5*p*h)))}
120     checks={'errors':errors, 'min_energy':float(min(e0.min(),e1.min())),'max_energy':float(max(e0
↪ .max(),e1.max()))},
121            'max_power_kw':float(max(c.max(),b.max())*6), 'simultaneous':int(((c>1e-7)&(b>1e-7)).sum(
↪ )),
122            'emergency_with_charge':int(((h>1e-7)&(c>1e-7)).sum()), 'min_flow':float(a[:,4].min())}
123     assert max(errors.values())<1e-6 and checks['min_energy']>=EMIN-1e-6 and
↪ checks['max_energy']<=EMAX+1e-6
124     assert checks['max_power_kw']<=5000+1e-6 and checks['min_flow']>=-1e-6
125     assert checks['simultaneous']==checks['emergency_with_charge']==0
126     return checks
127
128 def metrics(a):
129     return {'cost_yuan':float(a[:,7:9].sum()), 'plan_cost_yuan':float(a[:,7].sum()), 'emergency_co
↪ st_yuan':float(a[:,8].sum()),
130            'emergency_kwh':float(a[:,1].sum()), 'emergency_slots':int((a[:,1]>1e-7).sum()), 'emergenc
↪ y_fraction':float((a[:,1]>1e-7).mean()),
131            'unused_kwh':float(a[:,6].sum()), 'start_energy':float(a[0,4]), 'end_energy':float(a[-1,5]
↪ )}

```

## 4.9 battery\_controller.py

源码/Q2 前期敏感性/核心代码/battery\_controller.py

```

1  """CURRENT10-v4: observe current exogenous input, choose action, then advance.
2
3  The user assumes load/PV for [t,t+10min) are known at t. This is an
4  information assumption separate from Excel's left-endpoint labels.
5  No actual future inventory or later interval observation is an input.
6  """
7  from dataclasses import dataclass
8  import numpy as np
9
10  PROTOCOL = 'CURRENT10-v4'
11  EMIN, EMAX, ETA, CAP = 1200., 10800., .9, 5000/6
12
13  @dataclass(frozen=True)
14  class BatteryState:
15      slot: int
16      energy: float

```

```

17
18 @dataclass(frozen=True)
19 class CurrentObservation:
20     slot: int
21     net_kwh: float
22
23 @dataclass(frozen=True)
24 class Action:
25     slot: int
26     start_energy: float
27     charge: float
28     discharge: float
29     emergency: float
30     unused: float
31     reserve: float
32
33 def dispatch_flows(net, g, z, energy, beta=.5):
34     """Same decision equation for actual operation and scenario-time simulation."""
35     surplus = np.asarray(g)-np.asarray(net)
36     reserve = EMIN+beta*(np.asarray(z)-EMIN)
37     charge = np.minimum(np.maximum(surplus,0),np.minimum(CAP,np.maximum((EMAX-energy)/ETA,0)))
38     discharge =
39     ↪ np.minimum(np.maximum(-surplus,0),np.minimum(CAP,np.maximum(ETA*(energy-reserve),0)))
40     emergency = np.maximum(-surplus,0)-discharge
41     unused = np.maximum(surplus,0)-charge
42     return charge,discharge,emergency,unused,reserve
43
44 def choose_action(state, observation, purchase, reserve_target, beta=.5):
45     if observation.slot != state.slot:
46         raise ValueError('Only the current interval observation may enter the controller')
47     vals=[state.energy,observation.net_kwh,purchase,reserve_target,beta]
48     if not np.isfinite(vals).all() or not 0<=beta<=1:
49         raise ValueError('Invalid controller inputs')
50     if not EMIN-1e-6<=state.energy<=EMAX+1e-6 or not EMIN-1e-6<=reserve_target<=EMAX+1e-6 or
51     ↪ purchase < -1e-6:
52         raise ValueError('Inventory, purchase or reserve outside bounds')
53     c,b,h,u,r=dispatch_flows(observation.net_kwh,purchase,reserve_target,state.energy,beta)
54     return Action(state.slot,state.energy,*map(float,(c,b,h,u,r)))
55
56 def advance(state, action):
57     """Apply already chosen actions; next inventory is calculated, never supplied."""
58     if state.slot!=action.slot or abs(state.energy-action.start_energy)>1e-8:
59         raise ValueError('Stale action or attempted inventory overwrite')
60     if min(action.charge,action.discharge,action.emergency,action.unused)<-1e-6 or
61     ↪ action.charge+action.discharge>CAP+1e-6:
62         raise ValueError('Invalid action quantity')
63     if action.charge>1e-7 and (action.discharge>1e-7 or action.emergency>1e-7):
64         raise ValueError('Incompatible charge/discharge or emergency charging')
65     energy=state.energy+ETA*action.charge-action.discharge/ETA
66     if not EMIN-1e-6<=energy<=EMAX+1e-6:
67         raise ValueError('Executed inventory violates physical bounds')
68     return BatteryState(state.slot+1,float(energy))
69
70 def project_current(net, g, z, energy, beta=.5):
71     """Pure model projection of the locked segment; does not advance live state."""
72     state=BatteryState(0,float(energy))
73     action=choose_action(state,CurrentObservation(0,float(net)),float(g),float(z),beta)
74     return action,advance(state,action).energy
75
76 def audit_times(slot):
77     return dict(observation_available_slot=slot,action_decision_slot=slot,
78                 state_before_slot=slot,state_after_slot=slot+1,
79                 controller_protocol=PROTOCOL)

```

## 4.10 local\_bootstrap.py

源码/Q2 前期敏感性/核心代码/local\_bootstrap.py

```

1  """Resolve the numbered Q2 module in this self-contained experiment."""
2  import sys
3  import importlib.abc
4  import importlib.util
5  from pathlib import Path
6  HERE = Path(__file__).resolve().parent
7  class _NumberedFiles(importlib.abc.MetaPathFinder):
8      def find_spec(self, fullname, path=None, target=None):
9          if fullname == "q2_forecast_controller":
10             return importlib.util.spec_from_file_location(fullname, HERE /
11                 ↪ "02_q2_forecast_controller.py")
12 sys.meta_path.insert(0, _NumberedFiles())

```

## 4.11 q2\_reproducibility.py

源码/Q2 前期敏感性/核心代码/q2\_reproducibility.py

```

1  """Preserve the original deterministic calendar and record the current runtime."""
2  from functools import lru_cache
3  from pathlib import Path
4  import numpy as np
5
6  @lru_cache(maxsize=1)
7  def clock_features():
8      a = np.load(Path(__file__).resolve().parent/"q2_clock_features.npy", allow_pickle=False)
9      if a.shape != (144,4) or not np.isfinite(a).all():
10         raise ValueError("Invalid calendar features")
11     a.flags.writeable = False
12     return a
13
14  @lru_cache(maxsize=1)
15  def validate_runtime():
16      import scipy, sklearn
17      clock_features()
18      return dict(numpy=np.__version__, scipy=scipy.__version__, sklearn=sklearn.__version__)

```

## 4.12 Q3\_Q4 敏感性测试.py

源码/Q3\_Q4 敏感性/Q3\_Q4 敏感性测试.py

```

1  """ 六月单因素敏感性：读取已冻结模型的因果预测，独立续跑，不修改正式工程。
2
3  运行：python Q3_Q4 敏感性测试.py
4  先按运行说明生成完整正式期预测与执行档案，再用相对路径指定运行目录。
5  所有情景在读取六月费用前写入实验协议。无需重新选参或重新拟合相同预测器。
6  """
7  from __future__ import annotations
8  import os
9  os.environ['OMP_NUM_THREADS']='1'
10 os.environ['OPENBLAS_NUM_THREADS']='1'
11 import sys, json, hashlib, importlib, time, platform, datetime
12 from pathlib import Path
13 import numpy as np
14
15 OUT=Path(__file__).resolve().parent
16 import argparse
17 parser=argparse.ArgumentParser(description=__doc__)
18 parser.add_argument('--q4-runtime',type=Path,default=Path('../复现运行/q4_3'))
19 parser.add_argument('--q3-runtime',type=Path,default=Path('../复现运行/q3'))
20 parser.add_argument('--q4-2-runtime',type=Path,default=None)

```

```

21 parser.add_argument('--formal-folder',default='本次复算')
22 args=parser.parse_args()
23 def relative_input(p):
24     if p.is_absolute():raise ValueError('请使用相对于本脚本目录的路径')
25     return (OUT/p).resolve()
26 CORE=relative_input(args.q4_runtime)
27 Q3ROOT=relative_input(args.q3_runtime)
28 Q42ROOT=CORE if args.q4_2_runtime is None else relative_input(args.q4_2_runtime)
29 def relative_name(p):
30     return Path(os.path.relpath(p,OUT)).as_posix()
31 sys.path.insert(0,str(CORE/'代码'))
32 m=importlib.import_module('第 4 至 7 节 _ 合同优化与储能执行')
33 fm=importlib.import_module('第 6 节 _Q3 逐节点负载与光伏预测')
34 pm=importlib.import_module('第 7 节 _Q4 每日负载与电价预测')
35 START=151*144; STOP=181*144
36 BASE={
37     'q3':dict(tau=.65,beta=.5,alpha=.75,nodes=[6,12,18]),
38     'q4_2':dict(tau=.8,beta=0.,alpha=0.,nodes=[]),
39     'q4_3':dict(tau=.65,beta=0.,alpha=.75,nodes=[6,12,18]),
40 }
41
42 def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
43 def read(p):return json.loads(Path(p).read_text(encoding='utf-8'))
44 def dump(p,x):m.dump(p,x)
45 def formal(branch):
46     return (Q3ROOT if branch=='q3' else Q42ROOT if branch=='q4_2' else
47         ↪ CORE)/args.formal_folder/branch/'正式期'
48
49 def scenarios(branch):
50     base=dict(BASE[branch],residual_scale=1.,spread_scale=1.)
51     rows=[dict(base,id='baseline',group='baseline')]
52     groups={'tau': [.5,.65,.8,.9,.95], 'beta': [0.,.5,1.]}
53     if branch!='q4_2':groups.update(alpha=[0.,.25,.5,.75,1.],nodes=[list(s) for s in
54         ↪ m.NODE_SETS])
55     if branch!='q3':groups.update(residual_scale=[0.,.5,1,1.5],spread_scale=[.8,1.,1.2])
56     for key,values in groups.items():
57         for value in values:
58             if value==base[key]:continue
59             label='-'.join(map(str,value)) or 'none' if isinstance(value,list) else str(value)
60             rows.append(dict(base,**{key:value},id=key+'_'+label,group=key))
61     return rows
62
63 def register():
64     protocol=dict(created_at=datetime.datetime.now().astimezone().isoformat(),
65         purpose='Descriptive sensitivity of frozen paper branches; no post-January parameter
66         ↪ reselection',
67         window=['2025-06-01 00:00','2025-07-01 00:00'],slots=4320,
68         month_rule='Use the reference Q2 June window. Independently recompute its quantity-based
69         ↪ representativeness; no cost-based choice. Not a claim of price representativeness.',
70         common_state='Use each branch original uninterrupted formal trace at START, including
71         ↪ original/effective contract and reference inventory. Across scenarios within branch
72         ↪ state is identical; across branches it can differ.',
73         forecast_rules='Frozen tree family and original fit cadence. Reuse archived five
74         ↪ quantiles/PV/price predictions after fingerprint and historical-label audit; no future
75         ↪ truth in forecast construction.',
76         price_residual='p_new=max(1e-6, seven-day complete-day same-clock mean +
77         ↪ residual_scale*(original price forecast - that mean))',
78         price_spread='Within each 144-target forecast block, p_new=max(1e-6, block mean +
79         ↪ spread_scale*(original forecast - block mean)). Only forecast prices change;
80         ↪ settlement prices unchanged.',
81         primary_metric='Actual June contract plus emergency bill; no terminal inventory deduction',
82         secondary_metric='Delta(C - v*E_end), v=.9*min of branch original forecast at June30
83         ↪ 00:00; common across scenarios, diagnostic only',
84         factors={b:scenarios(b) for b in BASE},
85         source_sha256={relative_name(p):sha(p) for p in (CORE/'代码').glob('*.py')},
86         input_sha256={relative_name(p):sha(p) for p in (CORE/'输入').iterdir() if p.is_file()},
87         runner_sha256=sha(__file__))
88     dump(OUT/'实验协议 _ 运行前冻结.json',protocol)
89     return protocol
90
91 def representative(data):
92     load=np.asarray(data.raw['load_kw']);pv=np.asarray(data.raw['pv_kw']);net=(load-pv)/6
93     months=np.asarray([int(x[5:7]) for x in data.raw['dates']])

```

```

82     def desc(mask): return np.array([net[mask].mean(), net[mask].std(), np.quantile(net[mask], .95),
    ↪ pv[mask].sum()/load[mask].sum()])
83     overall=desc(months>=2); a=np.vstack([desc(months==j) for j in range(2,13)])
84     dist=np.linalg.norm((a-overall)/a.std(axis=0, ddof=1), axis=1)
85     return sorted([dict(month=j, distance=float(v)) for j,v in zip(range(2,13), dist)], key=lambda
    ↪ x: x['distance'])
86
87 def check_record(path, asset, expected_source=None):
88     meta=read(path.with_suffix('.json')); audit=meta['audit']; o=audit['origin']
89     assert sha(path)==meta['prediction_sha256'], path
90     assert sha(path.with_suffix('.joblib'))==meta['model_sha256'], path
91     assert audit['max_training_target']<o if asset=='price' else audit['max_training_target']<=o
92     assert audit.get('feature_cutoff', audit.get('training_cutoff'))==(o-1 if asset=='price' else
    ↪ o)
93     if expected_source:
94         for name, h in meta['configuration']['source_sha256'].items():
95             assert sha(expected_source/'代码'/name)==h, (name, 'code changed')
96     with np.load(path) as z: a={k: z[k].copy() for k in z.files}
97     return a, audit
98
99 def prepare(branch, data):
100     folder=formal(branch); done=read(folder/'正式计算完成.json'); winner=done['selected']
101     for key, value in BASE[branch].items(): assert winner[key]==value, (branch, key)
102     assert winner['kind']=='tree'
103     root=Q3ROOT if branch=='q3' else Q42ROOT if branch=='q4_2' else CORE
104     for name, h in done['configuration']['source_sha256'].items(): assert
    ↪ sha(root/'代码'/name)==h, (name, 'formal source changed')
105     tracepath=folder/('正式节点_'+('-' if branch=='q3' else 'q4_2').join(map(str, winner['nodes']))) or 'none')/'执行轨迹.npz'
106     with np.load(tracepath) as z: trace=z['trace'].copy(); warm=z['warmup'].copy()
107     assert len(trace)==48096 and np.array_equal(trace[:,0], np.arange(31*144, 365*144))
108     assert np.max(abs(trace[1:,8]-trace[:,-1,9]))<1e-7
109     assert abs(warm[-1,9]-trace[0,8])<1e-7
110     window=trace[(trace[:,0]>=START)&(trace[:,0]<STOP)].copy()
111     assert window.shape==(4320,16)
112     assert abs(trace[START-31*144-1,9]-window[0,8])<1e-7
113     with np.load(folder/'正式预测.npz') as z: q=z['q'].copy(); p=z['price'].copy()
114     audits=[]; pvdelta=np.zeros((120,144)); pricebase=np.zeros((120,432))
115     for issue in range(START//36, STOP//36):
116         i=issue-START//36
117         if branch!='q4_2':
118             pvfolder=folder/'预测'/('Q3 光伏' if branch=='q3' else 'Q3 同规则光伏')
119             # Q3 current archive names are inspected explicitly below.
120             if not pvfolder.exists(): pvfolder=folder/'预测/Q3 同规则光伏'
121             a, audit=check_record(pvfolder/f'{issue:04d}.npz', 'pv', root); audits.append(dict(asset='pv', **audit))
122             pvdelta[i]=(a['raw']-a['base'])/6
123             loadfolder=folder/'预测'/('Q3 负载_tree' if branch=='q3' else '继承 Q3 负载_tree')
124             if not loadfolder.exists(): loadfolder=folder/'预测/继承 Q3 负载_tree'
125             l, la=check_record(loadfolder/f'{issue:04d}.npz', 'load', root); audits.append(dict(asset='load', **la))
126             origin=issue*36; targets=origin+np.arange(1,433); past=data.prefix(origin).pv
127             latest=origin-(origin-targets%144)%144; ix=latest[None,:]-144*np.arange(7)[:,-1, None]
128             solar=np.nanmean(np.where(ix>=1, past[np.maximum(ix,1)], np.nan), axis=0)
129             solar[:144]=(1-winner['alpha'])*a['base']+winner['alpha']*a['raw']
130             assert np.max(abs(q[issue]-(l['q']-solar[:,-1, None]/6)))<1e-9
131             assert la['max_training_target']==origin
132             assert la['fit_issue']==issue
133         elif issue%4==0:
134             a, audit=check_record(folder/'预测/继承 Q2 净负荷_tree'/f'{issue:04d}.npz', 'net', root)
135             audits.append(dict(asset='net', **audit)); assert np.array_equal(a['q'], q[issue])
136         if branch!='q3':
137             a, audit=check_record(folder/'预测/价格'/f'{issue:04d}.npz', 'price', root)
138             audits.append(dict(asset='price', **audit)); assert np.array_equal(a['q'], p[issue])
139             _, base, _=pm.features(data, 'actual_price', issue*36)
140             assert np.max(abs(base-a['base']))<1e-12
141             pricebase[i]=base
142     # Fixed terminal valuation is available before window end, shared by all scenarios.
143     value=.9*(data.price.min() if branch=='q3' else p[(181-1)*4].min())
144     return dict(q=q, p=p, window=window, warm=warm, pvdelta=pvdelta, pricebase=pricebase, value=float(
    ↪ value), audits=audits,
145     sources={relative_name(folder/'正式预测.npz'): sha(folder/'正式预测.npz'), relative_name(trace
    ↪ path): sha(tracepath), relative_name(folder/'正式计算完成.json'): sha(folder/'正式计算完成.
    ↪ json')})

```

```

146
147 def simulate(branch, spec, ctx, data):
148     first=ctx['window'][0];E=float(first[8]);n=STOP-START
149     g=np.zeros(n+145);g0=g.copy();z=np.full(n+145,m.EMIN);version=np.full(n+145,-1,dtype=int)
150     g[0],g0[0],z[0],version[0]=first[[2,3,13,15]]
151     qi=int(np.flatnonzero(np.isclose(m.TAUS,spec['tau']))[0]);records=[];events=[]
152     plan_balance=plan_state=0.;positive_clips=0
153     for t in range(START,STOP):
154         k=t-START;c,b,h,u,ee,r=m.control(data.net[t],g[k],z[k],E,spec['beta'])
155         p=data.price[t%144] if branch=='q3' else data.actual_price[t]
156         row=[t,data.net[t],g[k],g0[k],c,b,h,u,E,ee,m.fee(g[k],g0[k],p),5*p*h,p,z[k],r,version[k]]
157         E=float(ee);records.append(row)
158         node=t%144==0 or (t%36==0 and t%144//6 in spec['nodes'])
159         if not node or t+1>=STOP:continue
160         issue=t//36;i=issue-START//36;targets=t+np.arange(1,433)
161         predicted=ctx['q'][issue,:qi].copy()
162         if branch!='q4_2':predicted[:144]=-(spec['alpha']-BASE[branch]['alpha'])*ctx['pvdelta'][:144]
163         if branch=='q3':pf=data.price[targets%144].copy()
164         else:
165             pf=ctx['p'][issue].copy()
166             if spec['residual_scale']!=1:pf=ctx['pricebase'][i]+spec['residual_scale']*(pf-ctx['pricebase'][i])
167             if spec['spread_scale']!=1:
168                 a=pf.reshape(3,144);mean=a.mean(axis=1,keepdims=True);pf=(mean+spec['spread_scale']*(a-mean)).ravel()
169             positive_clips+=int(np.sum(pf<1e-6));pf=np.maximum(pf,1e-6)
170             nt=144-t%144;original=None if t%144==0 else g0[k+1:k+1+nt].copy()
171             old=g[k+1:k+1+nt].copy();a,obj=m.lp(predicted,pf,E,original)
172             gp,cp,bp,up,ep,hp=a
173             plan_balance=max(plan_balance,float(np.max(abs(gp+bp+hp-predicted-cp-up))))
174             plan_state=max(plan_state,float(np.max(abs(ep-np.r_[E,ep[:-1]]-.9*cp+bp/.9))))
175             assert ep.min()>=1200-1e-6 and ep.max()<=10800+1e-6 and (cp+bp).max()<=m.CAP+1e-6
176             assert hp.max()<1e-7
177             g[k+1:k+1+nt]=gp[z[k+1:k+1+nt]=ep[:nt];version[k+1:k+1+nt]=t
178             if t%144==0:g0[k+1:k+145]=gp[:144]
179             events.append(dict(origin=t,first_target=t+1,slots=nt,initial=E,changed=bool(np.max(abs(old-gp[:nt]))>1e-6),
180                 forecast_sha256=m.digest(predicted),price_sha256=m.digest(pf),contract_sha256=m.digest(gp[:nt]),objective=obj))
181             a=np.asarray(records);t=a[:,0].astype(int);net,g,g0,c,b,h,u,e0,e1,pc,ec,p,z,r,v=a[:,1:].T
182             expected_contract=p*g0+1.5*p*np.maximum(g-g0,0)-.5*p*np.maximum(g0-g,0)
183             checks=dict(balance=float(abs(g+b+h-net-c-u).max()),state=float(abs(e1-e0-.9*c+b/.9).max()),
184                 continuity=float(abs(e1[:-1]-e0[1:]).max()),contract_fee=float(abs(pc-expected_contract).max()),
185                 emergency_fee=float(abs(ec-5*p*h).max()),plan_balance=plan_balance,plan_state=plan_state,
186                 simultaneous_slots=int(np.sum((c>1e-8)&(b>1e-8))),inventory_min=float(min(e0.min(),e1.min())),
187                 inventory_max=float(max(e0.max(),e1.max())),power_max=float((c+b).max()),future_contract_slots=int(np.sum(v>=t)))
188             for key in ['balance','state','continuity','contract_fee','emergency_fee','plan_balance','plan_state']:assert checks[key]<1e-6,(key,checks)
189             assert checks['simultaneous_slots']==checks['future_contract_slots']==0
190             assert checks['inventory_min']>=1200-1e-7 and checks['inventory_max']<=10800+1e-7 and checks['power_max']<=m.CAP+1e-7
191             assert np.array_equal(p,data.price[t%144] if branch=='q3' else data.actual_price[t])
192             row=dict(branch=branch,**spec,slots=n,cost_yuan=float((pc+ec).sum()),contract_yuan=float(pc.sum()),
193                 emergency_yuan=float(ec.sum()),emergency_kwh=float(h.sum()),unused_kwh=float(u.sum()),
194                 emergency_slots=int(np.sum(h>1e-8)),start_energy=float(e0[0]),end_energy=float(e1[-1]),
195                 adjustments=sum(e['origin']%144!=0 and e['changed'] for e in events),
196                 node_calls=sum(e['origin']%144!=0 for e in events),positive_price_clips=positive_clips,checks=checks)
197             row['terminal_adjusted_cost']=row['cost_yuan']-ctx['value']*row['end_energy']
198             return row,a,events
199
200 def main():
201     began=time.perf_counter();protocol=register();data=m.UnifiedData();ranking=representative(data)
202     dump(OUT/'审计/运行目录.json',dict(q4_runtime=relative_name(CORE),q3_runtime=relative_name(Q3),root=ROOT)))

```

```

203     allrows=[];branchmeta={}
204     for branch in BASE:
205         print('PREPARE',branch,flush=True);ctx=prepare(branch,data)
206         dump(OUT/'审计'/f'{branch}_预测来源核验.json',dict(sources=ctx['sources'],audits=ctx['audi
            ↳ ts']))
207         # Run and compare the baseline before any perturbation costs are inspected.
208         base,trace,events=simulate(branch,protocol['factors'][branch][0],ctx,data)
209         error=float(np.max(abs(trace-ctx['window'])));assert error<1e-7,(branch,'baseline
            ↳ mismatch',error)
210         repeat,trace2,_=simulate(branch,protocol['factors'][branch][0],ctx,data)
211         assert np.array_equal(trace,trace2)
212         first=ctx['window'][0]
213         branchmeta[branch]=dict(baseline_trace_max_abs_error=error,baseline_repeat_cost_error=ab
            ↳ s(base['cost_yuan']-repeat['cost_yuan']),
214             common_state=dict(inventory=float(first[8]),effective_contract=float(first[2]),midnigh
            ↳ t=t_original_contract=float(first[3]),
215                 reference_inventory=float(first[13]),version_origin=int(first[15]),terminal_value=c
            ↳ tx['value'],sources=ctx['sources'],
216                 forecast_records_checked=len(ctx['audits']))
217         for spec in protocol['factors'][branch]:
218             if spec['id']=='baseline':row,a,ev=base,trace,events
219             else:row,a,ev=simulate(branch,spec,ctx,data)
220             row['delta_cost_yuan']=row['cost_yuan']-base['cost_yuan']
221             row['delta_percent']=100*row['delta_cost_yuan']/base['cost_yuan']
222             row['delta_terminal_adjusted']=row['terminal_adjusted_cost']-base['terminal_adjusted
            ↳ _cost']
223             row['terminal_confound_yuan']=ctx['value']*(row['end_energy']-base['end_energy'])
224             folder=OUT/'逐情景结果'/branch/spec['id'];folder.mkdir(parents=True,exist_ok=True)
225             np.savez_compressed(folder/'轨迹.npz',trace=a);dump(folder/'结果.json',row);dump(fold
            ↳ er/'节点审计.json',ev)
226             allrows.append(row);print(branch,spec['id'],round(row['cost_yuan'],2),round(row['del
            ↳ ta_percent'],4),flush=True)
227     import scipy,sklearn
228     result=dict(protocol_sha256=sha(OUT/'实验协议_运行前冻结.json'),representative_month_ranking=r
            ↳ anking,branches=branchmeta,rows=allrows,
229             runtime=dict(python=platform.python_version(),numpy=np.__version__,scipy=scipy.__version__
            ↳ ,sklearn=sklearn.__version__,seconds=time.perf_counter()-began))
230     dump(OUT/'敏感性分析完整结果.json',result)
231     m.write_csv(OUT/'敏感性分析汇总.csv',({k:json.dumps(v,ensure_ascii=False) if
            ↳ isinstance(v,(dict,list)) else v for k,v in r.items()}) for r in allrows))
232     print('COMPLETE',len(allrows),'scenarios',time.perf_counter()-began,flush=True)
233
234 if __name__=='__main__':main()

```

#### 4.13 第 2 至 3 节 \_\_ 历史数据与储能基础.py

源码/代码/第 2 至 3 节 \_\_ 历史数据与储能基础.py

```

1  """ 第 2 至 3 节共同基础；旧三候选控制器已移除，当前调度入口见第 4 至 7 节文件。"""
2  import os,sys,json,hashlib,datetime,time
3  from pathlib import Path
4  from dataclasses import dataclass
5  os.environ.setdefault('OMP_NUM_THREADS','1')
6  os.environ.setdefault('OPENBLAS_NUM_THREADS','1')
7  ROOT=Path(__file__).resolve().parents[1]
8  sys.path.insert(0,str(ROOT/'依赖'))
9  import numpy as np
10 from scipy.optimize import linprog
11 from scipy.sparse import coo_matrix
12 from sklearn.ensemble import HistGradientBoostingRegressor
13 from sklearn.linear_model import QuantileRegressor
14 from sklearn.preprocessing import StandardScaler
15 from threadpoolctl import threadpool_limits
16 import lightgbm as lgb
17 TAUS=np.array([.5,.65,.8,.9,.95]); BETAS=[0.,.5,1.]
18 EMIN=1200.;EMAX=10800.;ETA=.9;CAP=5000/6;EPS=1e-6
19 JAN_END=31*144;FORMAL_END=365*144
20 START=datetime.datetime(2025,1,1)
21

```

```

22 def dump(path,obj):
23     p=Path(path);p.parent.mkdir(parents=True,exist_ok=True)
24     p.write_text(json.dumps(obj,ensure_ascii=False,indent=2,default=lambda x:x.item() if
    ↪ isinstance(x,np.generic) else str(x),allow_nan=False),encoding='utf-8')
25
26 def digest(a):return hashlib.sha256(np.ascontiguousarray(a).tobytes()).hexdigest()
27
28 @dataclass(frozen=True)
29 class Prefix:
30     net:np.ndarray
31     pv:np.ndarray
32     cutoff:int
33     def labels(self,indices):
34         ix=np.asarray(indices,dtype=int)
35         if ix.min()<1 or ix.max()>self.cutoff:raise ValueError('Training target outside observed
    ↪ prefix / initialization proxy')
36         return self.net[ix]
37
38 class Data:
39     def __init__(self):
40         p=ROOT/'输入/inputs.json'
41         self.raw=json.loads(p.read_text(encoding='utf-8'))
42         for source in self.raw['sources'].values():
43             local=ROOT/'输入'/Path(source['path']).name
44             if not local.exists() or
    ↪ hashlib.sha256(local.read_bytes()).hexdigest()!=source['sha256']:
45                 raise ValueError('Copied Excel changed or missing: regenerate audited inputs')
46         rows=self.raw['q1_records'];order=rows[-1:]+rows[:-1]
47         self.price=np.array([r['price'] for r in order])
48         self.reference=np.array([(r['load_kw']-r['pv_kw'])/6 for r in order])
49         rawpv=np.array(self.raw['pv_kw']).ravel()
50         self.net=np.r_[self.reference[0],(np.array(self.raw['load_kw']).ravel()-rawpv)/6]
51         self.pv=np.r_[rows[-1]['pv_kw']]
52         expected=[(START+datetime.timedelta(hours=6*i)).isoformat(sep=' ') for i in range(1460)]
53         if [r['issue_time'] for r in self.raw['forecasts']]!=expected:raise ValueError('Official
    ↪ issue timestamps differ')
54         self.official=np.array([[r[f'预报{h}小时'] for h in range(1,25)] for r in
    ↪ self.raw['forecasts']]])
55     def prefix(self,cutoff):
56         if not 0<=cutoff<len(self.net):raise ValueError('Observation cutoff outside supplied
    ↪ history')
57         n=self.net[:cutoff+1].copy();p=self.pv[:cutoff+1].copy()
58         n.flags.writeable=False;p.flags.writeable=False
59         return Prefix(n,p,int(cutoff))
60     def bulletin(self,issue,now):
61         if issue*36>now:raise ValueError('Unpublished official forecast')
62         a=self.official[issue].copy();a.flags.writeable=False;return a
63
64 def net_features(view,origin,targets,pv_pred=None,pv_bias=0.):
65     if view.cutoff!=origin:raise ValueError('Feature view must end at historical decision time')
66     t=np.asarray(targets);j=t%144;lead=(t-origin-1)//144
67     n=len(t);clock=2*np.pi*j/144
68     # Latest same-clock observations, including the current slot only when already known.
69     latest=origin-((origin-j)%144)
70     rows=np.maximum(latest[None,:]-144*np.arange(7)[: ,None],0)
71     h=view.net[rows]
72     dow=((t//144)+2)%7
73     x=np.column_stack([np.sin(clock),np.cos(clock),lead,*[(dow==d).astype(float) for d in
    ↪ range(6)],
74                     h[0]/100,h[:3].mean(0)/100,h.mean(0)/100,
75                     np.full(n,view.net[max(1,origin-143):origin+1].mean()/100),
76                     np.full(n,view.net[max(1,origin-431):origin+1].mean()/100)])
77     if pv_pred is not None:
78         avail=t-origin<=144
79         pf=view.pv[rows].mean(0);pf[avail]=pv_pred[(t-origin-1)[avail]]
80         short=np.full(n,view.net[max(1,origin-m+1):origin+1].mean()/100) for m in [1,3,6,36]]
81         x=np.column_stack([x,np.full(n,(origin%144)/144),(t-origin)/144,pf/600,
82                             np.full(n,pv_bias/100),*short,avail.astype(float)])
83     if not np.isfinite(x).all():raise ValueError('Non-finite historical features')
84     return x
85
86 def rearrange(q):
87     # Equal-weight isotonic least-squares projection (PAVA), not sorting values.

```

```

88     a=np.asarray(q).copy()
89     for row in a:
90         values=[];weights=[];sizes=[]
91         for z in row:
92             values.append(z);weights.append(1);sizes.append(1)
93             while len(values)>1 and values[-2]>values[-1]:
94                 w=weights[-2]+weights[-1];z=(weights[-2]*values[-2]+weights[-1]*values[-1])/w
95                 s=sizes[-2]+sizes[-1];values[-2:]=[z];weights[-2:]=[w];sizes[-2:]=[s]
96         row[:]=np.repeat(values,sizes)
97     return a
98
99
100
101
102
103
104
105 def fee(g,g0,p):
106     return p*np.minimum(g,g0)+.5*p*np.maximum(g0-g,0)+1.5*p*np.maximum(g-g0,0)
107
108 def terminal_value(p,mode='displacement'):
109     # 七项审阅决定第 4 项: 只允许冻结 Q2 的库存替代价值, 拒绝旧口径。
110     if mode!='displacement':raise ValueError('Q3 only supports p_min * eta terminal value')
111     return float(np.min(p))*ETA
112
113 def plan(q,p,E,g0=None,terminal=None):
114     if terminal is None:terminal=terminal_value(p)
115     q=np.asarray(q);p=np.asarray(p);n=len(q);nt=0 if g0 is None else len(g0);nv=5*n+nt
116     i=np.arange(n);rr=[i,i,i,i,i+n,i+n,i+n,i+n,(i+n)[1:]]
117     cc=[i,i+n,i+2*n,i+3*n,i+4*n,i+n,i+2*n,(i+4*n)[:1]]
118     vv=[np.ones(n),-np.ones(n),np.ones(n),-np.ones(n),np.ones(n),-ETA*np.ones(n),np.ones(n)/ETA,
119         ↪ -np.ones(n-1)]
119     a=coo_matrix((np.concatenate(vv),(np.concatenate(rr),np.concatenate(cc))),shape=(2*n,nv)).to_
120         ↪ csr()
121     rhs=np.r_[q,np.zeros(n)];rhs[n]=E
122     ur=[i,i];uc=[i+n,i+2*n];uv=[np.ones(n),np.ones(n)];ub=[np.full(n,CAP)]
123     obj=np.r_[p,np.full(2*n,EPS),np.zeros(2*n+nt)];obj[5*n-1]=-terminal
124     if nt:
125         j=np.arange(nt);base=np.asarray(g0)
126         # phi = max(1.5*p*g-.5*p*g0, .5*p*g+.5*p*g0)
127         for off,slope,bias in [(n,1.5,-.5),(n+nt,.5,.5)]:
128             ur.extend([j+off,j+off]);uc.extend([j,j+5*n]);uv.extend([slope*p[:nt],-np.ones(nt)])
129             ub.append(-bias*p[:nt]*base)
130             obj[nt]=0.;obj[5*n]=1.
131     u=coo_matrix((np.concatenate(uv),(np.concatenate(ur),np.concatenate(uc))),shape=(n+2*nt,nv))
132         ↪ .tocsr()
133     bounds=np.zeros((nv,2));bounds[:,1]=np.inf;bounds[n:3*n,1]=CAP;bounds[4*n:5*n]=[EMIN,EMAX]
134     res=linprog(obj,A_ub=u,b_ub=np.concatenate(ub),A_eq=a,b_eq=rhs,bounds=bounds,method='highs')
135     if not res.success:raise RuntimeError(res.message)
136     z=res.x[:5*n].reshape(5,n)
137     if np.max(abs(a@res.x-rhs))>1e-6:raise ValueError('LP physical failure')
138     return z
139
140 def control(net,g,z,E,beta):
141     r=EMIN+beta*(np.asarray(z)-EMIN);delta=np.asarray(g)-np.asarray(net)
142     c=np.minimum(np.maximum(delta,0),np.minimum(CAP,np.maximum((EMAX-E)/ETA,0)))
143     b=np.minimum(np.maximum(-delta,0),np.minimum(CAP,np.maximum(ETA*(E-r),0)))
144     h=np.maximum(-delta,0)-b;unused=np.maximum(delta,0)-c
145     end=E+ETA*c-b/ETA
146     return c,b,h,unused,end,r
147
148 def warmup(data,until=JAN_END):
149     E=6000.;trace=[]
150     for t in range(until):
151         g=max(data.reference[t%144],0);c,b,h,u,end,r=control(data.net[t],g,EMIN,E,0.)
152         trace.append([t,data.net[t],g,g,c,b,h,u,E,end,data.price[t%144]*g,5*data.price[t%144]*h])
153     E=float(end)
154     return E,max(data.reference[0],0),EMIN,np.asarray(trace)
155
156 def summary(trace,price):
157     a=np.asarray(trace);t=a[:,0].astype(int);p=price[t%144]
158     n,g,g0,c,b,h,u,e0,e1,pc,ec=a[:,1:].T

```

```

157     errors={'balance':float(np.max(abs(g+b+h-n-c-u))), 'inventory':float(np.max(abs(e1-e0-ETA*c+b
    ↪ /ETA))),
158     'continuity':float(np.max(abs(e1[:-1]-e0[1:]))), 'fee':float(np.max(abs(pc-fee(g,g0,p)))),
159     'emergency_fee':float(np.max(abs(ec-5*p*h)))}
160     if max(errors.values())>1e-6 or e0.min()<EMIN-1e-6 or e1.max()>EMAX+1e-6 or
    ↪ (c+b).max()>CAP+1e-6:
161         raise ValueError(str(errors))
162     cost=float(np.sum(pc+ec));end=float(e1[-1])
163     return dict(cost_yuan=cost,contract_cost_yuan=float(pc.sum()),emergency_cost_yuan=float(ec.s
    ↪ um()),
164     emergency_kwh=float(h.sum()),unused_kwh=float(u.sum()),start_energy=float(e0[0]),end_energ
    ↪ y=end,
165     adjusted_displacement=cost-terminal_value(price,'displacement')*end,slots=len(a),checks=er
    ↪ rors,
166     monthly=[{'month':m,'cost':float((pc+ec)[np.array([(START+datetime.timedelta(minutes=int(k
    ↪ )*10)),month==m for k in t])).sum()]} for m in
    ↪ range(1,13)])
167
168
169 def pv_features(view,origin,anchors,previous=None):
170     if view.cutoff!=origin:raise ValueError('PV history cutoff mismatch')
171     # Current entire segment is known under the user's explicit assumption.
172     current=view.pv[origin];an=np.r_[current,anchors]
173     leads=np.arange(1,145);hour=np.minimum(leads//6,23);pos=leads-hour*6;q=pos/6
174     target=origin+leads;delta=np.diff(an);base=an[hour]+q*delta[hour];env=4*q*(1-q)
175     lags=[view.pv[max(0,origin-m)] for m in [1,3,6]]
176     bias=0. if previous is None else current-previous[5]
177     clk=2*np.pi*(target%144)/144
178     X=np.column_stack([np.full(144,origin%144),leads,pos,an[hour],an[hour+1],delta[hour],
179     ↪ delta[np.maximum(hour-1,0)],delta[np.minimum(hour+1,23)],np.full(144,current),
180     ↪ [np.full(144,z) for z in lags],np.full(144,bias),np.sin(clk),np.cos(clk)])
181     return X,base,env,bias
182
183
184
185 # Sequential issue order is essential: a node uses the previous published median
186 # forecast's now-observed error. Only independent quantile fits run concurrently.
187

```

#### 4.14 第4至7节 \_\_ 合同优化与储能执行.py

源码/代码/第4至7节 \_\_ 合同优化与储能执行.py

```

1  """ 统一物理与合同账本；Q4 数量信息、初始化和库存交接完整继承前问。"""
2  from 第2至3节 __ 历史数据与储能基础 import *
3  from itertools import combinations
4  import csv
5
6  NODE_SETS=[tuple(c) for size in range(4) for c in combinations((6,12,18),size)]
7  JAN_START=19*144+1;JAN_STOP=30*144+1
8
9  class UnifiedData(Data):
10     def __init__(self):
11         super().__init__()
12         price=json.loads((ROOT/'输入/电价.json').read_text(encoding='utf-8'))
13         if hashlib.sha256((ROOT/'输入/附件4.xlsx').read_bytes()).hexdigest()!=price['source_sha25
    ↪ 6']:raise ValueError('Price Excel
    ↪ changed')
14         if price['dates']!=self.raw['dates']:raise ValueError('Price dates mismatch')
15         self.actual_price=np.r_[np.nan,np.array(price['actual_price']).ravel()]
16         self.load=np.r_[np.nan,np.array(self.raw['load_kw']).ravel()/6]
17     def january(self):
18         import copy
19         d=copy.copy(self)
20         for key in
    ↪ ['net','pv','load','actual_price']:setattr(d,key,getattr(self,key)[:JAN_END].copy())
21         d.official=self.official[:124].copy()
22         return d
23

```

```

24 def lp(q,p,E,g0=None):
25     """432 段 LP, 含冗余的预测应急变量; 正电价下其最优值应为 0."""
26     q=np.asarray(q);p=np.asarray(p);n=len(q);nt=0 if g0 is None else len(g0)
27     if n!=432 or p.shape!=(432,) or not np.isfinite(p).all() or np.min(p)<=0:raise
28     ↪ ValueError('Need 432 positive planning prices')
29     if not EMIN-1e-8<=E<=EMAX+1e-8:raise ValueError('Invalid initial inventory')
30     nv=6*n+nt;i=np.arange(n)
31     # 变量为 g,c,b,u,e,h, 以及前 nt 段的合同费用 z。
32     rr=[i,i,i,i,i,i+n,i+n,i+n,(i+n)[1:]]
33     cc=[i,i+n,i+2*n,i+3*n,i+5*n,i+4*n,i+n,i+2*n,(i+4*n)[:1]]
34     vv=[np.ones(n),-np.ones(n),np.ones(n),-np.ones(n),np.ones(n),np.ones(n),-ETA*np.ones(n),np.o
35     ↪ nes(n)/ETA,-np.ones(n-1)]
36     A=coo_matrix((np.concatenate(vv),(np.concatenate(rr),np.concatenate(cc))),shape=(2*n,nv)).to
37     ↪ csr()
38     rhs=np.r_[q,np.zeros(n)];rhs[n]=E
39     ur=[i,i];uc=[i+n,i+2*n];uv=[np.ones(n),np.ones(n)];ub=[np.full(n,CAP)]
40     obj=np.r_[p,np.full(2*n,EPS),np.zeros(2*n),5*p,np.zeros(nt)];obj[5*n-1]=-ETA*p.min()
41     if nt:
42         j=np.arange(nt);g0=np.asarray(g0)
43         for offset,slope,bias in [(n,.5,.5),(n+nt,1.5,-.5)]:
44             ur.extend([j+offset,j+offset]);uc.extend([j,j+6*n]);uv.extend([slope*p[:nt],-np.ones
45             ↪ (nt)]);ub.append(-bias*p[:nt]*g0)
46         obj[:nt]=0;obj[6*n:]=1
47         U=coo_matrix((np.concatenate(uv),(np.concatenate(ur),np.concatenate(uc))),shape=(n+2*nt,nv))
48         ↪ .tocsr()
49         bounds=np.zeros((nv,2));bounds[:,1]=np.inf;bounds[n:3*n,1]=CAP;bounds[4*n:5*n]=[EMIN,EMAX]
50         result=linprog(obj,A_eq=A,b_eq=rhs,A_ub=U,b_ub=np.concatenate(ub),bounds=bounds,method='high
51         ↪ s')
52         if not result.success:raise RuntimeError(result.message)
53         z=result.x[:6*n].reshape(6,n)
54         if abs(A@result.x-rhs).max()>1e-6:raise ValueError('LP equality failure')
55         return z,float(result.fun)
56
57 def initialize(data,branch,until):
58     """ 四问沿用共同物理预热。Q4 首段缺真实价格只不计预热费用, 不静置电池或改变库存。"""
59     E=6000.;records=[]
60     for t in range(until):
61         g=max(data.reference[t%144],0);c,b,h,u,ee,_=control(data.net[t],g,EMIN,E,0.)
62         p=data.price[t%144] if branch=='q3' else (0. if t==0 else data.actual_price[t])
63         records.append([t,data.net[t],g,g,c,b,h,u,E,ee,p*g,5*p*h]);E=float(ee)
64     return E,np.asarray(records)
65
66 def replay(data,q,prices,branch,tau,beta,nodes,jan=False,end=None,keep_plans=False,record_events
67 ↪ =True):
68     if branch not in ['q3','q4_2','q4_3']:raise ValueError(branch)
69     nodes=tuple(nodes)
70     if nodes not in NODE_SETS or (branch=='q4_2' and nodes):raise ValueError('Invalid branch
71     ↪ node set')
72     start=JAN_START if jan else JAN_END;stop=(JAN_STOP if jan else FORMAL_END) if end is None
73     ↪ else end
74     begin=start-1 if jan else start
75     E,warm=initialize(data,branch,begin)
76     g=np.zeros(stop+145);g0=g.copy();z=np.full(stop+145,EMIN);version=np.full(stop+145,-1,int)
77     g[begin]=g0[begin]=max(data.reference[begin%144],0)
78     records=[];plans=[];events=[];bridge=[]
79     def decision(o,initial):
80         issue=o//36;nt=144-o%144;targets=o+np.arange(1,433)
81         pf=data.price[targets%144] if branch=='q3' else prices[issue]
82         predicted=q[issue,:tau]
83         if not np.isfinite(predicted).all():raise ValueError('Missing forecast')
84         original=None if o%144==0 else g0[o+1:o+1+nt].copy()
85         old=g[o+1:o+1+nt].copy();oldref=z[o+1:o+1+nt].copy()
86         a,obj=lp(predicted,pf,initial,original)
87         # 日内直接 LP, 不调用三候选或 1SE。只安装当天原合同剩余区间。
88         g[o+1:o+1+nt]=a[0,:nt];z[o+1:o+1+nt]=a[4,:nt];version[o+1:o+1+nt]=o
89         if o%144==0:g0[o+1:o+145]=a[0,:144]
90         changed=bool(np.max(abs(old-a[0,:nt]))>1e-6)
91         ev=dict(origin=o,first_target=o+1,installed_slots=nt,changed=changed,initial_predicted=i
92         ↪ nitial,
93             observation_cutoff=o,terminal=float(ETA*pf.min()),objective=obj,
94             q_sha=digest(predicted),price_sha=digest(pf),contract_sha=digest(a[0,:nt]))
95         if record_events:events.append(ev)

```

```

86         if keep_plans: plans.append(dict(**ev, targets=targets, values=a, forecast=predicted.copy(),
87         ↪ price=pf.copy(), g0=original, previous_contract=old, previous_reference=oldref))
88     return ev
89     for t in range(begin, stop):
90         isnode=(t%144==0 or (t%36==0 and (t%144)//6 in nodes))
91         # 年末最后已结算段不再生成无效新合同。
92         isnode=isnode and t+1<stop
93         ev=None
94         c,b,h,u,ee,r=control(data.net[t],g[t],z[t],E,beta)
95         p=data.price[t%144] if branch=='q3' else data.actual_price[t]
96         row=[t,data.net[t],g[t],g0[t],c,b,h,u,E,ee,fee(g[t],g0[t],p),5*p*h,p,z[t],r,version[t]]
97         E=float(ee)
98         if isnode: ev=decision(t,E)
99         if ev is not None:
100             ev['initial_actual']=E; ev['handoff_error']=E-ev['initial_predicted']
101             if keep_plans: plans[-1].update(initial_actual=E, handoff_error=ev['handoff_error'])
102             if t>=start: records.append(row)
103             else: bridge.append(row)
104         trace=np.asarray(records); endE=float(trace[-1,9]); actual=float(trace[:,10:12].sum())
105         # 一月选型公共末端系数: 最后公共评价节点当时发布的价格预测, 候选间相同。
106         score_v=.33417 if branch=='q3' else float(ETA*np.min(prices[((JAN_STOP-1)//144)*4])) if jan
107         ↪ else None
108         summary=dict(branch=branch, tau=float(TAUS[tau]), tau_index=tau, beta=float(beta), nodes=list(no
109         ↪ des), slots=len(trace), cost_yuan=actual,
110         contract_cost=float(trace[:,10].sum()), emergency_cost=float(trace[:,11].sum()), emergency
111         ↪ _kwh=float(trace[:,6].sum()),
112         start_energy=float(trace[0,8]), end_energy=endE, throughput=float(trace[:,4:6].sum()),
113         score_terminal=score_v, selection_score=actual-score_v*endE+EPS*float(trace[:,4:6].sum())
114         ↪ if jan else None,
115         adjustments=sum(e['origin']%144!=0 and e['changed'] for e in
116         ↪ events), node_calls=sum(e['origin']%144!=0 for e in events))
117     return dict(trace=trace, summary=summary, plans=plans, events=events, warmup=warm, bridge=np.asar
118     ↪ ray(bridge))
119
120 def write_csv(path, rows):
121     rows=iter(rows)
122     try: first=next(rows)
123     except StopIteration: return
124     with Path(path).open('w', encoding='utf-8-sig', newline='') as f:
125         w=csv.DictWriter(f, fieldnames=list(first)); w.writeheader(); w.writerow(first); w.writerows
126         ↪ (rows)
127
128 def save_replay(folder, result):
129     folder=Path(folder); folder.mkdir(parents=True, exist_ok=True)
130     np.savez_compressed(folder/'执行轨迹.npz', trace=result['trace'], warmup=result['warmup'], bridg
131     ↪ e=result['bridge'])
132     dump(folder/'费用汇总.json', result['summary']); dump(folder/'节点审计.json', result['events'])
133     cols=['slot', 'net', 'purchase', 'original', 'charge', 'discharge', 'emergency', 'unused', 'inventor
134     ↪ y_start', 'inventory_end', 'contract_fee', 'emergency_fee', 'price', 'reference', 'reserve', 'v
135     ↪ ersion_origin']
136     write_csv(folder/'执行轨迹.csv', (dict(zip(cols, r)) for r in result['trace']))
137     ps=result['plans']
138     if ps:
139         original=np.full((len(ps), 144), np.nan); sizes=np.zeros(len(ps), int)
140         for i, p in enumerate(ps):
141             if p['g0'] is not None: sizes[i]=len(p['g0']); original[i, :sizes[i]]=p['g0']
142         np.savez_compressed(folder/'规划档案.npz', origins=[p['origin'] for p in
143         ↪ ps], initial=[p['initial_predicted'] for p in ps],
144         actual_initial=[p['initial_actual'] for p in ps], targets=np.stack([p['targets'] for
145         ↪ p in ps]), values=np.stack([p['values'] for p in ps]),
146         price=np.stack([p['price'] for p in ps]), forecast=np.stack([p['forecast'] for p in
147         ↪ ps]), g0=original, remaining=sizes)
148     days=[]; a=result['trace']; slots=a[:,0].astype(int)
149     for day in np.unique(slots//144):
150         r=a[slots//144==day]; days.append(dict(day=int(day), cost=float(r[:,10:12].sum()), original
151         ↪ _cost=float(np.sum(r[:,12]*r[:,3])),
152         increase=float(np.sum(1.5*r[:,12]*np.maximum(r[:,2]-r[:,3], 0))), refund=float(np.sum(
153         ↪ .5*r[:,12]*np.maximum(r[:,3]-r[:,2], 0))),
154         emergency=float(r[:,11].sum()), end_energy=float(r[-1,9]))
155     write_csv(folder/'每日费用.csv', days)

```

## 4.15 第 5 至 7 节 \_ 联合选型与正式回放.py

源码/代码/第 5 至 7 节 \_ 联合选型与正式回放.py

```

1  """ 入口: Q3 1800 组、Q4-2 45 组、Q4-3 1800 组; 先一月选型再正式期。"""
2  import argparse,platform,itertools
3  from 第 4 至 7 节 _ 合同优化与储能执行 import *
4  from 第 6 节 _Q3 逐节点负载与光伏预测 import pv_forecasts,load_forecasts,q3_net_from_load,sha
5  from 第 7 节 _Q4 每日负载与电价预测 import
    ↳ pv_archive_closed,series_archive,q4_net_from_load,q4_q2_net_archive
6  KINDS=['history','linear','tree'];ALPHAS=[0.,.25,.5,.75,1.];TOL=1e-6
7
8  def protocol(branch):
9      return dict(schema='Q1Q4_INHERITANCE_V3',branch=branch,
10         q3='current-slot known; old initialization; load quantile minus PV; every-node fit;
            ↳ direct LP; joint225x8',
11         q4='Q4-2 inherits frozen Q2 quantity model and midnight fit; Q4-3 inherits Q3 load minus
            ↳ PV and every-node fit; both inherit current-slot-known quantities, warmup and actual
            ↳ handoff inventory; price model daily from completed price history',
12         execution='all branches use approved current-slot-known quantity assumption; future
            ↳ actual quantities and future actual prices never enter planning',
13         horizon=432,first_target='origin+1',terminal='eta*minimum planning price within432',
14         january_score='Jan20 00:10 through Jan31 00:10 exclusive; common terminal coefficient
            ↳ from Jan31 00:00 price forecast',
15         warmup='fixed typical-day positive net purchase, beta0; all branches retain marked
            ↳ initial quantity proxy; Q4 missing first price only leaves that warmup fee unbilled,
            ↳ no physical idling',
16         kinds=KINDS,taus=TAUS.tolist(),betas=BETAS,alphas=ALPHAS,node_sets=NODE_SETS,
17         score_tie='within1e-6: emergency, fewer nodes, tau,beta,alpha,kind order,node-set order',
18         source_sha256={p.name:sha(p) for p in sorted((ROOT/'代码').glob('*.*py'))},
19         input_sha256={p.name:sha(p) for p in sorted((ROOT/'输入').iterdir()) if p.is_file()},
20         libraries={n:__import__(n).__version__ for n in
            ↳ ['numpy','scipy','sklearn','lightgbm','joblib']},python=platform.python_version())
21
22  def output_folder(out,cfg):
23      out=Path(out);out.mkdir(parents=True,exist_ok=True);p=out/'实验协议.json'
24      # 序列化一次规范 tuple, 保证同协议续跑时不会因 tuple/list 误拒绝。
25      cfg=json.loads(json.dumps(cfg,ensure_ascii=False))
26      if p.exists() and json.loads(p.read_text(encoding='utf-8'))!=cfg:raise ValueError('Output
            ↳ belongs to a different configuration')
27      dump(p,cfg);return out,cfg
28
29  class Forecasts:
30      def __init__(self,data,out,cfg,branch,upto):
31          self.data=data;self.out=out;self.cfg=cfg;self.branch=branch;self.upto=upto;self.pv=None;
            ↳ self.price=None;self.loads={}
32          if branch!='q3':self.price,_=series_archive(data,out/'预测/价格',cfg,'actual_price','mean'
            ↳ ',upto)
33          if branch in ['q3','q4_3']:self.pv=pv_forecasts(data,out/'预测/Q3 同规则光伏',cfg,upto)
34      def get(self,kind,alpha):
35          if self.branch in ['q3','q4_3']:
36              if kind not in self.loads:
37                  self.loads[kind]=load_forecasts(self.data,self.out/'预测'/f'继承Q3负载_{kind}',se
                    ↳ lf.cfg,kind,self.upto)[0]
38                  return q3_net_from_load(self.data,self.loads[kind],self.pv,alpha)
39              if kind not in self.loads:
40                  self.loads[kind]=q4_q2_net_archive(self.data,self.out/'预测'/f'继承Q2净负荷_{kind}',se
                    ↳ lf.cfg,kind,self.upto)[0]
41              return self.loads[kind]
42
43
44  def choose(rows):
45      minimum=min(r['selection_score'] for r in rows);ties=[r for r in rows if
            ↳ r['selection_score']<=minimum+TOL]
46      return min(ties,key=lambda r:(r['emergency_kwh'],len(r['nodes']),r['tau'],r['beta'],r['alpha'
            ↳ ''],KINDS.index(r['kind']),NODE_SETS.index(tuple(r['nodes'])))))
47
48  def select(branch,out):
49      out,cfg=output_folder(out,protocol(branch));data=UnifiedData().january();forecasts=Forecasts
            ↳ (data,out,cfg,branch,124);rows=[]
50      alphas=[0.] if branch=='q4_2' else ALPHAS;sets=[()] if branch=='q4_2' else NODE_SETS
51      for kind in KINDS:

```

```

52     for alpha in alphas:
53         q=forecasts.get(kind,alpha)
54         for qi,beta,nodes in itertools.product(range(5),BETAS,sets):
55             name=f'{kind}_t{TAUS[qi]:g}_b{beta:g}_a{alpha:g}_s'+('-'.join(map(str,nodes)) or
56                 ↪ 'none');folder=out/'一月候选'/name
57             manifest=folder/'记录指纹.json'
58             if manifest.exists():
59                 hashes=json.loads(manifest.read_text(encoding='utf-8'))
60                 if not all(sha(folder/p)==h for p,h in hashes.items()):raise
61                 ↪ ValueError('Corrupt candidate checkpoint')
62                 summary=json.loads((folder/'费用汇总.json').read_text(encoding='utf-8'))
63             else:
64                 result=replay(data,q,forecasts.price,branch,qi,beta,nodes,jan=True);summary=
65                 ↪ result['summary'];summary.update(kind=kind,alpha=alpha)
66                 save_replay(folder,result);dump(manifest,{p.name:sha(p) for p in
67                 ↪ folder.iterdir() if p.is_file()})
68             rows.append(summary)
69             print('Selection progress',branch,kind,alpha,len(rows),flush=True)
70             expected=45 if branch=='q4_2' else 1800
71             if len(rows)!=expected:raise ValueError('Incomplete grid')
72             write_csv(out/'一月全部策略比较.csv',({**r,'nodes':','.join(map(str,r['nodes']))} for r in
73             ↪ rows))
74             winner=choose(rows);winner=dict(winner,configuration=cfg,candidate_count=expected,selection_
75             ↪ period_only='January',frozen_at=datetime.datetime.now().astimezone().isoformat())
76             dump(out/'选定参数.json',winner)
77             print('SELECTION FINISHED',branch,winner['kind'],winner['tau'],winner['beta'],winner['alpha']
78             ↪ ,winner['nodes'],flush=True)
79             return winner
80
81 def formal(branch,out,selection,end_day=365,node_diagnostics=False):
82     if not 32<=end_day<=365:raise ValueError('Formal end-day')
83     out,cfg=output_folder(out,protocol(branch));selection=Path(selection);winner=json.loads(sele
84     ↪ ction.read_text(encoding='utf-8'))
85     if winner['configuration']!=cfg or winner['candidate_count']!=(45 if branch=='q4_2' else
86     ↪ 1800):raise ValueError('Selection is not from this full January protocol')
87     dump(out/'正式参数锁定.json',dict(selection_sha256=sha(selection),selected=winner,locked_befo
88     ↪ re_formal_forecasts=datetime.datetime.now().astimezone().isoformat()))
89     data=UnifiedData();f=Forecasts(data,out,cfg,branch,end_day*4);q=f.get(winner['kind'],winner[
90     ↪ 'alpha']);allrows=[]
91     node_sets=([()] if branch=='q4_2' else NODE_SETS) if node_diagnostics else
92     ↪ [tuple(winner['nodes'])]
93     for nodes in node_sets:
94         result=replay(data,q,f.price,branch,winner['tau_index'],winner['beta'],nodes,end=end_day
95         ↪ *144,keep_plans=True)
96         result['summary'].update(kind=winner['kind'],alpha=winner['alpha'],is_january_selected_n
97         ↪ odes=list(nodes)==winner['nodes'])
98         folder=out/('正式节点_'+('-'.join(map(str,nodes)) or 'none'));save_replay(folder,result)
99         from 第9节_独立核验 import verify_replay
100         checks=verify_replay(result,data,branch);dump(folder/'物理费用核验.json',checks);allrows.
101         ↪ append(result['summary'])
102     write_csv(out/'正式期节点对照.csv',({**r,'nodes':','.join(map(str,r['nodes']))} for r in
103     ↪ allrows))
104     np.savez_compressed(out/'正式预测.npz',q=q,price=np.array([]) if f.price is None else f.price)
105     dump(out/'正式计算完成.json',dict(end_day=end_day,configuration=cfg,selected=winner,rows=allr
106     ↪ ows,scope='Fixed January-selected parameters; other node sets are descriptive
107     ↪ comparisons'))
108     print('FORMAL FINISHED',branch,flush=True)
109
110 if __name__=='__main__':
111     p=argparse.ArgumentParser();p.add_argument('branch',choices=['q3','q4_2','q4_3']);p.add_argu
112     ↪ ment('stage',choices=['select','formal']);p.add_argument('--output',type=Path,required=T
113     ↪ rue)
114     p.add_argument('--selection',type=Path);p.add_argument('--end-day',type=int,default=365);p.a
115     ↪ dd_argument('--node-diagnostics',action='store_true',help='Optional descriptive
116     ↪ comparison only; never reselect the January winner');a=p.parse_args()
117     if a.stage=='select':select(a.branch,a.output)
118     else:
119         if a.selection is None:p.error('formal requires --selection')
120         formal(a.branch,a.output,a.selection,a.end_day,a.node_diagnostics)

```

## 4.16 第 6 节 \_Q3 逐节点负载与光伏预测.py

源码/代码/第 6 节 \_Q3 逐节点负载与光伏预测.py

```

1  """Q3负载预测与光伏降尺度分别建模，再合成规划净需电量。每条档案由原发布节点产生；续跑必须匹配输入及代码指纹。
   ↪ """
2  from 第 2 至 3 节 _ 历史数据与储能基础 import *
3  import joblib
4  from concurrent.futures import ThreadPoolExecutor
5
6  def sha(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()
7
8  def save_record(path,arrays,audit,cfg,model):
9      np.savez_compressed(path,**arrays)
10     weight=path.with_suffix('.joblib');joblib.dump(model,weight,compress=1)
11     dump(path.with_suffix('.json'),dict(configuration=cfg,audit=audit,prediction_sha256=sha(path)
   ↪ ),model_sha256=sha(weight)))
12
13  def read_record(path,cfg):
14     meta=json.loads(path.with_suffix('.json').read_text(encoding='utf-8'))
15     if meta['configuration']!=cfg or sha(path)!=meta['prediction_sha256'] or
   ↪ sha(path.with_suffix('.joblib'))!=meta['model_sha256']:
16         raise ValueError('模型/预测档案的输入、配置、代码或文件指纹不匹配: '+str(path))
17     with np.load(path) as a:arrays={k:a[k].copy() for k in a.files}
18     return arrays,meta['audit']
19
20  def pv_forecasts(data,folder,cfg,upto,ignore_cache=False):
21     """ 官方锚点按目标整点功率解释；样本不足 2000 时保持插值。"""
22     folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
23     xs=[];bases=[];envs=[];raws=[];audits=[];biases=[]
24     for issue in range(upto):
25         path=folder/f'{issue:04d}.npz';o=issue*36
26         if path.exists() and not ignore_cache:
27             a,audit=read_record(path,cfg);X,base,env,raw,bias=[a[k] for k in
   ↪ ['x','base','env','raw','bias']]
28         else:
29             view=data.prefix(o);previous=None if issue==0 else data.bulletin(issue-1,o)
30             X,base,env,bias=pv_features(view,o,data.bulletin(issue,o),previous)
31             tx=[];ty=[];label_indices=[]
32             for j in range(max(0,issue-120),issue):
33                 targets=j*36+np.arange(1,145);mask=(targets<=o)&(envs[j]>0)
34                 if mask.any():
35                     tx.append(xs[j][mask]);ty.append((view.pv[targets[mask]]-bases[j][mask])/env
   ↪ s[j][mask]);label_indices.extend(targets[mask].tolist())
36             raw=base.copy();model=None;rows=sum(map(len,ty))
37             if rows>=2000:
38                 model=lgb.LGBMRegressor(objective='regression_l1',n_estimators=240,learning_rate
   ↪ =.035,num_leaves=31,
39                 min_child_samples=50,reg_alpha=.1,reg_lambda=1.,random_state=2026,n_jobs=1,
40                 deterministic=True,force_col_wise=True,verbosity=-1)
41                 with threadpool_limits(limits=1):model.fit(np.vstack(tx),np.concatenate(ty));raw
   ↪ +=env*model.predict(X)
42             raw=np.maximum(raw,0)
43             audit=dict(issue=issue,origin=o,training_cutoff=o,max_training_target=max(label_inde
   ↪ ces,default=0),rows=rows,
44                 training_features_sha=digest(np.vstack(tx)) if tx else
   ↪ None,training_labels_sha=digest(np.concatenate(ty)) if ty else None,
45                 feature_sha=digest(X),fallback=rows<2000,interpretation='Official values are
   ↪ target-hour power anchors; not hourly average energy')
46             save_record(path,dict(x=X,base=base,env=env,raw=raw,bias=bias),audit,cfg,dict(model=
   ↪ model,fallback=rows<2000))
47             if issue%40==0:print('PV fresh',issue,'rows',rows,flush=True)
48             xs.append(X);bases.append(base);envs.append(env);raws.append(raw);biases.append(float(bi
   ↪ as));audits.append(audit)
49     return dict(base=np.array(bases),raw=np.array(raws),bias=np.array(biases),audit=audits)
50
51  def fit_quantiles(kind,X,y,future):
52     scaler=StandardScaler().fit(X);a=scaler.transform(X);b=scaler.transform(future)
53     def one(tau):
54         # 同一稀疏线性分位数目标，使用 HiGHS 内点法；五分位数等价核另存。
55         if kind=='linear':model=QuantileRegressor(quantile=float(tau),alpha=.002,solver='highs-i
   ↪ pm')

```

```

56         elif kind=='tree':model=HistGradientBoostingRegressor(loss='quantile',quantile=float(tau)
57         ↪ ),max_iter=60,max_leaf_nodes=7,
58         ↪ min_samples_leaf=30,l2_regularization=1.,early_stopping=False,random_state=2)
59         ↪ 0260911)
60         else:raise ValueError('Unknown model family')
61         model.fit(a,y/100);return model,model.predict(b)*100
62     with threadpool_limits(limits=1):
63         with ThreadPoolExecutor(max_workers=5) as pool:results=list(pool.map(one,TAUS))
64     return rearrange(np.column_stack([p for m,p in results])),dict(scaler=scaler,models=[m for
65     ↪ m,p in results])
66
67 def clock_aligned_history(labels,origins,current_origin):
68     """ 按未来 24 小时块和实际钟点对齐; 使用同一批完整成熟的 432 标签。"""
69     aligned=[np.roll(y.reshape(3,144),-((current_origin-o)%144),axis=1).ravel() for y,o in
70     ↪ zip(labels,origins)]
71     return rearrange(np.quantile(np.asarray(aligned),TAUS,axis=0).T)
72
73 def q3_load_features(data,origin):
74     """Q3 保留段初当量量可知假设; 负载特征仅取截至 origin 的负载和日历。"""
75     if not 1 <= origin < len(data.load):raise ValueError('Invalid Q3 load cutoff')
76     past=data.load[:origin+1].copy();past.flags.writeable=False
77     t=origin+np.arange(1,433);j=t%144
78     latest=origin-(origin-j)%144;rows=latest[None,:]-144*np.arange(7)[:,:None]
79     h=np.where(rows>=1,past[np.maximum(rows,1)],np.nan)
80     current=past[max(1,origin-143):origin+1];clk=2*np.pi*j/144;dow=(t//144+2)%7
81     X=np.column_stack([np.sin(clk),np.cos(clk),*((dow==d).astype(float) for d in range(6)),
82     ↪ (t-origin-1)//144,(t-origin)/432,np.full(432,(origin%144)/144),
83     ↪ h[0],np.nanmean(h[:3],axis=0),np.nanmean(h,axis=0),np.nanstd(h,axis=0),
84     ↪ *[np.full(432,current[-k:].mean()) for k in [1,6,36,144]],np.full(432,current.std())])
85     if not np.isfinite(X).all():raise ValueError('Non-finite Q3 load features')
86     return X,t
87
88 def load_forecasts(data,folder,cfg,kind,upto,ignore_cache=False):
89     """ 每个 0/6/12/18 点拟合负载分位数; alpha 不进入训练, 三日标签须完整成熟。"""
90     folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
91     xs={};q=np.full((upto,432,5),np.nan);audits=[]
92     localcfg=dict(cfg,prediction_target='load_kwh',load_kind=kind,pv_as_feature=False)
93     for issue in range(28,upto):
94         o=issue*36;path=folder/f'{issue:04d}.npz'
95         if path.exists() and not ignore_cache:
96             a,audit=read_record(path,localcfg);X=a['x'];pred=a['q']
97         else:
98             X,target=q3_load_features(data,o)
99             mature=list(range(max(28,issue-120),issue-12+1));tx=None;y=None
100             if mature:
101                 past=data.load[:o+1].copy()
102                 labels=[past[j*36+np.arange(1,433)].copy() for j in mature]
103                 if any(j*36+432>o for j in mature):raise ValueError('Immature load labels')
104                 tx=np.vstack([xs[j] for j in mature]);y=np.concatenate(labels)
105                 if kind=='history':
106                     pred=clock_aligned_history(labels,[j*36 for j in mature],o)
107                     model=dict(kind='clock_aligned_history',aligned_quantiles=pred)
108                 else:pred,model=fit_quantiles(kind,tx,y,X)
109             else:
110                 days=o//144;hist=data.load[1:days*144+1].reshape(days,144)
111                 pred=np.quantile(hist[-30:],TAUS,axis=0).T[(targets-1)%144]
112                 model=dict(kind='load_cold_start',quantiles=pred)
113                 pred=np.maximum(pred,0.)
114                 audit=dict(issue=issue,origin=o,fit_issue=issue,training_cutoff=o,feature_cutoff=o,
115                 ↪ prediction_target='load_kwh',pv_as_feature=False,training_origins=mature,
116                 ↪ max_training_target=mature[-1]*36+432 if mature else 0,rows=len(mature)*432,
117                 ↪ training_features_sha=digest(tx) if mature else None,
118                 ↪ training_labels_sha=digest(y) if mature else None,feature_sha=digest(X),
119                 ↪ predictor_kind=kind,fallback=not bool(mature))
120                 save_record(path,dict(x=X,q=pred),audit,localcfg,model)
121                 if issue%20==0:print('Q3 LOAD fresh',kind,issue,'rows',audit['rows'],flush=True)
122                 xs[issue]=X;q[issue]=pred;audits.append(audit)
123     return q,audits
124
125 def q3_net_from_load(data,loadq,pv,alpha):
126     """ 规划净需电量 = 负载分位数-光伏点预测电量; 不称为真实净负荷分位数。"""

```

```

125     if alpha not in [0.,.25,.5,.75,1.]:raise ValueError('Unregistered PV shrinkage')
126     result=loadq.copy()
127     for issue in range(28,len(loadq)):
128         origin=issue*36;past=data.prefix(origin).pv;targets=origin+np.arange(1,433);j=targets%144
129         latest=origin-(origin-j)%144;rows=latest[None,:]-144*np.arange(7)[:,None]
130         solar=np.nanmean(np.where(rows>=1,past[np.maximum(rows,1)],np.nan),axis=0)
131         solar[:144]=(1-alpha)*pv['base'][issue]+alpha*pv['raw'][issue]
132         result[issue]=solar[:None]/6
133     return result

```

## 4.17 第 7 节 \_Q4 每日负载与电价预测.py

源码/代码/第 7 节 \_Q4 每日负载与电价预测.py

```

1  """Q4 数量模块继承 Q2/Q3;新增价格模型每日午夜拟合,使用已完成价格历史."""
2  from 第 6 节 _Q3 逐节点负载与光伏预测 import *
3
4  def past_array(data,asset,origin):
5      arr=getattr(data,asset)
6      if not 1<=origin<=len(arr):raise ValueError('Invalid completed-history origin')
7      a=arr[:origin].copy();a.flags.writeable=False
8      return a
9
10 def features(data,asset,origin):
11     past=past_array(data,asset,origin);cut=origin-1;t=origin+np.arange(1,433);j=t%144
12     latest=cut-(cut-j)%144;rows=latest[None,:]-144*np.arange(7)[:None]
13     h=np.where(rows>=1,past[np.maximum(rows,1)],np.nan)
14     historical=np.nanmean(h,axis=0);current=past[max(1,origin-144):origin]
15     clk=2*np.pi*j/144;dow=(t//144+2)%7
16     X=np.column_stack([np.sin(clk),np.cos(clk),*((dow==d).astype(float) for d in range(6)),
17                       (t-origin-1)//144,(t-origin)/432,np.full(432,(origin%144)/144),h[0],np.nanmean(h[:3],axis=0),
18                       *[(np.full(432,current[-k:].mean()) for k in [1,6,36,144]),np.full(432,current.std())])
19     if asset=='actual_price':
20         # 只用完整历史自然日;首日缺少 00:00 实价,因此第 0 日不能列作完整日。
21         days=np.arange(max(1,origin//144-7),origin//144)
22         if not len(days):raise ValueError('No complete price day')
23         baseline=past[days[:None]*144+j[None,:]].mean(axis=0)
24         X=np.column_stack([X,baseline])
25     else:baseline=np.zeros(432)
26     if not np.isfinite(X).all():raise ValueError('Invalid Q4 feature')
27     return X,baseline,t
28
29 def pv_features_closed(data,issue):
30     origin=issue*36;anchors=data.bulletin(issue,origin)
31     if origin==0:
32         # 首个小时没有真实起点,采用首个官方未来锚值回推,仅为预测,不制造真实观测或费用。
33         cur=float(anchors[0]);lags=[cur]*3;bias=0.
34     else:
35         past=past_array(data,'pv',origin);cur=float(past[-1]);lags=[float(past[max(1,origin-m]))
36                               for m in [1,3,6]]
37         previous=data.bulletin(issue-1,origin)
38         # 最近完成段是前节点后第 35 段,对应 5 小时 50 分的官方线性插值。
39         old_forecast=previous[4]+5/6*(previous[5]-previous[4]);bias=cur-old_forecast
40         an=np.r_[cur,anchors];leads=np.arange(1,145);hour=np.minimum(leads//6,23);pos=leads-hour*6;v
41         ↪ =pos/6
42         delta=np.diff(an);base=an[hour]+v*delta[hour];env=4*v*(1-v);clk=2*np.pi*((origin+leads)%144)
43         ↪ /144
44         X=np.column_stack([np.full(144,origin%144),leads,pos,an[hour],an[hour+1],delta[hour],delta[n
45         ↪ earest],
46         ↪ p.maximum(hour-1,0)],
47         ↪ delta[np.minimum(hour+1,23)],np.full(144,cur),*[np.full(144,a) for a in
48         ↪ lags],np.full(144,bias),np.sin(clk),np.cos(clk)])
49     return X,base,env
50
51 def pv_archive_closed(data,folder,cfg,upto):
52     folder=Path(folder);folder.mkdir(parents=True,exist_ok=True);xs=[];bases=[];envs=[];raws=[];
53     ↪ audits=[];model=None;fit_audit=None
54     cfg=dict(cfg,asset='q4_pv_daily')
55     for issue in range(upto):

```

```

49     origin=issue*36;path=folder/f'{issue:04d}.npz'
50     if path.exists():
51         a,audit=read_record(path,cfg);X,base,env,row=[a[k] for k in ['x','base','env','raw']]
52         if issue%4==0:payload=joblib.load(path.with_suffix('.joblib'));model=payload['model']
53         ↪ ];fit_audit=payload['fit_audit']
54     else:
55         X,base,env=pv_features_closed(data,issue)
56         if issue%4==0:
57             tx=[];ys=[];indices=[]
58             for j in range(max(0,issue-120),issue):
59                 targets=j*36+np.arange(1,145);mask=(targets<origin)&(envs[j]>0)
60                 if mask.any():tx.append(xs[j][mask]);ys.append((data.pv[targets[mask]]-bases_
61                 ↪ [j][mask])/envs[j][mask]);indices.extend(targets[mask].tolist())
62             model=None
63             if sum(map(len,ys))>=2000:
64                 model=lgb.LGBMRegressor(objective='regression_l1',n_estimators=240,learning_
65                 ↪ rate=.035,num_leaves=31,min_child_samples=50,
66                 ↪ reg_alpha=.1,reg_lambda=1.,random_state=2026,n_jobs=1,deterministic=True)
67                 ↪ ,force_col_wise=True,verbosity=-1)
68                 with threadpool_limits(limits=1):model.fit(np.vstack(tx),np.concatenate(ys))
69             fit_audit=dict(fit_issue=issue,training_cutoff=origin-1,max_training_target=max(
70             ↪ indices,default=-1),rows=len(indices),
71             ↪ training_x_sha=digest(np.vstack(tx)) if tx else
72             ↪ None,training_y_sha=digest(np.concatenate(ys)) if ys else None)
73             raw=base.copy()
74             if model is not None:raw+=env*model.predict(X)
75             raw=np.maximum(raw,0)
76             audit=dict(**fit_audit,issue=issue,origin=origin,feature_cutoff=origin-1,feature_sha_
77             ↪ =digest(X),fallback=model is
78             ↪ None)
79             save_record(path,dict(x=X,base=base,env=env,row=raw),audit,cfg,dict(model=model,fit_
80             ↪ audit=fit_audit))
81             xs.append(X);bases.append(base);envs.append(env);rows.append(row);audits.append(audit)
82             if issue%120==0:print('Q4 PV',issue,flush=True)
83             return dict(base=np.asarray(bases),row=np.asarray(rows),audit=audits)
84
85 def fit_series(kind,X,y,origins,labels):
86     if kind=='history':return dict(kind=kind,labels=labels,origins=origins)
87     q,payload=fit_quantiles(kind,X,y,X[:1]);return dict(kind=kind,**payload)
88
89 def predict_series(model,X,origin):
90     if model['kind']=='history':return
91     ↪ clock_aligned_history(model['labels'],model['origins'],origin)
92     return rearrange(np.column_stack([m.predict(model['scaler'].transform(X))*100 for m in
93     ↪ model['models']]))
94
95 def series_archive(data,folder,cfg,asset,kind,upto,midnight_origins=False):
96     """net/load/actual_price。历史训练起点及成熟标签在每天午夜固定。"""
97     folder=Path(folder);folder.mkdir(parents=True,exist_ok=True);xs={};baselines={};audits=[];mo_
98     ↪ del=None;fit_audit=None
99     prices=asset=='actual_price';q=np.full((upto,432) if prices else (upto,432,5),np.nan)
100     local=dict(cfg,asset=asset,kind=kind,midnight_origins=midnight_origins)
101     for issue in range(28,upto):
102         o=issue*36;path=folder/f'{issue:04d}.npz'
103         if path.exists():
104             a,audit=read_record(path,local);X=a['x'];base=a['base'];pred=a['q']
105             if issue%4==0:payload=joblib.load(path.with_suffix('.joblib'));model=payload['model']
106             ↪ ];fit_audit=payload['fit_audit']
107         else:
108             X,base,t=features(data,asset,o)
109             if issue%4==0:
110                 mature=[j for j in range(max(28,issue-120),issue) if j*36+432<o and (not
111                 ↪ midnight_origins or j%4==0)]
112                 labels=[getattr(data,asset)[j*36+np.arange(1,433)].copy() for j in mature]
113                 if mature:
114                     tx=np.vstack([xs[j] for j in mature]);y=np.concatenate(labels)
115                     if prices:
116                         y=y-np.concatenate([baselines[j] for j in
117                         ↪ mature]);scaler=StandardScaler().fit(tx)
118                     m=HistGradientBoostingRegressor(loss='squared_error',max_iter=60,max_lea_
119                     ↪ f_nodes=7,min_samples_leaf=30,l2_regularization=1.,early_stopping=Fa_
120                     ↪ lse,random_state=20260911)

```

```

104         with ThreadPoolLimits(limits=1):m.fit(scaler.transform(tx),y)
105         model=dict(model=m,scaler=scaler)
106     else:model=fit_series(kind,tx,y,[j*36 for j in mature],labels)
107     else:model=None
108     fit_audit=dict(fit_issue=issue,training_cutoff=o-1,max_training_target=max([j*36
    ↪ +432 for j in
    ↪ mature],default=-1),
109         training_origins=mature,rows=len(mature)*432,training_x_sha=digest(tx) if
    ↪ mature else None,training_y_sha=digest(y) if mature else None)
110 if prices:pred=np.maximum(base if model is None else
    ↪ base+model['model'].predict(model['scaler'].transform(X)),1e-6)
111 elif model is not None:pred=predict_series(model,X,o)
112 else:
113     # 冷启动使用午夜已经完成的自然日；日内只按目标钟点重排。
114     fit_o=(issue//4)*144;days=np.arange(max(1,fit_o//144-30),fit_o//144)
115     hist=getattr(data,asset)[days[:,None]*144+np.arange(144)[None,:]]
116     pred=np.quantile(hist,TAUS,axis=0).T[t%144]
117     if asset=='load':pred=np.maximum(pred,0)
118     audit=dict(**fit_audit,issue=issue,origin=o,feature_cutoff=o-1,feature_sha=digest(X)
    ↪ ,baseline_sha=digest(base),fallback=model is
    ↪ None)
119     save_record(path,dict(x=X,base=base,q=pred),audit,local,dict(model=model,fit_audit=f
    ↪ it_audit))
120     xs[issue]=X;baselines[issue]=base;q[issue]=pred;audits.append(audit)
121     if issue%40==0:print('Q4 series',asset,kind,issue,flush=True)
122 return q,audits
123
124 def q4_net_from_load(data,loadq,pv,alpha):
125     result=loadq.copy()
126     for issue in range(28,len(loadq)):
127         origin=issue*36;past=past_array(data,'pv',origin);targets=origin+np.arange(1,433);j=targ
    ↪ ets%144;cut=origin-1
128         latest=cut-(cut-j)%144;rows=latest[None,:]-144*np.arange(7)[None]
129         hist=np.where(rows>=1,past[np.maximum(rows,1)],np.nan).mean(0)
130         # 此处需要 nanmean, 否则首个缺失 00:00 代理会污染未来对应钟点。
131         hist=np.nanmean(np.where(rows>=1,past[np.maximum(rows,1)],np.nan),axis=0)
132         solar=hist;solar[:144]=(1-alpha)*pv['base'][issue]+alpha*pv['raw'][issue]
133         result[issue]-=solar[:,None]/6
134     return result
135
136
137 def q4_q2_net_archive(data,folder,cfg,kind,upto):
138     """ 直接调用冻结 Q2 的数量模型；当前量可知、午夜拟合、成熟 432 标签均不另改。"""
139     import importlib
140     sys.path.insert(0,str(ROOT/'Q2_第5节独立代码'))
141     q2=importlib.import_module('第5节_Q2分位数预测与滚动调度')
142     folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
143     q=np.full((upto,432,5),np.nan);audits=[]
144     local=dict(cfg,asset='q2_inherited_net',kind=kind)
145     # Q2 首个可拟合原点为 1 月 11 日：1 月 8 日发布的 432 段标签已经完整成熟。
146     for issue in range(40,upto,4):
147         o=issue*36;path=folder/f'{issue:04d}.npz'
148         if path.exists():a,audit=read_record(path,local);pred=a['q']
149         else:
150             pred,audit=q2.q2_origin(data,issue//4,kind,artifact=path.with_suffix('.joblib'))
151             X=q2.net_features(data.prefix(o),o,o+np.arange(1,433))
152             model=joblib.load(path.with_suffix('.joblib'))
153             audit.update(issue=issue,fit_issue=issue,feature_cutoff=o,prediction_target='net_kwh'
    ↪ ,inherits='frozen_Q2')
154             save_record(path,dict(x=X,q=pred),audit,local,model)
155             if issue%40==0:print('Q4-2 inherited Q2',kind,issue,flush=True)
156             q[issue]=pred;audits.append(audit)
157     return q,audits

```

## 4.18 第 9 节 — 独立核验.py

源码/代码/第 9 节 — 独立核验.py

```

1  """ 独立复算物理和账单，并执行有限真实预测回放与因果扰动检查。 """
2  from 第 4 至 7 节 _ 合同优化与储能执行 import *
3  from 第 6 节 _Q3 逐节点负载与光伏预测 import *
4  from 第 7 节 _Q4 每日负载与电价预测 import features,pv_features_closed
5  from 第 5 至 7 节 _ 联合选型与正式回放 import Forecasts,protocol,output_folder
6  import copy,ast
7
8  def verify_replay(result,data,branch):
9      a=result['trace'];t=a[:,0].astype(int);n,g,g0,c,b,h,u,e0,e1,pc,ec,p,z,r,version=a[:,1:].T;ch_j
        ↳ ecks=[]
10     def ck(name,ok,**detail):
11         checks.append(dict(name=name,passed=bool(ok),**detail))
12         if not ok:raise AssertionError((name,detail))
13     ck('实际能量平衡',np.max(abs(g+b+h-n-c-u))<1e-7)
14     ck('库存递推和连续性',np.max(abs(e1-e0-.9*c+b/.9))<1e-7 and np.max(abs(e1[:-1]-e0[1:]))<1e-7)
15     ck('容量总功率与实际互斥',min(e0.min(),e1.min())>=1200-1e-7 and
        ↳ max(e0.max(),e1.max())<=10800+1e-7 and np.max(c+b)<=5000/6+1e-7 and np.max(c*b)<1e-7)
16     cost=p*g0+1.5*p*np.maximum(g-g0,0)-.5*p*np.maximum(g0-g,0)
17     ck('最终原合同费用独立分段复算',np.max(abs(pc-cost))<1e-7 and np.max(abs(ec-5*p*h))<1e-7)
18     ck('真实价格只用于执行账单',np.array_equal(p,data.price[t%144] if branch=='q3' else
        ↳ data.actual_price[t]))
19     ck('费用汇总复算',abs(float((cost+5*p*h).sum())-result['summary']['cost_yuan'])<1e-7)
20     ck('合同只由过去节点生效',np.all((version==1)|(version<t)))
21     lookup={int(row[0]):row for row in np.vstack([result['bridge'],a])} if result['bridge'].size
        ↳ else {int(row[0]):row for row in a}
22     physical=True;aligned=True;init=True;commit=True;emergency=True
23     for plan in result['plans']:
24         o=plan['origin'];vals=plan['values'];gp,cp,bp,up,ep,hp=vals;old=np.r_[plan['initial_pred']
        ↳ icted'],ep[:-1]]
25         physical &= abs(gp+bp+hp-plan['forecast']-cp-up).max()<1e-6 and
        ↳ abs(ep-old-.9*cp+bp/.9).max()<1e-6
26         physical &= ep.min()>=1200-1e-7 and ep.max()<=10800+1e-7 and (cp+bp).max()<=5000/6+1e-7
27         aligned &= len(ep)==432 and np.array_equal(plan['targets'],o+np.arange(1,433)) and
        ↳ abs(plan['terminal']-.9*plan['price'].min())<1e-12
28         row=lookup[o]
29         initial=row[9]
30         init &= abs(initial-plan['initial_predicted'])<1e-8 and
        ↳ abs(row[9]-plan['initial_actual'])<1e-8
31         nextrow=lookup.get(o+1)
32         if nextrow is not None:commit &= abs(nextrow[2]-gp[0])<1e-8
33         emergency &= hp.max()<1e-7
34     ck('所有记录规划守恒容量功率',physical);ck('全年432段和窗口内p乘效率',aligned);ck('交接初值符合各'
        ↳ 分支信息口径',init)
35     ck('LP 直接安装下一段原合同范围',commit);ck('正价自由购电下预测应急为零',emergency)
36     if branch=='q3':ck('继承首段物理预热且不虚构缺失价格费用',np.isfinite(result['warmup'][0,1]) and
        ↳ result['warmup'][0,8]==6000 and np.all(result['warmup'][0,10:12]==0))
37     return dict(passed=True,count=len(checks),checks=checks)
38
39 # 全流程核验由新的继承一致性核验和正式 verify_replay 组成。

```

## 4.19 compact\_plot\_source.py

源码/正文绘图/compact\_plot\_source.py

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5
6  import matplotlib as mpl
7  import matplotlib.pyplot as plt
8  import numpy as np
9  import pandas as pd
10 from matplotlib.colors import LinearSegmentedColormap

```

```

11 from matplotlib.font_manager import FontProperties, fontManager
12 from matplotlib.lines import Line2D
13 from matplotlib.patches import Patch
14
15
16 SOURCE_DIR = Path(__file__).resolve().parent / " 原始附件"
17 OUTPUT_DIR = Path(__file__).resolve().parent
18 SLOTS_PER_DAY = 144
19 RNG = np.random.default_rng(20260913)
20
21 def portable_font(candidates):
22     available={f.name for f in fontManager.ttflist}
23     return FontProperties(family=next((n for n in candidates if n in available), 'DejaVu Sans'))
24 SERIF=portable_font(['Noto Serif CJK SC', 'SimSun', 'Songti SC'])
25 SANS=portable_font(['Noto Sans CJK SC', 'Microsoft YaHei', 'SimHei', 'PingFang SC'])
26
27 INK = "#173F48"
28 TEXT = "#3F5052"
29 MUTED = "#7A8585"
30 RULE = "#DDE4E1"
31 GRID = "#E8ECEA"
32 TEAL = "#2D7778"
33 BLUE = "#365F78"
34 TERRACOTTA = "#C87963"
35 SAND = "#D4B36E"
36 PALE_TEAL = "#BFD9D2"
37 PALE_BLUE = "#CEDDE3"
38 PALE_CLAY = "#E8D2C9"
39
40 MONTH_COLORS = [
41     "#315B73", "#386D7C", "#44848A", "#5C9990",
42     "#78AA94", "#9AB99A", "#B9C09A", "#D0BC8C",
43     "#D8A77B", "#CE8C70", "#B96F65", "#98565D",
44 ]
45 HEAT_CMAP = LinearSegmentedColormap.from_list(
46     "paper_tea",
47     ["#F4F6EE", "#D8E6DA", "#9FC9B9", "#5C9E99", "#2D737D", "#173E55"],
48 )
49
50
51 def read_wide(path: Path, sheet_name: str | int) -> tuple[pd.DatetimeIndex, np.ndarray]:
52     frame = pd.read_excel(path, sheet_name=sheet_name)
53     dates = pd.to_datetime(frame.iloc[:, 0], errors="raise")
54     values = frame.iloc[:, 1:].apply(pd.to_numeric, errors="raise").to_numpy(dtype=float)
55     if values.shape != (365, SLOTS_PER_DAY):
56         raise ValueError(f"{path.name}/{sheet_name} 的数据形状为 {values.shape}, 预期为 (365, 144)")
57     return pd.DatetimeIndex(dates), values
58
59
60 def read_attachment1(path: Path) -> pd.DataFrame:
61     frame = pd.read_excel(path)
62     frame.columns = ["time", "price", "load", "pv"]
63     for col in ("price", "load", "pv"):
64         frame[col] = pd.to_numeric(frame[col], errors="raise")
65     return frame
66
67
68 def set_style() -> None:
69     mpl.rcParams.update(
70         {
71             "font.family": SANS.get_name(),
72             "font.size": 8.5,
73             "axes.unicode_minus": False,
74             "axes.facecolor": "white",
75             "figure.facecolor": "white",
76             "savefig.facecolor": "white",
77             "axes.edgecolor": "#AFBCB9",
78             "axes.labelcolor": TEXT,
79             "xtick.color": MUTED,
80             "ytick.color": MUTED,
81             "text.color": TEXT,
82             "grid.color": GRID,
83             "grid.linewidth": 0.7,

```

```

84         "grid.alpha": 0.85,
85         "legend.frameon": False,
86         "svg.fonttype": "none",
87         "pdf.fonttype": 42,
88     }
89 )
90
91
92 def page_header(fig: plt.Figure, eyebrow: str, number: str, title: str, subtitle: str) -> None:
93     return None
94
95
96 def finish_page(fig: plt.Figure, filename: str, takeaway: str, note: str) -> list[str]:
97     outputs = []
98     path = OUTPUT_DIR / f"{filename}.png"
99     fig.savefig(path, dpi=240, bbox_inches="tight", pad_inches=0.06, metadata={"Creator":
100     ↪ "Matplotlib 3.11.1"})
101     outputs.append(str(path))
102     plt.close(fig)
103     return outputs
104
105 def monthly_slot_mean(dates: pd.DatetimeIndex, values: np.ndarray) -> np.ndarray:
106     return np.vstack([values[dates.month == month].mean(axis=0) for month in range(1, 13)])
107
108
109 def draw_heatmap_page(dates: pd.DatetimeIndex, load: np.ndarray, pv: np.ndarray, price:
110 ↪ np.ndarray) -> list[str]:
111     fig = plt.figure(figsize=(6.3, 7.2))
112     page_header(
113         fig,
114         " 输入数据诊断",
115         "01 / 03",
116         " 看趋势与时间规律",
117         " 月份 × 十分钟时刻: 颜色看日内高峰, 右侧折线看季节变化",
118     )
119     grid = fig.add_gridspec(
120         3,
121         2,
122         left=0.09,
123         right=0.92,
124         bottom=0.065,
125         top=0.945,
126         width_ratios=[5.8, 1.35],
127         hspace=0.43,
128         wspace=0.17,
129     )
130     specs = [
131         (" 小区负载", load, "kW"),
132         (" 光伏实际功率", pv, "kW"),
133         (" 波动电价", price, " 元/kWh"),
134     ]
135     hours = [0, 4, 8, 12, 16, 20, 24]
136     months = np.arange(1, 13)
137     month_labels = [f"{m} 月" for m in months]
138
139     for row, (name, values, unit) in enumerate(specs):
140         matrix = monthly_slot_mean(dates, values)
141         ax = fig.add_subplot(grid[row, 0])
142         side = fig.add_subplot(grid[row, 1], sharey=ax)
143         image = ax.imshow(
144             matrix,
145             aspect="auto",
146             interpolation="nearest",
147             cmap=HEAT_CMAP,
148             origin="upper",
149             extent=[0, 24, 12.5, 0.5],
150         )
151         ax.set_xticks(hours)
152         ax.set_yticks(months)
153         ax.set_yticklabels(month_labels)
154         ax.set_xlabel(" 日内时刻 (小时) ")
155         ax.set_title(f"({chr(97 + row)}) {name}", loc="left", color=INK, fontsize=9, pad=7,
156             ↪ fontproperties=SANS, weight="bold")

```

```

155     ax.tick_params(length=0)
156     for spine in ax.spines.values():
157         spine.set_visible(False)
158     ax.set_xticks(np.arange(0, 25, 1), minor=True)
159     ax.set_yticks(np.arange(0.5, 12.5, 1), minor=True)
160     ax.grid(which="minor", color="white", linewidth=0.45, alpha=0.65)
161     ax.grid(False, which="major")
162
163     monthly_mean = matrix.mean(axis=1)
164     side.plot(monthly_mean, months, color=TERRACOTTA, linewidth=1.8, marker="o",
165             ↪ markersize=3.8)
166     side.fill_betweenx(months, monthly_mean, np.min(monthly_mean) * 0.98, color=PALE_CLAY,
167             ↪ alpha=0.5)
168     side.set_xlabel(" 月均值")
169     side.tick_params(axis="y", left=False, labelleft=False)
170     side.grid(axis="x")
171     side.spines[["top", "right", "left"]].set_visible(False)
172
173     cax = ax.inset_axes([0.68, 1.10, 0.31, 0.04])
174     colorbar = fig.colorbar(image, cax=cax, orientation="horizontal")
175     colorbar.outline.set_visible(False)
176     colorbar.ax.tick_params(labelsize=7, length=2, colors=MUTED)
177     colorbar.set_label(unit, fontsize=7.5, color=MUTED, labelpad=-22, x=-0.08)
178
179     return finish_page(
180         fig,
181         "01_ 月份与日内时间结构",
182         " 负载呈阶梯式日内规律, 光伏集中在白天, 电价峰谷时段清晰。",
183         " 色标按变量分别计算, 不用于跨变量比较",
184     )
185
186 def clean_axis(ax: plt.Axes) -> None:
187     ax.spines[["top", "right"]].set_visible(False)
188     ax.spines[["left", "bottom"]].set_color("#B5BFBD")
189     ax.grid(axis="y")
190     ax.tick_params(length=3, width=0.7)
191
192 def draw_profile_page(load: np.ndarray, pv: np.ndarray, price: np.ndarray, a1: pd.DataFrame) ->
193     ↪ list[str]:
194     fig = plt.figure(figsize=(6.3, 5.0))
195     page_header(
196         fig,
197         " 输入数据诊断",
198         "02 / 03",
199         " 比较日内曲线",
200         " 阴影表示全年同一时刻的经验 5%-95% 区间, 虚线为附件 1",
201     )
202     grid = fig.add_gridspec(2, 2, left=0.09, right=0.93, bottom=0.075, top=0.83, hspace=0.48,
203             ↪ wspace=0.36)
204     hours = np.arange(1, SLOTS_PER_DAY + 1) / 6
205     net = load - pv
206     reference = {
207         "load": a1["load"].to_numpy(),
208         "pv": a1["pv"].to_numpy(),
209         "net": a1["load"].to_numpy() - a1["pv"].to_numpy(),
210         "price": a1["price"].to_numpy(),
211     }
212     panels = [
213         (" 小区负载", load, reference["load"], " 功率 (kW) ", PALE_BLUE),
214         (" 光伏功率", pv, reference["pv"], " 功率 (kW) ", PALE_TEAL),
215         (" 净负荷", net, reference["net"], " 功率 (kW) ", "#D8D3DE"),
216         (" 电价", price, reference["price"], " 元/kWh", PALE_CLAY),
217     ]
218     for index, (name, values, ref, ylabel, band_color) in enumerate(panels):
219         ax = fig.add_subplot(grid[index // 2, index % 2])
220         q05, median, q95 = np.quantile(values, [0.05, 0.5, 0.95], axis=0)
221         ax.fill_between(hours, q05, q95, color=band_color, alpha=0.72, linewidth=0)
222         ax.plot(hours, median, color=INK, linewidth=1.9)
223         ax.plot(hours, ref, color=TERRACOTTA, linewidth=1.6, linestyle=(0, (5, 3)))
224         if name == " 净负荷":
225             ax.axhline(0, color="#8A9593", linewidth=0.8)

```

```

224     ax.set_title(f"({chr(97 + index)}) {name}", loc="left", color=INK, fontsize=9, pad=7,
    ↪ fontproperties=SANS, weight="bold")
225     ax.set_xlabel(" 日内时刻 (小时)")
226     ax.set_ylabel(ylabel)
227     ax.set_xlim(0, 24)
228     ax.set_xticks([0, 4, 8, 12, 16, 20, 24])
229     clean_axis(ax)
230
231     handles = [
232         Patch(facecolor=PALE_TEAL, edgecolor="none", alpha=0.72, label=" 全年经验区间"),
233         Line2D([0], [0], color=INK, linewidth=1.9, label=" 全年中位数"),
234         Line2D([0], [0], color=TERRACOTTA, linewidth=1.6, linestyle=(0, (5, 3)), label=" 附件 1"),
235     ]
236     fig.legend(handles=handles, loc="upper left", bbox_to_anchor=(0.09, 0.995), ncol=3,
    ↪ fontsize=8, handlelength=2.4)
237     return finish_page(
238         fig,
239         "02_ 日内曲线与经验区间",
240         " 附件 1 的四条曲线均未超出全年同一时刻的经验范围。",
241         " 附件 1 无日期, 只能判断典型性, 不能判断季节适配性",
242     )
243
244
245 def daily_std(values: np.ndarray) -> np.ndarray:
246     return np.std(values, axis=1, ddof=0)
247
248
249 def draw_distribution_page(dates: pd.DatetimeIndex, load: np.ndarray, pv: np.ndarray, price:
    ↪ np.ndarray) -> list[str]:
250     fig = plt.figure(figsize=(6.3, 6.2))
251     page_header(
252         fig,
253         " 输入数据诊断",
254         "03 / 03",
255         " 比较逐日波动分布",
256         " 每个点代表一天; 小提琴宽度表示同月日标准差的集中程度",
257     )
258     grid = fig.add_gridspec(3, 1, left=0.09, right=0.93, bottom=0.065, top=0.885, hspace=0.42)
259     specs = [
260         (" 小区负载", daily_std(load), " 日标准差 (kW) "),
261         (" 光伏实际功率", daily_std(pv), " 日标准差 (kW) "),
262         (" 波动电价", daily_std(price), " 日标准差 (元/kWh) "),
263     ]
264     months = dates.month.to_numpy()
265     positions = np.arange(1, 13)
266     for row, (name, values, ylabel) in enumerate(specs):
267         ax = fig.add_subplot(grid[row, 0])
268         groups = [values[months == month] for month in positions]
269         parts = ax.violinplot(groups, positions=positions, widths=0.72, showmeans=False,
    ↪ showmedians=False, showextrema=False)
270         for body, color in zip(parts["bodies"], MONTH_COLORS):
271             body.set_facecolor(color)
272             body.set_edgecolor(color)
273             body.set_alpha(0.30)
274             body.set_linewidth(0.8)
275         medians = np.array([np.median(group) for group in groups])
276         q1, q3 = np.quantile(values, [0.25, 0.75])
277         fence = q3 + 1.5 * (q3 - q1)
278         for month, group, color in zip(positions, groups, MONTH_COLORS):
279             jitter = RNG.normal(0, 0.055, size=group.size)
280             ax.scatter(np.full(group.size, month) + jitter, group, s=10, color=color,
    ↪ alpha=0.46, linewidth=0, zorder=2)
281         ax.plot(positions, medians, color=INK, linewidth=1.25, marker="o", markersize=3.6,
    ↪ zorder=3)
282         ax.axhline(fence, color=TERRACOTTA, linewidth=1.0, linestyle=(0, (2, 3)))
283         ax.text(0.995, 0.94, f"全年上界 {fence:,.2f} · 超出 0 天", transform=ax.transAxes,
    ↪ ha="right", va="top", color=TERRACOTTA, fontsize=8)
284         ax.set_title(f"({chr(97 + row)}) {name}", loc="left", color=INK, fontsize=9, pad=7,
    ↪ fontproperties=SANS, weight="bold")
285         ax.set_ylabel(ylabel)
286         ax.set_xlim(0.45, 12.55)
287         ax.set_xticks(positions)
288         ax.set_xticklabels([f"{m} 月" for m in positions])

```

```

289         clean_axis(ax)
290
291     legend_handles = [
292         Line2D([0], [0], color=INK, marker="o", linewidth=1.25, markersize=4, label=" 月中位数"),
293         Line2D([0], [0], color=TERRACOTTA, linestyle=(0, (2, 3)), linewidth=1.0, label="Q3 +
        ↳ 1.5×IQR"),
294     ]
295     fig.legend(handles=legend_handles, loc="upper left", bbox_to_anchor=(0.09, 0.995), ncol=2,
        ↳ fontsize=8)
296     return finish_page(
297         fig,
298         "03_ 逐日波动的月度分布",
299         " 负载有稳定的周内差异, 光伏波动随季节变化; 三项均无高波动异常日。",
300         " 统计上界用于筛查, 不是设备安全上限",
301     )
302
303
304 def main() -> None:
305     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
306     set_style()
307     a1 = read_attachment1(SOURCE_DIR / " 附件 1.xlsx")
308     dates, load = read_wide(SOURCE_DIR / " 附件 2.xlsx", " 小区负载")
309     pv_dates, pv = read_wide(SOURCE_DIR / " 附件 2.xlsx", " 光伏发电实际功率")
310     price_dates, price = read_wide(SOURCE_DIR / " 附件 4.xlsx", 0)
311     if not dates.equals(pv_dates) or not dates.equals(price_dates):
312         raise ValueError(" 负载、光伏和电价日期不一致")
313
314     outputs = []
315     outputs.extend(draw_heatmap_page(dates, load, pv, price))
316     outputs.extend(draw_profile_page(load, pv, price, a1))
317     outputs.extend(draw_distribution_page(dates, load, pv, price))
318
319     manifest = {
320         "reference_style": "用户提供的国赛论文选图指南: 深青标题、低饱和蓝绿/陶土色、白底、
        ↳ 细分隔线和大留白",
321         "source_files": [f" 原始附件/附件 {i}.xlsx" for i in (1, 2, 4)],
322         "transformations": [
323             " 热力图: 按月份对每个十分钟位置取算术平均",
324             " 日内曲线: 同一十分钟位置计算中位数和 5%/95% 经验分位数",
325             " 月度分布: 每个日期行的 144 个值计算总体标准差; 按月份绘制小提琴和原始日点",
326         ],
327         "missing_data": " 附件 1、附件 2 数值区和附件 4 均无缺失; 未进行插补",
328         "layout_change": "已删除全部页眉文字、主标题及其说明、全部页脚文字, 并缩小上下页边距; 图表数据、
        ↳ 坐标、图例和配色不变",
329         "random_seed": 20260913,
330         "palette": {
331             "ink": INK,
332             "teal": TEAL,
333             "terracotta": TERRACOTTA,
334             "month_colors": MONTH_COLORS,
335         },
336         "fonts": [SERIF.get_name(), SANS.get_name()],
337         "outputs": outputs,
338     }
339     with (OUTPUT_DIR / " 重绘说明.json").open("w", encoding="utf-8") as handle:
340         json.dump(manifest, handle, ensure_ascii=False, indent=2)
341     print(json.dumps(manifest, ensure_ascii=False, indent=2))
342
343
344 if __name__ == "__main__":
345     main()

```

## 4.20 selection\_figures.py

源码/正文绘图/selection\_figures.py

```

1  from pathlib import Path
2  import pandas as pd, numpy as np, json, hashlib
3  import matplotlib; matplotlib.use('Agg')
4  import matplotlib.pyplot as plt
5  from matplotlib.font_manager import fontManager, FontProperties
6  base=Path(__file__).resolve().parent; dest=base/'图件'; dest.mkdir(exist_ok=True)
7  root=base/'结果数据'
8  f2=next(root.rglob('*Q2_ 一月 45 组候选评分.csv')); f3=next(root.rglob('*q3_ 一月全部策略比较.csv'))
9  q2=pd.read_csv(f2); q3=pd.read_csv(f3, keep_default_na=False)
10 best2=json.loads(next(root.rglob('*Q2_ 一月胜出冻结参数.json')).read_text(encoding='utf-8'))
11 best3=json.loads(next(root.rglob('*q3_ 选定参数.json')).read_text(encoding='utf-8'))
12 assert len(q2)==45 and len(q3)==1800
13 for d, b in [(q2, best2), (q3, best3)]: assert abs(d.selection_score.min()-b['selection_score'])<1e-7
14 assert not q2.duplicated(['kind', 'tau', 'beta']).any()
15 assert not q3.duplicated(['kind', 'tau', 'beta', 'alpha', 'nodes']).any()
16 font=next((f for f in ['Noto Sans CJK SC', 'Microsoft YaHei', 'SimHei', 'PingFang SC'] if
    ↪ any(x.name==f for x in fontManager.ttflist)), 'DejaVu Sans')
17 plt.rcParams.update({'font.family':font, 'font.size':8.3, 'axes.titlesize':9, 'axes.labelsize':8.3,
    ↪ 'axes.spines.top':False, 'axes.spines.right':False, 'axes.edgecolor': '#ADBAB9', 'axes.labelcolor':
    ↪ 'r': '#294B52', 'text.color': '#173F48', 'xtick.color': '#526769', 'ytick.color': '#526769', 'axes.un
    ↪ icode_minus': False, 'legend.frameon': False, 'savefig.facecolor': 'white'})
18 kinds=['history', 'linear', 'tree']; names=['历史同槽法', '线性分位数回归', '浅层分位数树']; colors=['#7DA
    ↪ 7A0', '#426B7B', '#C38D78']
19 fig, axs=plt.subplots(1,3,figsize=(6.5,2.6),sharey=True,layout='constrained')
20 lo=q2.selection_score.min()/1e4; hi=q2.selection_score.max()/1e4
21 for ax, k, name in zip(axs, kinds, names):
22     for beta, c, mark in zip([0, .5, 1], colors, ['o', 's', '^']):
23         d=q2[(q2.kind==k)&(q2.beta==beta)].sort_values('tau')
24         ax.plot(d.tau, d.selection_score/1e4, color=c, marker=mark, markersize=3, lw=.95, label=fr'$\beta$={b
    ↪ eta:g}$', alpha=.95)
25     ax.set_title(name); ax.set_xlabel(r'控制分位数 $\tau_N$'); ax.set_xticks([.5, .65, .8, .9, .95], ['.50'
    ↪ ', '.65', '.80', '.90', '.95']); ax.tick_params(axis='x', labelsz=7)
26     ax.grid(axis='y', color='#E6ECEA', lw=.6); ax.set_ylim(lo-1, hi+1)
27     axs[0].set_ylabel('一月选择评分 / 万元 (越低越好)')
28     axs[1].legend(loc='upper center', fontsize=7.5)
29     axs[2].scatter([.8], [lo], s=78, c='#AD7563', marker='*', zorder=6, edgecolors='white', linewidths=.5)
30     axs[2].annotate(f'最低 {lo:.4f}\n'+r'$\tau_N=0.80, \beta=0$', xy=(.8, lo), xytext=(.535, lo+5), arrow
    ↪ props={'arrowstyle':'-', 'color': '#AD7563', 'lw':.8, 'fontSize':7.5, 'color': '#966C5D'})
31     fig.savefig(dest/'selection_q2.png', dpi=300, bbox_inches='tight', pad_inches=.06); plt.close(fig)
32
33     fig, axs=plt.subplots(1,2,figsize=(6.5,2.8),gridspec_kw={'width_ratios':[1.15,1]},layout='constra
    ↪ ined')
34     d=q3.sort_values('selection_score', kind='stable').reset_index(drop=True); d['rank']=np.arange(1, l
    ↪ en(d)+1)
35     for k, c, name in zip(kinds, colors, names):
36         dd=d[d.kind==k]; axs[0].scatter(dd['rank'], dd.selection_score/1e4, s=4, c=c, label=name, alpha=.65, l
    ↪ inewidths=0, rasterized=True)
37     j=best3['selection_score']/1e4
38     axs[0].scatter([1], [j], s=78, c='#AD7563', marker='*', edgecolors='white', linewidths=.4, zorder=5)
39     axs[0].annotate(f'全局最低 {j:.4f}', xy=(1, j), xytext=(320, j+2), arrowprops={'arrowstyle':'-', 'color'
    ↪ ': '#AD7563', 'lw':.8, 'fontSize':7.4, 'color': '#966C5D'})
40     axs[0].set_title('(a) 1800组候选的完整评分排序'); axs[0].set_xlabel('按一月评分由低到高排列'); axs[0].
    ↪ set_ylabel('一月选择评分 / 万元')
41     axs[0].set_xticks([1, 600, 1200, 1800]); axs[0].legend(loc='upper
    ↪ left', fontsize=7); axs[0].grid(axis='y', color='#E6ECEA', lw=.6)
42     nodes=['6', '12', '18', '6, 12', '6, 18', '12, 18', '6, 12, 18']; nlabels=['无日内调整', '6', '12', '18', '6、
    ↪ 12', '6、18', '12、18', '6、12、18']
43     mins=q3.groupby('nodes').selection_score.min().reindex(nodes)/1e4
44     assert q3.groupby('nodes').size().eq(225).all()
45     bars=axs[1].barh(np.arange(8), mins-j, color=['#A8C6BF']*7+['#D2A38F'], height=.60, alpha=.78)
46     axs[1].scatter([0], [7], marker='*', s=78, c='#AD7563', clip_on=False, zorder=5)
47     axs[1].set_yticks(np.arange(8), nlabels); axs[1].invert_yaxis(); axs[1].set_xlabel('该组最低评分 -
    ↪ 全局最低 / 万元')
48     axs[1].set_title('(b) 每种节点集合取225组中的最低分'); axs[1].grid(axis='x', color='#E6ECEA', lw=.6); a
    ↪ x[1].set_axisbelow(True)
49     axs[1].text(.03, 7, '选定', va='center', fontsize=7.5, color='#966C5D')

```

```

50 fig.savefig(dest/'selection_q3.png',dpi=300,bbox_inches='tight',pad_inches=.06);plt.close(fig)
51 evidence={'q2':{'source':f2.relative_to(base).as_posix(),'sha256':hashlib.sha256(f2.read_bytes())
    ↪ ).hexdigest(),'candidate_count':45,'minimum_score_yuan':best2['selection_score'],'winner':{'k
    ↪ :best2[k] for k in
    ↪ ['kind','tau','beta']}},'q3':{'source':f3.relative_to(base).as_posix(),'sha256':hashlib.sha2
    ↪ 56(f3.read_bytes()).hexdigest(),'candidate_count':1800,'minimum_score_yuan':best3['selection
    ↪ _score'],'winner':{'k:best3[k] for k in
    ↪ ['kind','tau','beta','alpha','nodes']},'per_node_minimum_score_yuan':{'k:float(v*1e4) for k,v
    ↪ in mins.items()}}}
52 (base/'参数筛选图数据核验.json').write_text(json.dumps(evidence,ensure_ascii=False,indent=2),encod
    ↪ ing='utf-8')

```

## 4.21 第 6 至 7 节 — 固定模型的八节点价值对照.py

源码/辅助运行/第 6 至 7 节 — 固定模型的八节点价值对照.py

```

1  """ 第 6 至 7 节：固定一月胜出预测器和控制参数，比较正式期八种节点安排。
2
3  仅做解释性对照；不训练其他模型，不改写主方案选定参数或官方主结果。
4  """
5  from pathlib import Path
6  import sys,json,hashlib
7  ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT/'代码'))
8  from 第5至7节_联合选型与正式回放 import
    ↪ protocol,UnifiedData,np,NODE_SETS,replay,save_replay,dump,write_csv
9  from 第 9 节 _ 独立核验 import verify_replay
10
11 def compare(branch,run_root,output_root=None):
12     assert branch in ['q3','q4_3']
13     formal=Path(run_root)/branch/'正式期';done=json.loads((formal/'正式计算完成.json').read_text(e
    ↪ ↪ noding='utf-8'));selected=done['selected']
14     assert done['end_day']==365 and
    ↪ done['configuration']==json.loads(json.dumps(protocol(branch)))
15     primary_nodes=tuple(selected['nodes']);main=done['rows'][0]
16     forecasts_path=formal/'正式预测.npz';archive=np.load(forecasts_path);q=archive['q'];p=None if
    ↪ branch=='q3' else archive['price']
17     out=(Path(output_root) if output_root is not None else
    ↪ ROOT/'节点价值对照')/branch;out.mkdir(parents=True,exist_ok=True)
18     lock=dict(branch=branch,selected=selected,forecast_sha256=hashlib.sha256(forecasts_path.read
    ↪ _bytes()).hexdigest(),
19     ↪ role='Descriptive node-arrangement comparison; fixed January model family, tau, beta,
    ↪ ↪ alpha; primary node set remains January choice')
20     dump(out/'固定模型与对照协议.json',lock);data=UnifiedData();rows=[]
21     for nodes in NODE_SETS:
22         folder=out/('节点_'+(''.join(map(str,nodes)) or 'none'))
23         if tuple(nodes)==primary_nodes:
24             row=dict(main);row['source']='主方案正式回放';rows.append(row);continue
25             result=replay(data,q,p,branch,selected['tau_index'],selected['beta'],nodes,end=365*144,k
    ↪ ↪ eep_plans=True)
26             result['summary'].update(kind=selected['kind'],alpha=selected['alpha'],is_january_select
    ↪ ↪ ed_nodes=False)
27             checks=verify_replay(result,data,branch);save_replay(folder,result);dump(folder/'物理费用
    ↪ ↪ 核验.json',checks)
28             row=dict(result['summary']);row['source']='固定模型节点对照';rows.append(row)
29             print('NODE COMPARISON',branch,nodes,row['cost_yuan'],flush=True)
30             baseline=next(r['cost_yuan'] for r in rows if not r['nodes'])
31             for r in rows:r['saving_vs_midnight_yuan']=baseline-r['cost_yuan']
32             write_csv(out/'八种节点费用对照.csv',({**r,'nodes':''.join(map(str,r['nodes']))}) for r in
    ↪ ↪ rows))
33             dump(out/'对照计算完成.json',dict(passed=True,protocol=lock,rows=rows,primary_nodes=list(prim
    ↪ ↪ ary_nodes),
34             ↪ descriptive_lowest_cost_nodes=min(rows,key=lambda
    ↪ ↪ ↪ r:r['cost_yuan']['nodes'],selection_unchanged=True))
35             print('NODE COMPARISONS FINISHED',branch,flush=True)
36
37 if __name__=='__main__':
38     import argparse
39     p=argparse.ArgumentParser();p.add_argument('branch',choices=['q3','q4_3']);p.add_argument('-
    ↪ ↪ -run-root',type=Path,required=True)

```

```

40 | p.add_argument('--output-root',type=Path,help='重跑时指定独立目录，保留当前交付对照')
41 | a=p.parse_args();compare(a.branch,a.run_root,a.output_root)

```

## 4.22 第 10 节 — 官方结果填表.py

源码/输出工具/第 10 节 — 官方结果填表.py

```

1 | """ 便携填表与逐格读回；使用本机 Python/openpyxl，无专用平台依赖。"""
2 | from pathlib import Path
3 | from copy import copy
4 | import argparse,json,datetime,hashlib,math
5 | from openpyxl import load_workbook
6 | from openpyxl.styles import Font
7 |
8 | def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
9 | def put(sheet,start_row,start_col,data):
10 |     for i,row in enumerate(data,start_row):
11 |         for j,value in enumerate(row,start_col):sheet.cell(i,j,value)
12 | def verify(payload,file):
13 |     w=load_workbook(file,read_only=True,data_only=False);count=0
14 |     def cells(sheet,r,c,values):
15 |         nonlocal count
16 |         rows=w[sheet].iter_rows(min_row=r,max_row=r+len(values)-1,min_col=c,max_col=c+len(values)
17 |         ↪ [0])-1,values_only=True)
18 |         for actual,expected in zip(rows,values):
19 |             for x,y in zip(actual,expected):
20 |                 if isinstance(y,(int,float)):
21 |                     if not isinstance(x,(int,float)) or not math.isfinite(x) or
22 |                     ↪ abs(x-y)>1e-6:raise ValueError(f'单元格数值不一致: {sheet}')
23 |                 elif isinstance(x,(datetime.datetime,datetime.date)):
24 |                     if x.strftime('%Y-%m-%d')!=y:raise ValueError('日期不一致')
25 |                 elif x!=y:raise ValueError(f'标签不一致: {sheet}')
26 |                 count+=1
27 |     try:
28 |         cells('计划购电量',2,2,payload['original'])
29 |         if payload['branch']=='q1':
30 |             cells('充放电量',2,2,payload['storage']);cells('充放电量',2,5,[[x] for x in
31 |             ↪ payload['inventory']])
32 |         else:
33 |             if payload['branch'] in ['q3','q4_3']:cells('调整购电量',2,2,payload['effective'])
34 |             for sheet,key in [('充放电量','storage'),('紧急购电量','emergency')]:
35 |                 if w[sheet].max_row!=len(payload[key])+1:raise ValueError('输出行数不一致')
36 |                 cells(sheet,2,1,payload[key])
37 |             for sheet in ['计划购电量']+(['调整购电量'] if payload['branch'] in ['q3','q4_3'] else
38 |             ↪ []):
39 |                 cells(sheet,2,1,[[d] for d in payload['dates']])
40 |             return dict(passed=True,numeric_and_label_cells_checked=count,file=file.name,sha256=sha(
41 |             ↪ file))
42 |     finally:w.close()
43 | def export(payload_path,output,template):
44 |     p=json.loads(payload_path.read_text(encoding='utf-8'))
45 |     if not p['evidence']['checks_passed']:raise ValueError('输入结果未经核验')
46 |     output.mkdir(parents=True,exist_ok=True);file=output/p['filename']
47 |     if file.exists():raise FileExistsError('输出文件已存在，拒绝覆盖旧结果: '+file.name)
48 |     w=load_workbook(template);headers={s.title:[c.value for c in s[1]] for s in w}
49 |     if p['branch']=='q1':
50 |         put(w['计划购电量'],2,2,p['original']);put(w['充放电量'],2,2,p['storage']);put(w['充放电量']
51 |         ↪ ],2,5,[[v] for v in
52 |         ↪ p['inventory']])
53 |     else:
54 |         for name,key in [('计划购电量','original')+(['调整购电量','effective'] if p['branch']
55 |         ↪ in ['q3','q4_3'] else []):
56 |             s=w[name];put(s,2,2,p[key])
57 |             for i,date in enumerate(p['dates'],2):s.cell(i,1,datetime.datetime.fromisoformat(dat
58 |             ↪ e)).number_format='yyyy-mm-dd'
59 |         for name,key in [('充放电量','storage'),('紧急购电量','emergency')]:
60 |             s=w[name];s.delete_rows(2,s.max_row)
61 |             values=[[datetime.datetime.fromisoformat(v) if j==0 and v else v for j,v in
62 |             ↪ enumerate(row)] for row in p[key]]

```

```

53         put(s,2,1,values)
54         for row in s.iter_rows(min_row=2):
55             for cell in row:
56                 cell.font=Font(name='宋体',size=11)
57                 if cell.column==1:cell.number_format='yyyy-mm-dd'
58                 elif isinstance(cell.value,(int,float)):cell.number_format='0.000000'
59     for s in w:
60         if [c.value for c in s[1]]!=headers[s.title]:raise ValueError('表头改变')
61     w.properties.creator='';w.properties.lastModifiedBy='';w.properties.description=None;w.prope
    ↳ rties.keywords=None
62     w.save(file);w.close()
63     report=verify(p,file)
64     (output/(p['branch']+'_导出逐格核验.json')).write_text(json.dumps(report,ensure_ascii=False,i
    ↳ ndent=2),encoding='utf-8')
65     (output/(p['branch']+'_导出完成凭据.json')).write_text(json.dumps(dict(completed=True,filenam
    ↳ e=file.name,payload_sha256=sha(payload_path),workbook_sha256=sha(file),completed_at=date
    ↳ time.datetime.now().astimezone().isoformat()),ensure_ascii=False,indent=2),encoding='utf
    ↳ -8')
66     return report
67 if __name__=='__main__':
68     a=argparse.ArgumentParser(description=__doc__);a.add_argument('payload',type=Path);a.add_arg
    ↳ ument('output',type=Path);a.add_argument('--template',type=Path);v=a.parse_args()
69     filename=json.loads(v.payload.read_text(encoding='utf-8'))['filename']
70     template=v.template or Path(__file__).resolve().parent/'官方模板'/filename
71     print(json.dumps(export(v.payload,v.output,template),ensure_ascii=False))

```

## 4.23 第 10 节 \_ 提取官方填表数据.py

源码/输出工具/第 10 节 \_ 提取官方填表数据.py

```

1  """ 第 10 节：从已完成回放提取官方填表数据，不训练、选型或改写策略。
2
3  购电两张表使用合同日 (00:10 至次日 00:10)，充放/应急使用自然日。
4  计划表为午夜原合同；调整表为逐段最终有效合同，费用已含加购/退订。
5  年末表格末段从最后一次已安装合同读取，不新增未来决策。
6  """
7  from pathlib import Path
8  import sys,json,datetime,hashlib
9  ROOT=Path(__file__).resolve().parents[1]
10 sys.path.insert(0,str(ROOT/'代码'))
11 from 第 4 至 7 节 _ 合同优化与储能执行 import UnifiedData,fee,np,dump
12
13 def prepare(branch,run_root,output):
14     formal=Path(run_root)/branch/'正式期'
15     done=json.loads((formal/'正式计算完成.json').read_text(encoding='utf-8'))
16     assert done['end_day']==365
17     selected=done['selected'];nodes=selected['nodes']
18     folder=formal/('正式节点_'+('-'.join(map(str,nodes)) or 'none'))
19     assert json.loads((folder/'物理费用核验.json').read_text(encoding='utf-8'))['passed']
20     source=np.load(folder/'执行轨迹.npz');a=source['trace'];plans=np.load(folder/'规划档案.npz')
21     t=a[:,0].astype(int);assert np.array_equal(t,np.arange(4464,52560))
22     g0=np.full(52561,np.nan);g=g0.copy()
23     for o,v in zip(plans['origins'],plans['values']):
24         n=144-o%144
25         g[o+1:o+1+n]=v[0:n]
26         if o%144==0:g[o+1:o+145]=v[0:144]
27     slots=np.arange(4465,52561);assert np.isfinite(g[slots]).all() and
    ↳ np.isfinite(g0[slots]).all()
28     assert np.max(abs(g[t[1:]-a[1:2]]))<1e-8
29     assert np.max(abs(g0[t[1:]-a[1:3]]))<1e-8
30     data=UnifiedData();p=data.price[slots%144] if branch=='q3' else data.actual_price[slots]
31     original=g0[slots].reshape(334,144);effective=g[slots].reshape(334,144);prices=p.reshape(334
    ↳ ,144)
32     base_cost=(original*prices).sum(axis=1)
33     final_cost=(original*prices+1.5*prices*np.maximum(effective-original,0)-.5*prices*np.maximum
    ↳ (original-effective,0)).sum(axis=1)
34     first=datetime.date(2025,2,1)
35     dates=[(first+datetime.timedelta(days=i)).isoformat() for i in range(334)]
36     def table(matrix,cost):return [[*row.tolist(),float(row.sum()),float(cost[i])] for i,row in
    ↳ enumerate(matrix)]

```

```

37     storage=[];emergency=[]
38     for day in range(334):
39         dayrows=a[day*144:(day+1)*144];date=dates[day]
40         for block in range(6):
41             blockrows=dayrows[24*block:24*(block+1)]
42             storage.append([date if block==0 else None,f'{4*block}:00-{4*(block+1)}:00',float(bl
43                 ↪ ockrows[:,4].sum()),float(blockrows[:,5].sum()),
44                 '0:00' if block==0 else '24:00' if block==1 else None,
45                 float(dayrows[0,8]) if block==0 else float(dayrows[-1,9]) if block==1 else None])
46             i=0;count=0
47             def clock(k):return f'{k//6:02d}:{(k%6)*10:02d}'
48             while i<144:
49                 if dayrows[i,6]<=1e-9:i+=1;continue
50                 j=i+1
51                 while j<144 and dayrows[j,6]>1e-9:j+=1
52                 emergency.append([date,f'{clock(i)}-{clock(j)}',float(dayrows[i:j,6].sum()))];count+
53                 ↪ =1;i=j
54             if not count:emergency.append([date,'无',0.])
55             assert abs(sum(r[2] for r in emergency)-float(a[:,6].sum()))<1e-6
56             assert abs(sum(r[2] for r in storage)-float(a[:,4].sum()))<1e-6
57             assert abs(sum(r[3] for r in storage)-float(a[:,5].sum()))<1e-6
58             last_fee=float(fee(g[-1],g0[-1],p[-1]))
59             discrepancy=float(final_cost.sum()-a[:,10].sum())
60             assert abs(discrepancy-(last_fee-a[0,10]))<1e-6
61             evidence=dict(branch=branch,selected={k:selected[k] for k in
62                 ↪ ['kind','tau','beta','alpha','nodes']},
63                 natural_slots=[4464,52559],contract_slots=[4465,52560],
64                 natural_total_yuan=float(a[:,10:12].sum()),natural_contract_yuan=float(a[:,10].sum()),na
65                 ↪ tural_emergency_yuan=float(a[:,11].sum()),
66                 contract_table_original_yuan=float(base_cost.sum()),contract_table_final_yuan=float(fina
67                 ↪ l_cost.sum()),
68                 contract_minus_natural_yuan=discrepancy,excluded_first_contract_fee_yuan=float(a[0,10]),
69                 ↪ included_last_committed_contract_fee_yuan=last_fee,
70                 final_table_slot='2026-01-01 00:00-00:10; committed in 2025-12-31, no new decision',
71                 storage_rows=len(storage),emergency_rows=len(emergency),checks_passed=True,
72                 sources={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in
73                 ↪ [folder/'执行轨迹.npz',folder/'规划档案.npz',formal/'正式计算完成.json']})
74             payload=dict(branch=branch,filename={'q3':'result3.xlsx','q4_2':'result4-2.xlsx','q4_3':'res
75                 ↪ ult4-3.xlsx'}[branch],
76                 dates=dates,original=table(original,base_cost),effective=table(effective,final_cost),sto
77                 ↪ rage=storage,emergency=emergency,evidence=evidence)
78             output=Path(output);output.mkdir(parents=True,exist_ok=True)
79             dump(output/(branch+'_填表数据.json'),payload);dump(output/(branch+'_时间边界复核.json'),evide
80                 ↪ nce)
81             print(branch,'填表数据完成',len(storage),len(emergency),flush=True)
82
83 if __name__=='__main__':
84     import argparse
85     p=argparse.ArgumentParser();p.add_argument('branch',choices=['q3','q4_2','q4_3']);p.add_argu
86     ↪ ment('--run-root',type=Path,required=True);p.add_argument('--output',type=Path,required=
87     ↪ True);x=p.parse_args()
88     prepare(x.branch,x.run_root,x.output)

```

## 4.24 生成结果表与论文数据.py

生成结果表与论文数据.py

```

1  """ 只从本次计算输出填报五份官方表，并导出论文典型日/附录数据。"""
2  from pathlib import Path
3  import argparse, csv, datetime, importlib.util, json, subprocess, sys
4  import numpy as np
5  def read(p):return json.loads(p.read_text(encoding='utf-8'))
6  def dump(p,x):p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(x,ensure_ascii=
7  ↪ False,indent=2),encoding='utf-8')
8  def table(p):
9      with p.open(encoding='utf-8-sig',newline='') as f:return list(csv.DictReader(f))
10 def groups(a):
    storage=[];emergency=[]

```

```

11     dates=[(datetime.date(2025,2,1)+datetime.timedelta(days=i)).isoformat() for i in range(334)]
12     def clock(k):return f'{k//6:02d}:{k%6*10:02d}'
13     for day,date in enumerate(dates):
14         rows=a[day*144:(day+1)*144]
15         for b in range(6):
16             block=rows[b*24:(b+1)*24]
17             storage.append([date if b==0 else None,f'{4*b}:00-{4*(b+1)}:00',float(block[:,4].sum()
18                 ↪ ()),float(block[:,5].sum()),'0:00' if b==0 else '24:00' if b==1 else
19                 ↪ None,float(rows[0,8]) if b==0 else float(rows[-1,9]) if b==1 else None])
20         i=0;count=0
21         while i<144:
22             if rows[i,6]<=1e-9:i+=1;continue
23             j=i+1
24             while j<144 and rows[j,6]>1e-9:j+=1
25             emergency.append([date,f'{clock(i)}-{clock(j)}',float(rows[i:j,6].sum())]);i=j;count
26             ↪ +=1
27         if count==0:emergency.append([date,'无',0.])
28     return dates,storage,emergency
29 def main(work):
30     run=work/'本次复算';out=work/'论文结果';payloads=out/'填表中间数据';payloads.mkdir(parents=True)
31     ↪ ,exist_ok=True)
32     values={};traces={}
33     q1=read(run/'q1/完整结果.json');rows=q1['schedule'];contract=table(run/'q1/日前购电合同.csv')
34     assert read(run/'q1/第9节_Q1结果核验.json')['passed']
35     values['q1']=dict(branch='q1',filename='result1.xlsx',original=[[float(r['buy_kwh'])] for r
36         ↪ in contract],storage=[[sum(float(r['charge_kwh']) for r in
37         ↪ rows[b*24:(b+1)*24]),sum(float(r['discharge_kwh']) for r in rows[b*24:(b+1)*24])] for b
38         ↪ in range(6)],inventory=[float(rows[0]['e_start_kwh']),float(rows[-1]['e_end_kwh'])],evid
39         ↪ ence=dict(checks_passed=True,natural_total_yuan=q1['objective_yuan']))
40     a=np.load(run/'q2/trace.npz')['trace'];plans=np.load(run/'q2/plans.npz')
41     assert read(run/'q2/DONE.json')['complete'] and a.shape==(48096,12)
42     assert np.max(abs(a[:,2]+a[:,5]+a[:,6]-a[:,1]-a[:,4]-a[:,7]))<1e-7
43     assert np.max(abs(a[:,9]-a[:,8]-.9*a[:,4]+a[:,5]/.9))<1e-7
44     slots=plans['targets'][:,144].reshape(-1).astype(int);assert
45     ↪ np.array_equal(slots,np.arange(4465,52561))
46     g=plans['values'][:,0,144].reshape(-1)
47     execution=table(run/'q2/q2_execution.csv');price_cycle=np.array([float(r['ordinary_price'])
48         ↪ for r in execution[:144]])
49     prices=price_cycle[slots%144];assert np.max(abs(g[:1]-a[1:,2]))<1e-7
50     matrix=g.reshape(334,144);cost=(g*prices).reshape(334,144).sum(axis=1);dates,st,em=groups(a)
51     boundary=float(cost.sum()-a[:,10].sum());assert abs(boundary-(g[-1]*prices[-1]-a[0,10]))<1e-6
52     values['q2']=dict(branch='q2',filename='result2.xlsx',dates=dates,original=np.column_stack([
53         ↪ matrix,matrix.sum(axis=1),cost]).tolist(),storage=st,emergency=em,evidence=dict(checks_p
54         ↪ asse=True,natural_total_yuan=float(a[:,10:12].sum()),contract_minus_natural_yuan=bounda
55         ↪ ry,excluded_first_contract_fee_yuan=float(a[0,10]),included_last_committed_contract_fee_
56         ↪ yuan=float(g[-1]*prices[-1])))
57     traces['q2']=a
58     for branch in ['q3','q4_2','q4_3']:
59         subprocess.run([sys.executable,'-B','-X','utf8',str(work/'输出工具/第10节_提取官方填表数据.
60             ↪ py'),branch,'--run-root',str(run),'--output',str(payloads)],cwd=work,check=True)
61         values[branch]=read(payloads/f'{branch}_填表数据.json')
62         done=read(run/branch/'正式期/正式计算完成.json');nodes=done['selected']['nodes'];folder=ru
63             ↪ n/branch/'正式期'/'正式节点_'+('-').join(map(str,nodes)) or
64             ↪ 'none')
65         traces[branch]=np.load(folder/'执行轨迹.npz')['trace']
66     src=work/'输出工具/第10节_官方结果填表.py';spec=importlib.util.spec_from_file_location('excel_
67         ↪ export',src);exporter=importlib.util.module_from_spec(spec);spec.loader.exec_module(expo
68         ↪ rter)
69     reports=[]
70     for branch,v in values.items():
71         p=payloads/f'{branch}.json';dump(p,v)
72         reports.append(exporter.export(p,out/'官方结果表',work/'输出工具/官方模板'/v['filename']))
73     assert sorted(p.name for p in (out/'官方结果表').glob('*.xlsx'))==['result1.xlsx','result2.xl
74         ↪ sx','result3.xlsx','result4-2.xlsx','result4-3.xlsx']
75     paper={};selected_dates=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
76     for branch,a in traces.items():
77         paper[branch]=[]
78         for date in selected_dates:
79             day=(datetime.date.fromisoformat(date)-datetime.date(2025,2,1)).days;x=a[day*144:(da
80                 ↪ y+1)*144]

```

```

60     paper[branch].append(dict(date=date,purchases=[float(x[h*6,2]) for h in
    ↪ [10,12,14,16,18,20]],original_purchases=[float(x[h*6,3]) for h in
    ↪ [10,12,14,16,18,20]],daily_purchase=float(x[:,2].sum()),daily_original_purchase=
    ↪ float(x[:,3].sum()),daily_emergency=float(x[:,6].sum()),daily_total_purchase=flo
    ↪ at(x[:,2].sum()+x[:,6].sum()),daily_cost=float(x[:,10:12].sum()),contract_cost=f
    ↪ loat(x[:,10].sum()),emergency_cost=float(x[:,11].sum()),storage=[float(x[b*24:(
    ↪ b+1)*24,4].sum()),float(x[b*24:(b+1)*24,5].sum()) for b in
    ↪ range(6)],inventory=[float(x[0,8]),float(x[-1,9])],emergency=[[r[1],r[2]] for r
    ↪ in values[branch]['emergency'] if r[0]==date]))
61 paper['q1']=dict(purchases=[float(rows[h*6]['buy_kwh']) for h in
    ↪ [10,12,14,16,18,20]],daily_purchase=sum(float(r['buy_kwh']) for r in
    ↪ rows),daily_cost=sum(float(r['cost_yuan']) for r in
    ↪ rows),storage=values['q1']['storage'],inventory=values['q1']['inventory'])
62 dump(out/'paper_data.json',paper);dump(out/'五份官方结果表核验.json',dict(passed=True,reports=
    ↪ reports))
63 print('五份官方结果表与论文典型日、附录数据已生成',flush=True)
64 if __name__=='__main__':
65     p=argparse.ArgumentParser(description=__doc__);p.add_argument('--work',type=Path,required=Tr
    ↪ ue);a=p.parse_args();main(a.work.resolve())

```

## 4.25 输入转换.py

输入转换.py

```

1  """ 从另行提供的官方 Excel 生成相对路径输入；不捆绑或手抄原始观测。"""
2  from pathlib import Path
3  import json, hashlib, datetime, shutil, math
4
5  def sha(path):return hashlib.sha256(path.read_bytes()).hexdigest()
6  def dump(path,obj):
7      path.parent.mkdir(parents=True,exist_ok=True)
8      path.write_text(json.dumps(obj,ensure_ascii=False,indent=2,allow_nan=False),encoding='utf-8')
9  def number(v):
10     if isinstance(v,bool) or not isinstance(v,(int,float)) or not math.isfinite(v):raise
    ↪ ValueError('Excel存在空值或非有限数值')
11     return float(v)
12 def minute(v):
13     if hasattr(v,'hour'):return v.hour*60+v.minute
14     s=str(v).strip()
15     if s in ['0:00+1','00:00+1','24:00']:return 1440
16     h,m=map(int,s.split(':')[2]);return h*60+m
17 def day(v):
18     if isinstance(v,datetime.datetime):return v.date()
19     if isinstance(v,datetime.date):return v
20     return datetime.date(*map(int,str(v).strip().split('-')))
21 def rows(path,sheet=None):
22     import openpyxl
23     w=openpyxl.load_workbook(path,read_only=True,data_only=True)
24     try:
25         s=w[sheet] if sheet else w.worksheets[0]
26         return s.title,list(s.values)
27     finally:w.close()
28 def locate(config,folder):
29     candidates=list(folder.rglob('*xlsx')) if folder.exists() else []
30     index={}
31     for f in candidates:index.setdefault(sha(f),[]).append(f)
32     found={};missing=[]
33     for r in config['external']:
34         choices=index.get(r['sha256'],[])
35         if not choices:missing.append(r['name']);continue
36         found[r['runtime']]=next((p for p in choices if p.name==r['name']),choices[0])
37     if missing:raise FileNotFoundError('请将官方附件放入指定原始附件目录；缺少或文件指纹不符：'+
    ↪ '.join(sorted(set(missing))))
38     return found
39 def prepare_inputs(config,folder,work):
40     located=locate(config,folder)
41     for rel,src in located.items():
42         dst=work/rel;dst.parent.mkdir(parents=True,exist_ok=True);shutil.copy2(src,dst)
43     inp=work/'输入'
44     a1=inp/'附件1_Q1典型日负荷与光伏.xlsx';a2=inp/'附件2_Q2全年负荷与光伏.xlsx';a3=inp/'附件3_Q3分时
    ↪ 发布光伏预报.xlsx';a4=inp/'附件4.xlsx'

```

```

45 s1,r1=rows(a1)
46 if len(r1)!=145 or [minute(r[0]) for r in r1[1:]]!=list(range(10,1441,10)):raise
    ↳ ValueError('附件1时间标签不完整')
47 q1=[dict(t=i,label_minute=(i+1)*10,start_minute=(i+1)*10,end_minute=(i+2)*10,price=number(r[
    ↳ 1]),load_kw=number(r[2]),pv_kw=number(r[3]),source_sheet=s1,source_row=i+2) for i,r in
    ↳ enumerate(r1[1:])]
48 _,load=rows(a2,'小区负载');_,pv=rows(a2,'光伏发电实际功率')
49 expected=[datetime.date(2025,1,1)+datetime.timedelta(days=i) for i in range(365)]
50 for table in [load,pv]:
51     if len(table)!=366 or [day(r[0]) for r in table[1:]]!=expected or [minute(x) for x in
        ↳ table[0][1:]]!=list(range(10,1441,10)):raise ValueError('附件2日期或时段不完整')
52 base=dict(config['metadata'],sources={f'attachment{i}':dict(path=p.name,sha256=sha(p)) for
    ↳ i,p in enumerate([a1,a2],1)},dates=[d.isoformat() for d in
    ↳ expected],q1_records=q1,load_kw=[[number(v) for v in r[1:]] for r in
    ↳ load[1:]],pv_kw=[[number(v) for v in r[1:]] for r in pv[1:]])
53 dump(work/'Q2_第5节独立代码/输入/inputs.json',base)
54 _,forecast=rows(a3);items=[];current=None
55 for i,r in enumerate(forecast[1:]):
56     if r[0]:current=day(r[0])
57     if current is None:raise ValueError('附件3首条记录缺少日期')
58     issue=datetime.datetime.combine(current,datetime.time())+datetime.timedelta(minutes=mini_
        ↳ te(r[1]))
59     items.append(dict(issue_time=issue.isoformat(sep='
        ↳ ',source_row=i+2,**{f'预报{j}小时':number(v) for j,v in enumerate(r[2:],1)}))
60 if len(items)!=1460:raise ValueError('附件3必须包含1460次发布')
61 base['sources']['attachment3']=dict(path=a3.name,sha256=sha(a3));base['forecasts']=items
62 dump(inp/'inputs.json',base)
63 _,price=rows(a4)
64 if [day(r[0]) for r in price[1:]]!=expected or [minute(x) for x in
    ↳ price[0][1:]]!=list(range(10,1441,10)):raise ValueError('附件4日期或时段不一致')
65 dump(inp/'电价.json',dict(actual_price=[[number(v) for v in r[1:]] for r in
    ↳ price[1:]],dates=[d.isoformat() for d in expected],source_sha256=sha(a4)))
66 checked=[]
67 for r in config['generated']:
68     d=json.loads(work/r['runtime']).read_text(encoding='utf-8'))
69     actual=hashlib.sha256(json.dumps(d,sort_keys=True,ensure_ascii=False,separators=(',',':')
    ↳ ).encode()).hexdigest()
70     if actual!=r['canonical_sha256']:raise ValueError('Excel转换与归档输入逐值不一致:
    ↳ '+r['runtime'])
71     checked.append(dict(file=r['runtime'],canonical_sha256=actual,passed=True))
72 dump(work/'输入转换核验.json',dict(passed=True,checks=checked,scope='路径字符串规范为相对文件名,
    ↳ 其他元数据和每个数值与原归档一致'))
73 return checked

```

## 4.26 运行论文结果.py

运行论文结果.py

```

1 """ 从官方附件生成论文数据；不读取历史结果。计算输出可分阶段接续。"""
2 from pathlib import Path
3 import argparse,datetime,hashlib,importlib.util,json,os,shutil,subprocess,sys
4 ROOT=Path(__file__).resolve().parent
5 os.environ['OMP_NUM_THREADS']='1';os.environ['OPENBLAS_NUM_THREADS']='1'
6 sys.dont_write_bytecode=True
7 def read(p):return json.loads(p.read_text(encoding='utf-8'))
8 def dump(p,x):p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(x,ensure_ascii=
    ↳ False,indent=2),encoding='utf-8')
9 def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
10 def call(script,*args,work):
11     cmd=[sys.executable,'-B','-X','utf8',str(script),*map(str,args)]
12     subprocess.run(cmd,cwd=work,check=True)
13 def source_hashes():return {p.relative_to(ROOT/'源码').as_posix():sha(p) for p in
    ↳ (ROOT/'源码').rglob('*') if p.is_file()}
14 def prepare(work,inputs):
15     from 输入转换 import prepare_inputs,locate
16     config=read(ROOT/'输入配置.json');located=locate(config,inputs)
17     hashes=source_hashes();state=work/'复现输入与源码.json'
18     if state.exists():
19         old=read(state)

```

```

20     if old['source_sha256']!=hashes or
        ↳ old['input_config_sha256']!=sha(ROOT/'输入配置.json'):raise
        ↳ ValueError('源码或输入协议已变更, 请指定新的 --output 目录')
21     if any(not (work/p).is_file() or sha(work/p)!=h for p,h in hashes.items()):raise
        ↳ ValueError('运行目录源码发生改变, 请指定新的 --output 目录')
22     return
23     if work.exists() and any(work.iterdir()):raise ValueError('新计算必须使用空输出目录')
24     shutil.copytree(ROOT/'源码',work,dirs_exist_ok=True)
25     prepare_inputs(config,inputs,work)
26     # Two additional templates are external official files, never bundled.
27     for name in ['result1.xlsx','result2.xlsx']:
28         source=next((p for p in inputs.rglob(name) if p.is_file()),None)
29         if source is None:raise FileNotFoundError('请提供官方空模板 '+name)
30         dest=work/'输出工具/官方模板'/name;dest.parent.mkdir(parents=True,exist_ok=True);shutil.co
        ↳ py2(source,dest)
31     dest=work/'Q2 前期敏感性/输入/inputs.json';dest.parent.mkdir(parents=True,exist_ok=True)
32     shutil.copy2(work/'Q2_第5节独立代码/输入/inputs.json',dest)
33     for i in [1,2,4]:
34         r=next(r for r in config['external'] if r['name']==f'附件 {i}.xlsx')
35         dest=work/f'正文绘图/原始附件/附件{i}.xlsx';dest.parent.mkdir(parents=True,exist_ok=True);
        ↳ shutil.copy2(located[r['runtime']],dest)
36     dump(state,dict(source_sha256=hashes,input_config_sha256=sha(ROOT/'输入配置.json'),created_at
        ↳ =datetime.datetime.now().astimezone().isoformat()))
37 def module(path,name):
38     spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spe
        ↳ c);spec.loader.exec_module(m);return
        ↳ m
39 def frozen(work,branch):
40     sys.path.insert(0,str(work/'代码'))
41     m=module(work/'代码/第5至7节_联合选型与正式回放.py','entry')
42     winner=read(ROOT/'输入配置.json')['frozen'][branch];old=winner['configuration'];now=json.load
        ↳ s(json.dumps(m.protocol(branch)))
43     env={'source_sha256','input_sha256','libraries','python'}
44     if any(old.get(k)!=now.get(k) for k in (set(old)|set(now))-env):raise
        ↳ ValueError('冻结参数的模型协议不匹配')
45     if old['source_sha256']!=now['source_sha256']:raise ValueError('冻结参数的数值源码不匹配')
46     winner['configuration']=now
47     dest=work/'本次复算'/branch/'一月选型/选定参数.json';dump(dest,winner);return dest
48 def completed(work,stage,fn):
49     marker=work/'阶段完成'/f'{stage}.json'
50     if marker.exists():print('已完成, 跳过: '+stage,flush=True);return
51     print('开始: '+stage,flush=True);fn();dump(marker,dict(completed=True,finished_at=datetime.da
        ↳ tetime.now().astimezone().isoformat()))
52 def run(stage,work,inputs,end_day):
53     prepare(work,inputs)
54     if stage=='prepare':return
55     if stage=='check':
56         import ast
57         for p in work.rglob('*.py'):ast.parse(p.read_text(encoding='utf-8-sig'))
58         sys.path.insert(0,str(work/'代码'));core=module(work/'代码/第4至7节_合同优化与储能执行.py',
        ↳ 'core')
59         data=core.UnifiedData();assert len(data.net)==52561
60         for branch in ['q3','q4_2','q4_3']:frozen(work,branch)
61         print('输入、源码及三问冻结参数检查通过',flush=True);return
62     if stage=='all':
63         if end_day!=365:raise ValueError('all 只允许完整年度')
64         for st in
        ↳ ['q1','q2','q3-select','q4_2-select','q4_3-select','q3-formal','q4_2-formal','q4_3-f
        ↳ ormal','q3-nodes','q4_3-nodes','q2-sensitivity','sensitivity','export','figures']:
65             run(st,work,inputs,end_day)
66         dump(work/'全部论文数据完成.json',dict(completed=True,scope='全部选型、正式期、节点对照、
        ↳ 敏感性、五份结果表、论文数据及统计绘图'))
67         return
68     if stage in ['q1','q2']:
69         source=work/('Q1_第4节独立代码/第4节_Q1运行与结果导出.py' if stage=='q1' else
        ↳ 'Q2_第5节独立代码/第5节_Q2运行与结果导出.py')
70         completed(work,stage,lambda:call(source,'--output',work/'本次复算'/stage,work=work));retu
        ↳ rn
71     if stage=='q2-sensitivity':
72         def q2sens():
73             p=work/'Q2 前期敏感性/Q2 敏感性测试.py'
74             output=subprocess.run([sys.executable,'-B','-X','utf8',str(p)],cwd=work,capture_outp
        ↳ ut=True,text=True,encoding='utf-8')

```

```

75         (p.parent/'运行日志.txt').write_text(output.stdout+'\n'+output.stderr,encoding='utf-8')
76         ↪ ' ')
77         if output.returncode:raise RuntimeError('Q2 敏感性失败, 见运行日志')
78         result=next(line.split('RESULT_JSON=',1)[1] for line in output.stdout.splitlines())
79         ↪ if line.startswith('RESULT_JSON='))
80         dump(p.parent/'Q2 六月敏感性结果.json',json.loads(result))
81         completed(work,stage,q2sens);return
82     if stage=='sensitivity':
83         completed(work,stage,lambda:call(work/'Q3_Q4敏感性/Q3_Q4敏感性测试.py','--q3-runtime','..'')
84         ↪ , '--q4-runtime','..'','--q4-2-runtime','..' ',work=work))
85         return
86     if stage=='export':
87         completed(work,stage,lambda:call(ROOT/'生成结果表与论文数据.py','--work',work,work=work));
88         ↪ return
89     if stage=='figures':
90         def plots():
91             srcs={
92                 '第 5.7 节 _Q2_ 一月 45 组候选评分.csv':work/'本次复算/q2/q2_candidate_summary.csv',
93                 '第 5.7 节 _Q2_ 一月胜出冻结参数.json':work/'本次复算/q2/q2_selected_config.json',
94                 '第 6 节 _q3_ 一月全部策略比较.csv':work/'本次复算/q3/一月选型/一月全部策略比较.csv',
95                 '第 6 节 _q3_ 选定参数.json':work/'本次复算/q3/一月选型/选定参数.json'}
96             for name,p in srcs.items():
97                 d=work/'正文绘图/结果数据'/name;d.parent.mkdir(exist_ok=True);shutil.copy2(p,d)
98             for f in ['compact_plot_source.py','selection_figures.py']:call(work/'正文绘图'/f,work=work)
99             ↪ k=work)
100         completed(work,stage,plots);return
101     branch,action=stage.rsplit('-',1)
102     if branch not in ['q3','q4_2','q4_3']:raise ValueError('未知阶段')
103     entry=work/'代码/第 5 至 7 节 _ 联合选型与正式回放.py'
104     if action=='select':
105         completed(work,stage,lambda:call(entry,branch,'select','--output',work/'本次复算'/branch/'
106         ↪ 一月选型',work=work))
107     elif action=='formal':
108         selection=work/'本次复算'/branch/'一月选型/选定参数.json'
109         if not selection.exists():selection=frozen(work,branch)
110         completed(work,stage+f'-{end_day}',lambda:call(entry,branch,'formal','--selection',selection,
111         ↪ '--output',work/'本次复算'/branch/'正式期','--end-day',end_day,work=work))
112     elif action=='nodes':
113         completed(work,stage,lambda:call(work/'辅助运行/第6至7节_固定模型的八节点价值对照.py',branch,
114         ↪ , '--run-root',work/'本次复算','--output-root',work/'节点价值对照',work=work))
115     else:raise ValueError('未知阶段')
116 def main():
117     p=argparse.ArgumentParser(description=__doc__);p.add_argument('stage',nargs='?',default='all')
118     ↪ ' ')
119     p.add_argument('--input-dir',type=Path,default=ROOT/'官方附件');p.add_argument('--output',type=
120     ↪ e=Path,default=ROOT/'计算输出');p.add_argument('--end-day',type=int,default=365)
121     a=p.parse_args();work=a.output.resolve();inputs=a.input_dir.resolve()
122     if work==ROOT or work.is_relative_to(ROOT/'源码'):raise ValueError('输出目录不可覆盖提交源码')
123     run(a.stage,work,inputs,a.end_day)
124     if __name__=='__main__':main()

```

## 4.27 requirements.txt

requirements.txt

```

1  numpy==2.5.1
2
3  scipy==1.18.0
4
5  scikit-learn==1.9.0
6
7  lightgbm==4.7.0
8
9  joblib==1.5.3
10
11 openpyxl==3.1.5
12
13 threadpoolctl
14

```

```

15
16
17 # Original verified environment; install using your local Python.
18
19
20 # 正文绘图
21 matplotlib>=3.9
22 pandas>=2.2

```

## 4.28 输入配置.json

输入配置.json

```

1  {
2    "metadata": {
3      "schema": "Q1-Q3-STARTPOINT-DELAY10-v3",
4      "time": {
5        "data_label": "interval_start",
6        "step_minutes": 10,
7        "decision_delay_slots": 1,
8        "first_new_interval": "00:10-00:20",
9        "last_contract_interval": "next day 00:00-00:10"
10     },
11     "initialization": {
12       "missing_interval": "2025-01-01 00:00-00:10",
13       "handling": "Typical-day midnight net-load proxy, matched purchase, no battery action. Not
14         ↪ an observed value.",
15       "last_observed_interval": "2026-01-01 00:00-00:10",
16       "observation_available": "interval_end"
17     },
18     "external": [
19       {
20         "name": "附件 1.xlsx",
21         "runtime": "Q1_第 4 节独立代码/数据/附件 1.xlsx",
22         "sha256": "66b87134f5ecccd68184d3539bb1293ef039f9e0fdd955a589b9bfa7f227c377"
23       },
24       {
25         "name": "附件 1.xlsx",
26         "runtime": "Q2_第 5 节独立代码/输入/附件 1_Q1 典型日负荷与光伏.xlsx",
27         "sha256": "66b87134f5ecccd68184d3539bb1293ef039f9e0fdd955a589b9bfa7f227c377"
28       },
29       {
30         "name": "附件 2.xlsx",
31         "runtime": "Q2_第 5 节独立代码/输入/附件 2_Q2 全年负荷与光伏.xlsx",
32         "sha256": "2e95fd446bfafa0d8c59577b5c2e2ea8b3f1def20dde54a3062556f4da9b4c72"
33       },
34       {
35         "name": "附件 1.xlsx",
36         "runtime": "输入/附件 1_Q1 典型日负荷与光伏.xlsx",
37         "sha256": "66b87134f5ecccd68184d3539bb1293ef039f9e0fdd955a589b9bfa7f227c377"
38       },
39       {
40         "name": "附件 2.xlsx",
41         "runtime": "输入/附件 2_Q2 全年负荷与光伏.xlsx",
42         "sha256": "2e95fd446bfafa0d8c59577b5c2e2ea8b3f1def20dde54a3062556f4da9b4c72"
43       },
44       {
45         "name": "附件 3.xlsx",
46         "runtime": "输入/附件 3_Q3 分时发布光伏预报.xlsx",
47         "sha256": "8a61b06c52bd0d639a1cc37c61a7d9f5b75edcbca718f64c1bd3498ec9f9d843"
48       },
49       {
50         "name": "附件 4.xlsx",
51         "runtime": "输入/附件 4.xlsx",
52         "sha256": "20e9c93aeab5e8e21ae4dd15587f9e190f7408692504c1319598461cd654fe71"
53       },
54       {
55         "name": "result3.xlsx",
56         "runtime": "输出工具/官方模板/result3.xlsx",

```

```

57     "sha256": "c59da470cabd0be23f602c95c8aa9d11ec224a0cdac216b3e1f218e65d006bdc"
58   },
59   {
60     "name": "result4-2.xlsx",
61     "runtime": " 输出工具/官方模板/result4-2.xlsx",
62     "sha256": "1c26494cfc6d754e0bd9bfff7e13e1126a73d2d2da6c5336eb251d89b9a1a1a47"
63   },
64   {
65     "name": "result4-3.xlsx",
66     "runtime": " 输出工具/官方模板/result4-3.xlsx",
67     "sha256": "c59da470cabd0be23f602c95c8aa9d11ec224a0cdac216b3e1f218e65d006bdc"
68   },
69   {
70     "name": "result1.xlsx",
71     "runtime": " 输出工具/官方模板/result1.xlsx",
72     "sha256": "28360e0974e7d6065394a8aba4e14a86773ae0036cc7d3ea1b211b515b03d688"
73   },
74   {
75     "name": "result2.xlsx",
76     "runtime": " 输出工具/官方模板/result2.xlsx",
77     "sha256": "1c26494cfc6d754e0bd9bfff7e13e1126a73d2d2da6c5336eb251d89b9a1a1a47"
78   }
79 ],
80 "generated": [
81   {
82     "runtime": "Q2_第5节独立代码/输入/inputs.json",
83     "canonical_sha256": "be2fcccc6456c57e3d7f509c02eec810bda393ba0e184cacce72d6c5b248a69eb"
84   },
85   {
86     "runtime": " 输入/inputs.json",
87     "canonical_sha256": "0bc8e82b5d66407004e0a8b46507cb5611136b4597f509c0b3d989af40d7de67"
88   },
89   {
90     "runtime": " 输入/电价.json",
91     "canonical_sha256": "c4becab910ca0f868ac1e3b80cc2a99ccb1edc181d4a169c20ae494646a2b4f6"
92   }
93 ],
94 "frozen": {
95   "q3": {
96     "branch": "q3",
97     "tau": 0.65,
98     "tau_index": 1,
99     "beta": 0.5,
100    "nodes": [
101      6,
102      12,
103      18
104    ],
105    "slots": 1584,
106    "cost_yuan": 588851.6761770495,
107    "contract_cost": 584876.7947996474,
108    "emergency_cost": 3974.8813774021546,
109    "emergency_kwh": 746.2038339362085,
110    "start_energy": 1200.0,
111    "end_energy": 8663.960890391756,
112    "throughput": 481110.53597851767,
113    "score_terminal": 0.33417,
114    "selection_score": 585956.9214768433,
115    "adjustments": 33,
116    "node_calls": 33,
117    "kind": "tree",
118    "alpha": 0.75,
119    "configuration": {
120      "schema": "Q1Q4_INHERITANCE_V3",
121      "branch": "q3",
122      "q3": "current-slot known; old initialization; load quantile minus PV; every-node fit;
123      ↳ direct LP; joint225x8",
124      "q4": "Q4-2 inherits frozen Q2 quantity model and midnight fit; Q4-3 inherits Q3 load
125      ↳ minus PV and every-node fit; both inherit current-slot-known quantities, warmup and
126      ↳ actual handoff inventory; price model daily from completed price history",
127      "execution": "all branches use approved current-slot-known quantity assumption; future
128      ↳ actual quantities and future actual prices never enter planning",
129      "horizon": 432,

```

```

126     "first_target": "origin+1",
127     "terminal": "eta*minimum planning price within432",
128     "january_score": "Jan20 00:10 through Jan31 00:10 exclusive; common terminal coefficient
    ↳ from Jan31 00:00 price forecast",
129     "warmup": "fixed typical-day positive net purchase, beta0; all branches retain marked
    ↳ initial quantity proxy; Q4 missing first price only leaves that warmup fee unbilled,
    ↳ no physical idling",
130     "kinds": [
131         "history",
132         "linear",
133         "tree"
134     ],
135     "taus": [
136         0.5,
137         0.65,
138         0.8,
139         0.9,
140         0.95
141     ],
142     "betas": [
143         0.0,
144         0.5,
145         1.0
146     ],
147     "alphas": [
148         0.0,
149         0.25,
150         0.5,
151         0.75,
152         1.0
153     ],
154     "node_sets": [
155         [],
156         [
157             6
158         ],
159         [
160             12
161         ],
162         [
163             18
164         ],
165         [
166             6,
167             12
168         ],
169         [
170             6,
171             18
172         ],
173         [
174             12,
175             18
176         ],
177         [
178             6,
179             12,
180             18
181         ]
182     ],
183     "score_tie": "within1e-6: emergency, fewer nodes, tau,beta,alpha,kind order,node-set
    ↳ order",
184     "source_sha256": {
185         "第2至3节_历史数据与储能基础.py":
    ↳ "bdd14f54974e6fb0dd37d9f0ecacabe0c48decee37fd15daa46cd3fd439ff3e4",
186         "第4至7节_合同优化与储能执行.py":
    ↳ "7ed2ef3e235287a1ff56c46976f2bef7459ba9dd41d67eecb5f9d905c4b301bb",
187         "第5至7节_联合选型与正式回放.py":
    ↳ "6502b192d365248c8b9efc668bbabdcf5c6b9bf28a9f64573e6c132ceb03e2a0",
188         "第6节_Q3逐节点负载与光伏预测.py":
    ↳ "245fbbaa8090f9fc2dc17abad654a16dbbf69dd6f68116ce89e99339e072a21",
189         "第7节_Q4每日负载与电价预测.py":
    ↳ "caa8b7784cc9e811719372ce759d91d8ec1b8cd645fe4a2d582c051c48709e04",

```

```

190     "第9节_独立校验.py": "00d6ff3153d10e71d04a60ca4971d49a17cf7996ab3745703143b59e786ae718"
191 },
192 "input_sha256": {
193     "inputs.json": "71f70d805760f1fde3dce8fa79202f225ca6821e4c7b46d6bdf0342f8fe808e0",
194     "电价.json": "868547745fd128aa24435d1848a335762eac238b2c8b4dc75816a7d9c672fa4f",
195     "附件1_Q1典型日负荷与光伏.xlsx":
196     ↪ "66b87134f5eccc68184d3539bb1293ef039f9e0fdd955a589b9bfa7f227c377",
197     "附件2_Q2全年负荷与光伏.xlsx":
198     ↪ "2e95fd446bfafa0d8c59577b5c2e2ea8b3f1def20dde54a3062556f4da9b4c72",
199     "附件3_Q3分时发布光伏预报.xlsx":
200     ↪ "8a61b06c52bd0d639a1cc37c61a7d9f5b75edcbca718f64c1bd3498ec9f9d843",
201     "附件4.xlsx": "20e9c93aeab5e8e21ae4dd15587f9e190f7408692504c1319598461cd654fe71"
202 },
203 "libraries": {
204     "numpy": "2.5.1",
205     "scipy": "1.18.0",
206     "sklearn": "1.9.0",
207     "lightgbm": "4.7.0",
208     "joblib": "1.5.3"
209 },
210 "python": "3.14.6"
211 },
212 "candidate_count": 1800,
213 "selection_period_only": "January",
214 "frozen_at": "2026-09-13T05:33:43.906829+08:00",
215 "calculation_provenance": {
216     "original_selection_sha256":
217     ↪ "0f9905348c3eeddb7eac10676c4d6ab5c96b62345437733fe91a6046b3cae0f4",
218     "original_configuration": {
219         "schema": "Q1Q4_Q3_LOAD_MINUS_PV_V2",
220         "branch": "q3",
221         "q3": "current-slot known; old initialization; load quantile minus PV; every-node fit;
222         ↪ direct LP; joint225x8",
223         "q4": "completed-slot only; idle missing first slot; daily model fit; Q4-2 net; Q4-3
224         ↪ load minus PV; direct LP",
225         "execution": "Q4 intra-interval feedback approximated by aggregate ten-minute net; no
226         ↪ interval truth enters contract planning",
227         "horizon": 432,
228         "first_target": "origin+1",
229         "terminal": "eta*minimum planning price within432",
230         "january_score": "Jan20 00:10 through Jan31 00:10 exclusive; common terminal
231         ↪ coefficient from Jan31 00:00 price forecast",
232         "warmup": "fixed typical-day positive net purchase, beta0; Q3 includes marked initial
233         ↪ proxy; Q4 slot0 idle unbilled",
234         "kinds": [
235             "history",
236             "linear",
237             "tree"
238         ],
239         "taus": [
240             0.5,
241             0.65,
242             0.8,
243             0.9,
244             0.95
245         ],
246         "betas": [
247             0.0,
248             0.5,
249             1.0
250         ],
251         "alphas": [
252             0.0,
253             0.25,
254             0.5,
255             0.75,
256             1.0
257         ],
258         "node_sets": [
259             [],
260             [
261                 6
262             ]
263         ]
264     }
265 }

```

```

254         [
255             12
256         ],
257         [
258             18
259         ],
260         [
261             6,
262             12
263         ],
264         [
265             6,
266             18
267         ],
268         [
269             12,
270             18
271         ],
272         [
273             6,
274             12,
275             18
276         ]
277     ],
278     "score_tie": "within1e-6: emergency, fewer nodes, tau,beta,alpha,kind order,node-set
↳ order",
279     "source_sha256": {
280         "第2至3节_历史数据与储能基础.py":
↳ "bdd14f54974e6fb0dd37d9f0ecacabe0c48decee37fd15daa46cd3fd439ff3e4",
281         "第4至7节_合同优化与储能执行.py":
↳ "7d7b77fa8237ca33a589ff44b89c0a44b2afdac5cc9d68f6bfa0e10cc37a2291",
282         "第5至7节_联合选型与正式回放.py":
↳ "af2a4dbac3543032c50ab64940b6bd6a69948ef8e4d298d574a63db0f0135d78",
283         "第6节_Q3逐节点负载与光伏预测.py":
↳ "245fbbaa8090f9fc2dcb17abad654a16dbbf69ddf6f8116ce89e99339e072a21",
284         "第7节_Q4每日负载与电价预测.py":
↳ "95ace0a490baae10fb6634e5dc342bb73173ba5f7389caaf3f525efddc588bf6",
285         "第9节_独立核验.py":
↳ "023ecd1c3291b62d33f64a311309f8f4f1e3b2fefe6d7f057e6709c36246a9e1"
286     },
287     "input_sha256": {
288         "inputs.json": "71f70d805760f1fde3dce8fa79202f225ca6821e4c7b46d6bdf0342f8fe808e0",
289         "电价.json": "868547745fd128aa24435d1848a335762eac238b2c8b4dc75816a7d9c672fa4f",
290         "附件1_Q1典型日负荷与光伏.xlsx":
↳ "66b87134f5ecccd68184d3539bb1293ef039f9e0fdd955a589b9bfa7f227c377",
291         "附件2_Q2全年负荷与光伏.xlsx":
↳ "2e95fd446bfafa0d8c59577b5c2e2ea8b3f1def20dde54a3062556f4da9b4c72",
292         "附件3_Q3分时发布光伏预报.xlsx":
↳ "8a61b06c52bd0d639a1cc37c61a7d9f5b75edcbca718f64c1bd3498ec9f9d843",
293         "附件 4.xlsx": "20e9c93aeab5e8e21ae4dd15587f9e190f7408692504c1319598461cd654fe71"
294     },
295     "libraries": {
296         "numpy": "2.5.1",
297         "scipy": "1.18.0",
298         "sklearn": "1.9.0",
299         "lightgbm": "4.7.0",
300         "joblib": "1.5.3"
301     },
302     "python": "3.14.6"
303 },
304     "equivalence_report": " 核验记录/Q3 协议兼容与全期等价核验.json",
305     "reason": "Q3 January grid ran under the load revision protocol. Final unified code has
↳ identical Q3 forecast bytes and specialized optimization/execution AST; full formal
↳ primary trace also matches exactly. Only configuration identity is rebased; all
↳ chosen values and the original freeze timestamp are preserved."
306 }
307 },
308 "q4_2": {
309     "branch": "q4_2",
310     "tau": 0.8,
311     "tau_index": 2,
312     "beta": 0.0,
313     "nodes": [],

```

```

314     "slots": 1584,
315     "cost_yuan": 690600.4775628813,
316     "contract_cost": 687546.0055050799,
317     "emergency_cost": 3054.4720578014103,
318     "emergency_kwh": 445.9638772530984,
319     "start_energy": 1200.0,
320     "end_energy": 2457.8550142535923,
321     "throughput": 443980.367081937,
322     "score_terminal": 0.2487249171689285,
323     "selection_score": 689989.5917584149,
324     "adjustments": 0,
325     "node_calls": 0,
326     "kind": "tree",
327     "alpha": 0.0,
328     "configuration": {
329         "schema": "Q1Q4_INHERITANCE_V3",
330         "branch": "q4_2",
331         "q3": "current-slot known; old initialization; load quantile minus PV; every-node fit;
332             ⇨ direct LP; joint225x8",
333         "q4": "Q4-2 inherits frozen Q2 quantity model and midnight fit; Q4-3 inherits Q3 load
334             ⇨ minus PV and every-node fit; both inherit current-slot-known quantities, warmup and
335             ⇨ actual handoff inventory; price model daily from completed price history",
336         "execution": "all branches use approved current-slot-known quantity assumption; future
337             ⇨ actual quantities and future actual prices never enter planning",
338         "horizon": 432,
339         "first_target": "origin+1",
340         "terminal": "eta*minimum planning price within432",
341         "january_score": "Jan20 00:10 through Jan31 00:10 exclusive; common terminal coefficient
342             ⇨ from Jan31 00:00 price forecast",
343         "warmup": "fixed typical-day positive net purchase, beta0; all branches retain marked
344             ⇨ initial quantity proxy; Q4 missing first price only leaves that warmup fee unbilled,
345             ⇨ no physical idling",
346         "kinds": [
347             "history",
348             "linear",
349             "tree"
350         ],
351         "taus": [
352             0.5,
353             0.65,
354             0.8,
355             0.9,
356             0.95
357         ],
358         "betas": [
359             0.0,
360             0.5,
361             1.0
362         ],
363         "alphas": [
364             0.0,
365             0.25,
366             0.5,
367             0.75,
368             1.0
369         ],
370         "node_sets": [
371             [],
372             [
373                 6
374             ],
375             [
376                 12
377             ],
378             [
379                 18
380             ],
381             [
382                 6,
383                 12
384             ],
385             [
386                 6,
387                 12
388             ]
389         ]
390     }

```

```

380         18
381     ],
382     [
383         12,
384         18
385     ],
386     [
387         6,
388         12,
389         18
390     ]
391 ],
392 "score_tie": "within1e-6: emergency, fewer nodes, tau,beta,alpha,kind order,node-set
↪ order",
393 "source_sha256": {
394     "第2至3节_历史数据与储能基础.py":
↪ "bdd14f54974e6fb0dd37d9f0ecacabe0c48decee37fd15daa46cd3fd439ff3e4",
395     "第4至7节_合同优化与储能执行.py":
↪ "7ed2ef3e235287a1ff56c46976f2bef7459ba9dd41d67eecb5f9d905c4b301bb",
396     "第5至7节_联合选型与正式回放.py":
↪ "6502b192d365248c8b9efc668bbabdcf5c6b9bf28a9f64573e6c132ceb03e2a0",
397     "第6节_Q3逐节点负载与光伏预测.py":
↪ "245fbbaa8090f9fc2dcb17abad654a16dbbf69ddf6f8116ce89e99339e072a21",
398     "第7节_Q4每日负载与电价预测.py":
↪ "caa8b7784cc9e811719372ce759d91d8ec1b8cd645fe4a2d582c051c48709e04",
399     "第9节_独立核验.py": "00d6ff3153d10e71d04a60ca4971d49a17cf7996ab3745703143b59e786ae718"
400 },
401 "input_sha256": {
402     "inputs.json": "71f70d805760f1fde3dce8fa79202f225ca6821e4c7b46d6bdf0342f8fe808e0",
403     "电价.json": "868547745fd128aa24435d1848a335762eac238b2c8b4dc75816a7d9c672fa4f",
404     "附件1_Q1典型日负荷与光伏.xlsx":
↪ "66b87134f5ecccd68184d3539bb1293ef039f9e0fdd955a589b9bfa7f227c377",
405     "附件2_Q2全年负荷与光伏.xlsx":
↪ "2e95fd446bfafa0d8c59577b5c2e2ea8b3f1def20dde54a3062556f4da9b4c72",
406     "附件3_Q3分时发布光伏预报.xlsx":
↪ "8a61b06c52bd0d639a1cc37c61a7d9f5b75edcbca718f64c1bd3498ec9f9d843",
407     "附件 4.xlsx": "20e9c93aeab5e8e21ae4dd15587f9e190f7408692504c1319598461cd654fe71"
408 },
409 "libraries": {
410     "numpy": "2.5.1",
411     "scipy": "1.18.0",
412     "sklearn": "1.9.0",
413     "lightgbm": "4.7.0",
414     "joblib": "1.5.3"
415 },
416 "python": "3.14.6"
417 },
418 "candidate_count": 45,
419 "selection_period_only": "January",
420 "frozen_at": "2026-09-13T05:45:05.421611+08:00"
421 },
422 "q4_3": {
423     "branch": "q4_3",
424     "tau": 0.65,
425     "tau_index": 1,
426     "beta": 0.0,
427     "nodes": [
428         6,
429         12,
430         18
431     ],
432     "slots": 1584,
433     "cost_yuan": 664903.1571775961,
434     "contract_cost": 659908.7688832248,
435     "emergency_cost": 4994.3882943712415,
436     "emergency_kwh": 742.9059302507345,
437     "start_energy": 1200.0,
438     "end_energy": 2215.276958465653,
439     "throughput": 462911.32266571664,
440     "score_terminal": 0.2487249171689285,
441     "selection_score": 664352.6255109182,
442     "adjustments": 33,
443     "node_calls": 33,

```

```

444     "kind": "tree",
445     "alpha": 0.75,
446     "configuration": {
447         "schema": "Q1Q4_INHERITANCE_V3",
448         "branch": "q4_3",
449         "q3": "current-slot known; old initialization; load quantile minus PV; every-node fit;
450             ⇨ direct LP; joint225x8",
451         "q4": "Q4-2 inherits frozen Q2 quantity model and midnight fit; Q4-3 inherits Q3 load
452             ⇨ minus PV and every-node fit; both inherit current-slot-known quantities, warmup and
453             ⇨ actual handoff inventory; price model daily from completed price history",
454         "execution": "all branches use approved current-slot-known quantity assumption; future
455             ⇨ actual quantities and future actual prices never enter planning",
456         "horizon": 432,
457         "first_target": "origin+1",
458         "terminal": "eta*minimum planning price within432",
459         "january_score": "Jan20 00:10 through Jan31 00:10 exclusive; common terminal coefficient
460             ⇨ from Jan31 00:00 price forecast",
461         "warmup": "fixed typical-day positive net purchase, beta0; all branches retain marked
462             ⇨ initial quantity proxy; Q4 missing first price only leaves that warmup fee unbilled,
463             ⇨ no physical idling",
464         "kinds": [
465             "history",
466             "linear",
467             "tree"
468         ],
469         "taus": [
470             0.5,
471             0.65,
472             0.8,
473             0.9,
474             0.95
475         ],
476         "betas": [
477             0.0,
478             0.5,
479             1.0
480         ],
481         "alphas": [
482             0.0,
483             0.25,
484             0.5,
485             0.75,
486             1.0
487         ],
488         "node_sets": [
489             [],
490             [
491                 6
492             ],
493             [
494                 12
495             ],
496             [
497                 18
498             ],
499             [
500                 6,
501                 12
502             ],
503             [
504                 6,
505                 18
506             ],
507             [
508                 12,
509                 18
510             ]
511         ]
512     }
513 }

```

```

510     "score_tie": "within1e-6: emergency, fewer nodes, tau,beta,alpha,kind order,node-set
511     ↪ order",
512     "source_sha256": {
513         "第2至3节_历史数据与储能基础.py":
514         ↪ "bdd14f54974e6fb0dd37d9f0ecacabe0c48decee37fd15daa46cd3fd439ff3e4",
515         "第4至7节_合同优化与储能执行.py":
516         ↪ "7ed2ef3e235287a1ff56c46976f2bef7459ba9dd41d67eecb5f9d905c4b301bb",
517         "第5至7节_联合选型与正式回放.py":
518         ↪ "6502b192d365248c8b9efc668bbabdcf5c6b9bf28a9f64573e6c132ceb03e2a0",
519         "第6节_Q3逐节点负载与光伏预测.py":
520         ↪ "245fbbaa8090f9fc2dcb17abad654a16dbbf69ddf6f8116ce89e99339e072a21",
521         "第7节_Q4每日负载与电价预测.py":
522         ↪ "caa8b7784cc9e811719372ce759d91d8ec1b8cd645fe4a2d582c051c48709e04",
523         "第9节_独立核检.py": "00d6ff3153d10e71d04a60ca4971d49a17cf7996ab3745703143b59e786ae718"
524     },
525     "input_sha256": {
526         "inputs.json": "71f70d805760f1fde3dce8fa79202f225ca6821e4c7b46d6bdf0342f8fe808e0",
527         "电价.json": "868547745fd128aa24435d1848a335762eac238b2c8b4dc75816a7d9c672fa4f",
528         "附件1_Q1典型日负荷与光伏.xlsx":
529         ↪ "66b87134f5eccccd68184d3539bb1293ef039f9e0fdd955a589b9bfa7f227c377",
530         "附件2_Q2全年负荷与光伏.xlsx":
531         ↪ "2e95fd446bfafa0d8c59577b5c2e2ea8b3f1def20dde54a3062556f4da9b4c72",
532         "附件3_Q3分时发布光伏预报.xlsx":
533         ↪ "8a61b06c52bd0d639a1cc37c61a7d9f5b75edcbca718f64c1bd3498ec9f9d843",
534         "附件4.xlsx": "20e9c93aeab5e8e21ae4dd15587f9e190f7408692504c1319598461cd654fe71"
535     },
536     "libraries": {
537         "numpy": "2.5.1",
538         "scipy": "1.18.0",
539         "sklearn": "1.9.0",
540         "lightgbm": "4.7.0",
541         "joblib": "1.5.3"
542     },
543     "python": "3.14.6"
544 },
545 "candidate_count": 1800,
546 "selection_period_only": "January",
547 "frozen_at": "2026-09-13T05:45:05.661118+08:00"
548 }
549 }
550 }

```